MODUS ASCII Command Reference

ModuSystems / Douloi Motion Controllers

Converted from AsciiHelp.chm
Complete help system in document form

Table of Contents

Introduction 8
- Welcome 8
- Methods of Use 8
- Communication Matrix 9
- Version Information 10
- Bug Reporting 18

Setting Up the Controller 19
- Controller Preparation 19
- Choosing an IP Address 20
- First Motion 25
- Reading Digital Inputs 28

Controller Description 33
- Specifications 33
  - Communication Ports 33
  - Controller Power 33
  - Inputs and Outputs 33
  - LED Indicators 34
  - Motion System 34
- Mechanical 35
  - Mechanical 35
- Connectors 36
  - Power Connector 36
  - Motor Connector 37
  - Encoder Connector 37
  - IO Connector 37
  - EStop Connector 38
  - Serial Connector 38
  - Intermodule Plug Connector 39
- Options 39
  - 3ENC Option 39
  - 3PD Option 40
  - IOH and IOL Option 44

6ABS Option 44
Menu Reference 47
File 47
New Project 47
Open Project 48
Open Recents 48
Open Resources 49
Save 50
Save As 51
Deploys With Console 51
Save In Controller 52
View 54
Pascal Listing 54
Hints 55
Task Monitor 57
Assembly Listing 58
Symbol Tree 61
Configure 62
Communication Port 63
Auto Drop Down 64
Data Drop Down 65
Compile To Flash 68
Conditional Defines 70
Remote Network 73
Run 74
Start App 74
Stop App 75
Help 77
Block Help 77
Text Help 78
About 78
Protocol Description 79
Port Configuration 79
Procedure Message Format 80
Object Message Format 82

Command Reference 85
Motion 85
Abort 85
Accel 86
Actual Position 87
Arm Capture 88
Begin Move By 89
Begin Move To 91
Begin Stop 93
Capture Bit 94
Capture Has Tripped 95
Capture Position 96
Commanded Position 97
Coordinate Inversion 98
Counts Per User Unit 99
Decel 100
Destination Position 101
Initialize Group 102
Jog 104
Motor 105
Move Is Finished 106
Negative Limit 107
Positive Limit 108
Profile Phase 109
Profile Velocity 111
Set Capture Source 113
Set Capture Trip 114
Set Current To 114
Speed 115
Multitasking 117
User Booleans 117
User Longints 118
User Numbers 118
Abort User Task 119
Begin User Task 122

User Task Present 123
IO Operations 124
Configure IO Bit As Output 124
Input Bit 125
Sample Counter 126
Sample Rate 127
Set Output Bit 127
Safety 128
Reset Watchdog 128
User Has Disabled 129
Watchdog Has Tripped 130
Exception Handling 131
Math 131
Console Controls 131
Motion Concepts 132
User Units 132
Motor States 134
Homing 134
Escape Codes 136
10 Divide Overflow Escape Code 136
11 Sample Overrun Escape Code 136
12 Axis Not Valid Escape Code 136
13 Speed Negative Escape Code 136
15 Curve Buffer Not Linked Escape Code 137
16 User Accels 0 Or Negative Escape Code 137
17 User Decells 0 Or Negative Escape Code 137
19 Unable To Resume Task Escape Code 138
20 Unable To Begin Task Escape Code 138
21 Append Move Overflow Escape Code 138
23 Motion Overrun Escape Code 138
24 Axis Is Busy Escape Code 139
25 File Reset Escape Code 139
26 File Rewrite Escape Code 139
27 Read Escape Code 139
28 Write Escape Code 140

29 Object Not Initialized Escape Code 140
31 Parameter Out Of Range Escape Code 140
32 Watchdog Failed To Reset Escape Code 140
33 Option Not Present Escape Code 141
34 IO Authorization Escape Code 141
35 Name Not Found Escape Code 141
36 Library Not Found Escape Code 142
37 DLL Procedure Not Found Escape Code 142
38 Neighbor Communication Escape Code 142
39 String Error Escape Code 143
40 Floating Point Error Escape Code 143
41 Task Stack Overflow Escape Code 143
42 Address Is Nil Escape Code 144
44 Plate Not Present 144
45 Position Limit Escape Code 144
46 Arm Capture Failure Escape Code 144
47 Task Address Is Not Valid Escape Code 145
48 Bitmap File Not Found Escape Code 145
49 Capture Not Present Escape Code 145
50 Encoder Position Not Present Escape Code 145
51 Dac Number Out Of Range Escape Code 146
52 Function Not Present Escape Code 146
53 Axis Does Not Support Enable Status Escape Code 146
54 Firmware Read Escape Code 146
55 Ethernet Socket Initialization Failed Escape Code 147
56 Ethernet Socket Initialized With Port 0 Escape Code 147
57 Ethernet Socket Already In Use Escape Code 147
58 Ethernet Socket Limit Exceeded Escape Code 147
59 Modbus Communication Package Required Escape Code 148
60 Watchdog Has Tripped Escape Code 149
61 Unresponsive Remote Controller Escape Code 149
1001 Grammar Escape Code 149
1002 Unknown Command 150
1003 Axis Number Out Of Bounds 150
1004 Group Number Out Of Bounds 150

1005 Illegal Dimension 150
1006 Group Not Initialized 151
1007 Coordinated Group Required 151
1008 Too Few Parameters 151
1009 Too Many Parameters 151
1010 Assertion 152
1011 Out Of Curve Buffers 152
1012 Unable To Find Named Procedure 152
Deployment 154
Commissioning 154
Updating Firmware 155

Introduction

Welcome

Welcome to Modusystems Controllers and Snap2Motion Software, tools to simplify and accelerate the creation of motion control applications. Modusystems wants to encourage your project's success. Free technical support is available to answer your questions, assist you through trouble-spots in product use, and to recommend strategies and approaches for solving different aspects of a motion control problem. Sample code, application prototypes, and application notes can be provided to respond to specific questions you may have. We would much rather have you call and get answers than to be frustrated or slowed in your automation project. Please feel free to contact us at:

Voice: (408) 708-5883

EMail: support@Modusystems.com

Additional information is also available at the web site:

www.Modusystems.com

Methods of Use

Motion can be directed in a variety of ways. Programs can run directly on the controller itself. Motion can also be directed from a PC or PLC. Consider the following methods in light of your system objectives. All of these methods support directing motion, manipulating IO, sharing various types of data between a host and the controller, and managing resident tasks that have been placed in the controller. In order to provide the most relevant information with the least amount of clutter there is a seperate manual for each product and method-of-use combination.

Snap2Motion

Snap2Motion is used to describe controller-resident programs using a combination of block graphical programming and text oriented programming. Many applications can be directed by the controller itself and have no need of host PC or PLC. Programs written in Snap2Motion are compiled and run in the native object code of the controller's processor. This makes application programs extremely fast operating at command rates of several Mhz. Even if a PC or PLC is used to direct the controller it is likely that Snap2Motion will still be used to delegate to the controller those time-critical tasks that cannot be effectively handled by the host. Snap2Motion has a block programming manual for ease of use as well as a more advanced text programming manual.

ASCII Commands

This method uses 3 letter commands to direct the controller. Ascii Commands can be sent by a host computer or manually from a terminal emulator acting as a host for convient testing and problem isolation. This is a reactive interpreter with a Hash Table architecture implemented as a Snap2Motion application. This interpreter can be extended with new commands or tailored to meet application specific requirements. Ascii commands can be sent over RS232, RS485, or TCP/IP.

Binary Commands

This method uses small and efficient binary data packets to perform controller operations. This protocol can support multiple controllers over one communication channel using either node or resource based addressing. Self-addressing ring topologies and node-addressed multidrop topologies are supported based on the communication method chosen. This method employs a Windows DLL. The controller-side is implemented as a Snap2Motion application this can be modified to meet application specific needs.

Serial Commands

For operating systems other than Windows where serial communication is desired this method explains the low-level packet details necessary to direct the controller.

Modbus Slave

The popular Modbus protocol can be used to direct the controller through a mapping named "Modbus Motion Interface" (MMI). Modbus register operations direct the controller and report status usually with a single transaction. Modbus Slave protocols over RS232, RS485 and TCP/IP are supported allowing the controller to be directed through third party Modbus libraries available on a variety of operating systems and languages.

MotionLib32

This method permits a host PC to compile and download programs into the controller. Beyond resident programs that are saved as part of the project development, MotionLib32 allows a host PC to write a program, place it in the controller, and direct it for both maximum flexibility and maximum performance. This is a more advanced method best used when both flexibility and performance are required. This is the back end cross-compiler used by Snap2Motion.

External Signals

In some cases a master device, such as a PLC, may want to convey motion intent with electrical signals. Such signals could be step/dir signals that the controller is intended to follow or could be signals requesting a certain resident behavior be performed. This category contains various ways external signals can influence the controller.

Custom Application

Requirements such as non-reactive protocols, tagged concurrent commands, and protocol impersonation for system integration convenience can be built as application programs to meet project specific needs.

Communication Matrix

Communication Matrix

The following matrix indicates which communication transports are available for the various methods of use:

	RS232	RS485	TCP/IP	USB
Snap2Motion	✓			✓
Ascii Commands	✓	✓	✓	✓
Binary Commands	✓	✓		✓
Serial Commands	✓	✓		✓
Modbus Slave	✓	✓	✓	✓
MotionLib32	✓			✓
Custom Application	✓	✓	✓	✓

✓ Indicates this method is available in standard firmware.

✓ Indicates a port-swapping version of firmware is required.

Version Information

Description

The current Snap2Motion Software Version can be seen in the Help | About dialog. This numeric version number relates to the executable files for the installation. A suffix letter is associated with the installation file reflecting updates to the installation that are not executables, such as additional examples or documentation changes. Support files such as standard.inc or modbus master files may be changed under suffix letter updates. Check the list below to see if a particular concern has been addressed or contact Dings Motion directly.

Version 2.23 - 20251122

Network Detection on Setup Tab now shows more specific axis types on remote controllers. Corrections made to configuration of remote axes.

"Detect" button caption changed to "Detect Network" indicating that the primary use of this button is to find and present networked resources.

Remote Network Filename Dialog now displays the filepath in a multiline manner so entire path can be viewed.

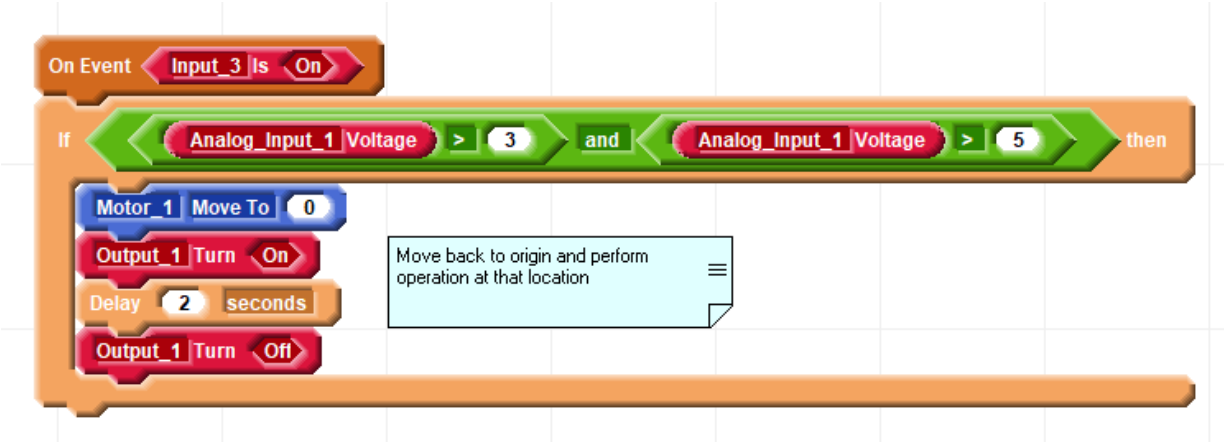
Version 2.22 - 20251110

Axis configuration on the Setup Tab now allows for choosing a configuration software package. This package permits further application level defined configuration properties through blocks.

Version 2.21 - 20251102

Comments can now be associated with blocks so that they travel with block movement. Additional information can be found under Menu Reference | Workpage Menu | Add Comment

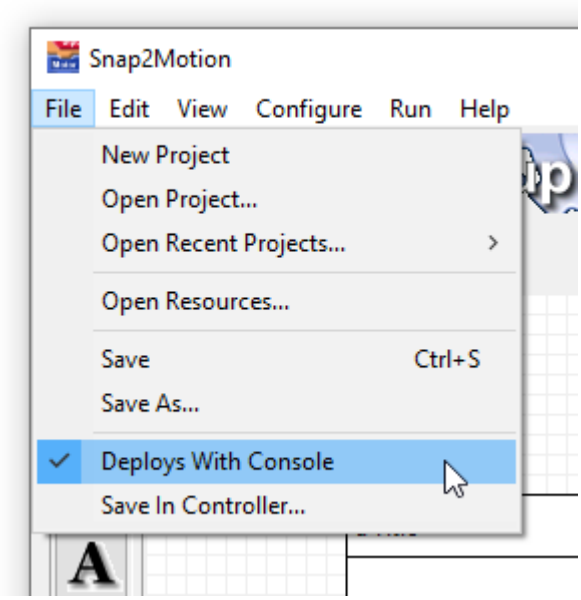
Space under broad expressions is now available for comments and other block lists. Previously items placed in this region would bounce to the right.



Improved the message indicating that a project cannot be saved in the installation directory.

Version 2.20 - 20250816

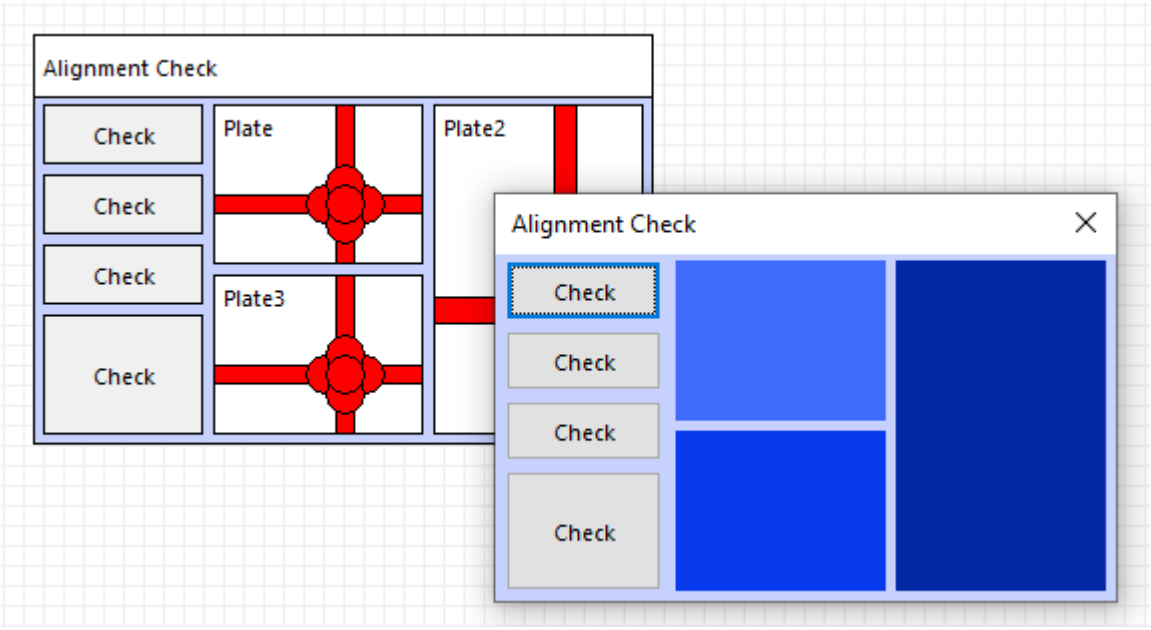
The setting "Deploys with Console" and related deployment procedure allows Console applications to start up almost immediately without the previous startup delay.



Details are provided in the Help documentation under Menu Reference | Main Menu | File | Deploys With Console

The application Window resulting from starting a Snap2Motion project now behaves more like an independent application rather than as a child to Snap2Motion. After starting an application Snap2Motion can be minimized and the application will continue to run. The task bar will show two Snap2Motion icons, one for the development environment and one for the application. Either application can be on top of the other.

Alignment between the drawn application and the running application is closer.



Version 2.19 - 20250711

Automatic controller detection based on the USB virtual port name has been changed requiring that "USB" be explicitly selected in order for a USB search to be performed. This selection is made in the menu selection "Configure | Communication Port" and select "USB".

Version 2.18 - 20240925

Automatic controller detection based on the USB virtual port name has been restored.

File directories presented during Open Resources, Open Projrect, and Save are improved.

Plate editors can now be accessed by right clicking on the Console object list and selecting "Edit Selected Object". An Object Filter may also be selected from the same menu to reduce the number of objects shown for more convenient navigation.

Version 2.17 - 20240811

Now recognizes sub-version codes for OEM-2T to properly establish compiler conditionals for each form. ct_OEM_2T = 1 corresponds to the 1 MByte Flash 80 MHz version. ct_OEM_2T = 2 corresponds to the 512 MByte 80 MHz version, and ct_OEM_2T = 3 corresponds to the 512 MByte 71.99 MHz version running from the microcontroller's internal clock to accomodate an external spread spectrum oscillator for reduced EMI.

Includes 460800 baud network communication speeds for OEM_2T sub-versions 2 and 3 allowing for better command response times between controllers.

Snap2Motion is now built with a more modern toolchain that produces Snap2Motion with a flatter presentation style consistent with more recent Windows operating systems.

Plates are now opaque and show their final color on the Console tab.

Version 2.16 - 20231129

Correction to skip evaluation of nested compiler conditionals.

Version 2.15 - 20231015

Support for web viewers in plates, factory special software configuration.

Version 2.14 - 20230722

Permit constants to be used in describing absolute variables.

Version 2.13 - 20230613

Modifications to support ATSAME70 based controllers.

Version 2.12 - 20230314

Using the Remote Network Selection dialog now compels Snap2Motion to incrementally recompile the network component. A restart of Snap2Motion is no longer required to use an alternate network file.

The "Change By" block in the data area now operates for the OEM-2T configuration.

For the OEM-2T configuration there was a startup sequence that would cause the controller to not respond to a "Motor On" command. This has been corrected.

Version 2.11 - 20230306

Snap2Motion now evaluates the extent of peristent data storage and writes compiler directives to adjust data structures in the Flash Manager. This makes the persistent data structure extent an evaluated and automatically generated property of the application rather than a settting in standard.inc.

Software packages that are check marked as not included in the project no longer appear in the Block Tab's More Blocks list. This removes the ambiguity of which package is which when there are number interchangeable packages with the same name but only one of which is active at a time.

Version 2.10 - 20230302

Compiler changes to support flash memory use in 512VDC variant of the OEM-2T. Changes made to OEM-2T standard.inc reflecting new memory layout.

Version 2.09 - 20230212

Microcontroller variant supported for OEM_2T. Compile time conditional ct_OEM_2T evaluates to 1 for original microcontroller and to 2 for 512VDC variant. Differences include clock rate, communication properties and flash storage. The factory standard file references the ct_OEM_2T compiler conditional to include appropriate software elements.

Version 2.08 - 20220117

Compare signal support restored for MAC controllers.

Version 2.07 - 20220117

The block find navigator has been changed to always stay on top.

Version 2.06 - 20211229

Detection support for networked controllers has been extended. Use the Detect button the setup page to identify controllers and related axis IO resources for allocation.

Version 2.05 - 20211030

Paste menu items on local menus are disabled until a Cut or Copy operation has been performed to provide something to paste.

A Block Find feature has been provided to search a project for a specific type of block. Details of use can be found in Find Menu Item .

Version 2.04 - 20211011

Improvements have been made to the Detect button on the setup page for the OEM-2T family of controllers. Networked controllers now report their resources to Snap2Motion for use in setting up named resources and axes.

Color selection for plates is improved to allow specifying a Windows systems color or a specific color. Any custom colors defined while working on a project are saved as part of the project.

In specific situations, such as including a network file into the project, the Compile-To-Flash property will be configured by Snap2Motion automatically.

Support has been provided for identifying the OEM-IO.

Version 2.03 - 20210804

Added TPlate.SaveImageToFilename command allowing an image drawn by Snap2Motion to be saved. The Block version is in the green graphics category as "Save Image"

In the block use a File Save dialog is produced to identify the name for the file.

Disappearing expression problem solved relating to placement of expressions in assignment statements.

Block Copy/Paste operation corrected for multiple block copies.

Issue with block expression cutting was corrected.

Problem related to resizing the application in Console mode, changing to the Block Tab, and then altering the application size was corrected.

Some of the example programs have been modified to have an AutoTest procedure expressed in text programming. This procedure simulates human interaction with the example program for the purpose of regression testing. It's not important to inspect or understand the Autotest procedure to benefit from the example.

Version 2.02 - 20210513

Modbus_Communication software package for Modbus RTU updated for persistent modbus objects and selection for identifying the port. After placing the component into an application and running compilation will stop requesting a choice be made about which ports should be used. After selecting a port save the project and and press "Play" again.

Standard.inc files now have controller family compiler directives to aid in sharing more software components across controller families.

AutoTest command line parameter available to begin an AutoTest procedure, if present, in an application. This feature is mainly used for in-house regression test procedures.

Version 2.01 - 20210320

Output readback status corrected for MMC.

Copy | Paste operations on nested block expressions corrected.

An unnecessary yield was removed from the else condition of block-described if..else statements.

Closed Loop Software Packages for MMC-3T-3ENC and MMC-T are provided.

An MMC Modbus Master Software Package for use with RS232 and RS485 is available.

Conditional compilation now supported to multiple levels. Value based conditional compilation with comparisons now supported for constant symbols.

Multiply operations on literal integer constants larger than 16 bits is now properly handled.

Version 2.00 - 20210307

Offline Programming - If a controller is not found a choice is offered to work offline. Offline Programming allows for compiling the project to confirm there are not syntax mistakes. The controller type is chosen manually allowing for various product types and related resources to be available even if that particular controller is not connected. Offline Programming does not permit execution or any console interaction. To connect to a controller exit Snap2Motion and restart with the controller attached.

Remote Network Selection - A choice can now be made regarding the type of remote network including Animatics Smartmotors or the Modusystems remote controller protocol. If the network type is not know a dialog is shown to make a choice. An alternative choice can be made selecting Configure | Advanced | Remote Networks.

Plate properties have been changed to support multiple screens. Child plate properties now include "Is Initially Shown" as well as "Is Attached". This allows a Snap2Motion project to have attached, but hidden, plates that can appear when popped up such as tabbed form HMIs. The Grahics Block category now provides plate Pop Up and Close commands to support screen manipulation. Refer to the Tabbed Screen example and the Popup Screen example in the Blocks example directory.

Version 1.98 - 20201201

There is now a network protocol file choice when detecting remote controllers. The choice can also be made from the menu choice Configuration | Advanced | Remote Network.

Improvements made in presentation of remote controller resources in Setup dialogs

Version 1.97 - 20201122

Changes made to insure nonconflicting persistent indexes.

Version 1.95 - 20200528

Corrections made to the path in front of some blocks that are located in different packages than the block list.

Correction made to prevent unincluded packages from providing block list content.

Correction to prevent inclusion of event blocks that are on the palette instead of the workpage.

Version 1.94 - 20200402

Saving a project retains the current view and location within the project so when the project is opened that view is restored. This permits have a project open to a specific place, such as a configuration setting block list, to make it easier to access that list.

Version 1.93c - 20200327

Refined Ascii Command documentation.

Digital IO options IOH and IOL are described.

Communication Matrix tables included explaining what protocols are available on which communication methods.

Instructions provided for identifying an IP address

Version 1.93 - 20200319

Added Ascii Commands as a Method Of Use and related preliminary related documentation.

Added Joystick Axis Software Package and related joystick wiring information.

Added a more general method of providing list text and statements when Blocks are being constructed from text described procedures and functions.

Corrected redundant component copying when plates are selected and copied in a project.

Version 1.92 - 20200310

Improvements were made to form popup and attachment behaviors. Child forms remain on top of parent forms. Closing a form closes related child forms whether they are popup or attached. AutoUpdate routines maintaining display controls on child forms are terminated prior to child forms closing. If a form is asked to close the CanClose routines in related child forms are also called to insure all forms that will be closed have the opportunity to block the closure.

The Task Monitor remains in front of the Snap2Motion application and does not sink behind.

Single line type definition aliases are now supported for singles and doubles allowing for late-binding of numerical resolution. In conjunction with Conditional Defines this permits selecting numerical resolution at compile time.

Version 1.91 - 20200301

Included IMC family of controllers.

Resolved incorrect "Insufficient Memory" notification

Version 1.90 - 20200225

Plates that were not included would still show their controls. This has been corrected so that those controls are concealed.

Version 1.89b - 20200217

Additional Help dialog, which appears when reporting an error and suggesting documentation to read, now stays in front of Snap2Motion and does not sink behind.

Version 1.89 - 20200217

This "unified" version of Snap2Motion supports all of Modusystems controller families. Current version MAC controllers and all others are supported with USB. Older MAC controllers and MMC family controllers that have swapped the Snap2Motion port for the host RS232 aux port are supported over RS232 with a port selection choice in the Configure menu. If Snap2Motion cannot identify the type of controller connected, because of older firmware, a dialog will be shown requesting the user to identify the controller type. Updated firmware answers this question automatically.

Corrections made to properly abort if controller communication cannot be established

Checks are now made to insure a language reserved word is not used for an object name.

Corrections made in plate form alignment so that an attached plate is constructed at the proper location on it's pop-up parent form.

There is a Menu Reference Chapter explaining all of the menu choices

Bug Reporting

Snap2Motion Bug Notification Process

All projects operate on tight schedules and it is hoped that a Snap2Motion bug does not interfere with progress. If a problem is encountered with Snap2Motion please screen capture any error messages and email the screen capture, problem descripition and the project demonstrating the issue as an attachment, to:

Support@Modusystems.com

A workaround or problem resolution will be offered to keep the project on track.

Setting Up the Controller

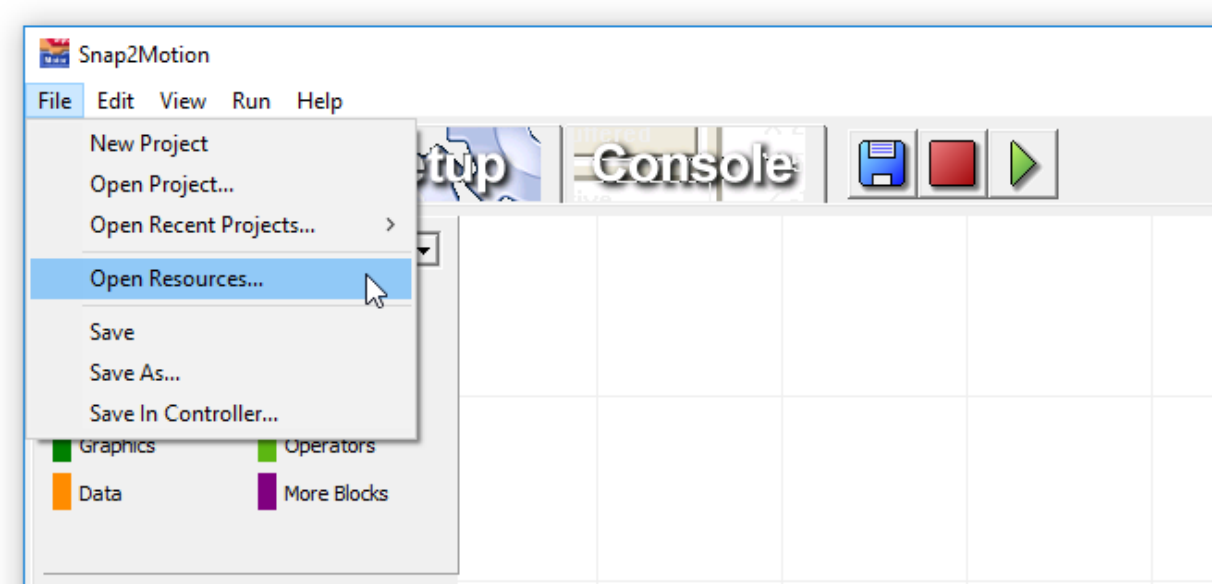
Setup Introduction

The following pages describe how to configure the controller for a specific method of use, how to find an IP address if one is needed, preliminary wiring for first motion and the use of IO.

Controller Preparation

Controller Preparation for a Specific Method of Use

For methods of use besides Snap2Motion and MotionLib it is necessary to place in the controller an application program that responds to that method. From a blank Snap2Motion project click the green "Play" button and run an empty application. This establishes a connection and Snap2Motion is aware of what controller it is connected to. Once _SQW recognizes the controller the resources it provides will be specific to that controller. After connecting select "File | Open Resources..."

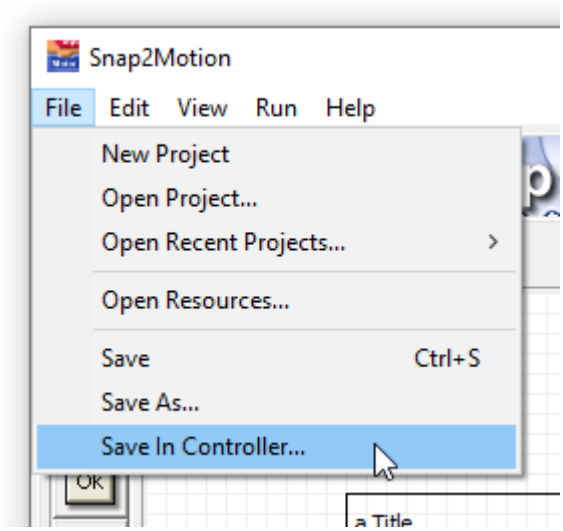


Open the "Methods Of Use" folder and identify the method you would like to use. In some cases a single project contains a number of transport alternatives, such as Modbus supporting Ethernet TCP/IP as well as serial RTU. Examine the opening page of the project for additional information regarding application configuration.

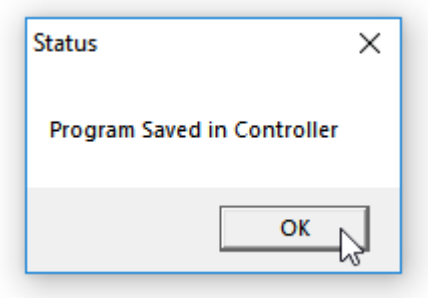
After opening the project press the "Play" button:"

After the status to the right of "Play" indicates "Running" the project is ready to be saved. Press the red stop button:

From the file menu select "Save In Controller..." to make the program resident and the default program that runs when the controller is power cycled.



This process should conclude by confirmation that the save is complete:



Power down the controller and prepare for wiring.

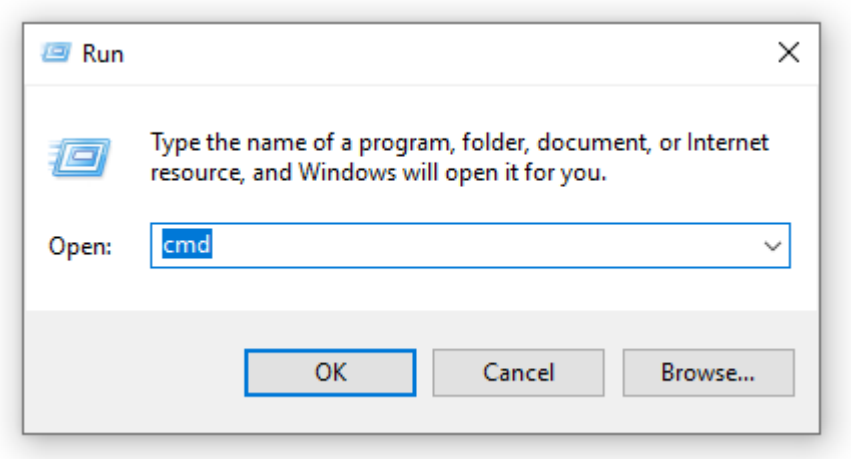
Choosing an IP Address

How To Identify a Compatible IP Address for TCP/IP Methods

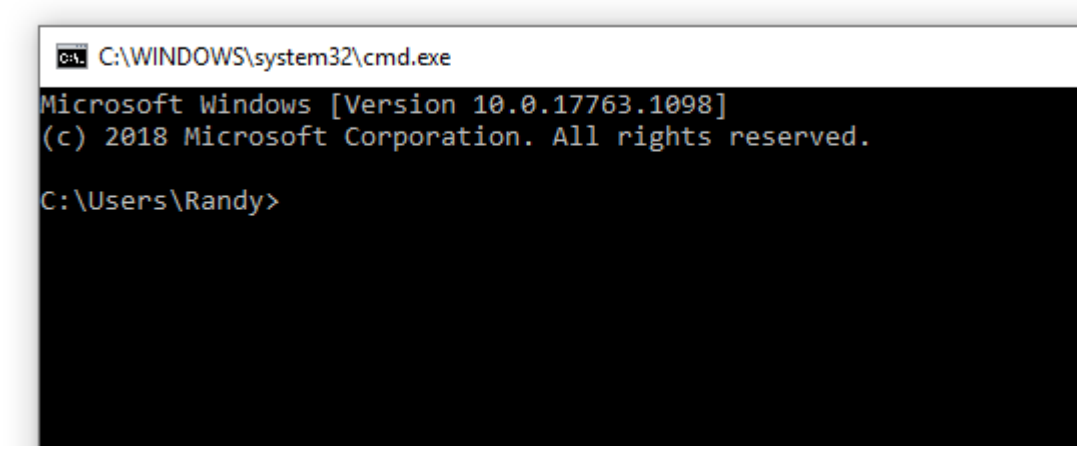
Frustration sometimes associated with using an Ethernet connection can be avoided by taking a series of small steps that verify incremental progress towards identifying an IP address and an effective connection. These instructions pertain to connecting a PC to a controller over Ethernet but are also useful for other types of Ethernet connections such as connecting an HMI to a controller with Ethernet.

Identify your PC IP Address

Right click on the Windows Icon in the lower left, select "Run" and type in "CMD":

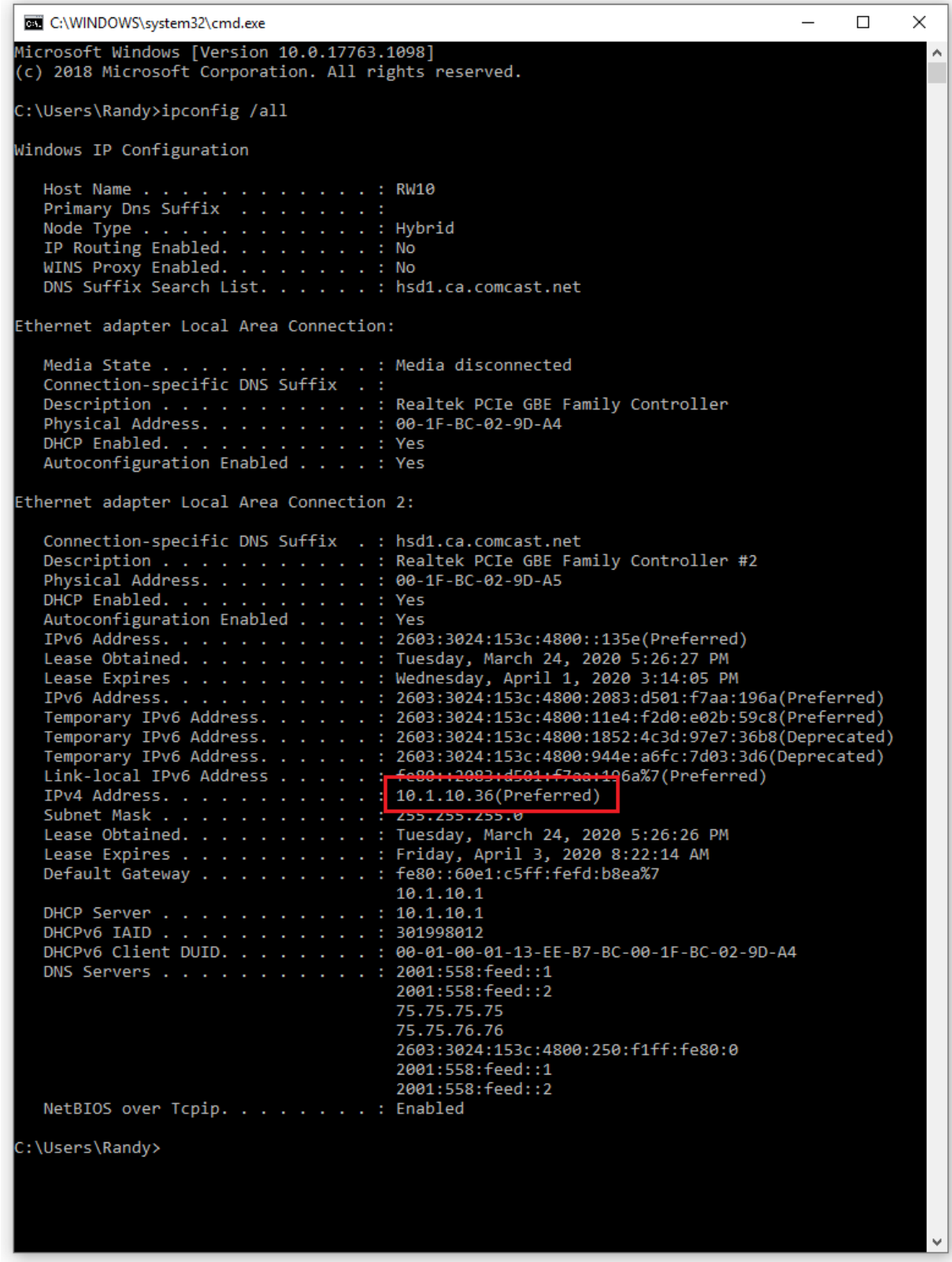


Selecting "OK" should produce a Command Window:



Connect the controller either directly to your PC or to a router on your network. We expect to see some LED activity if there is a cable attached.

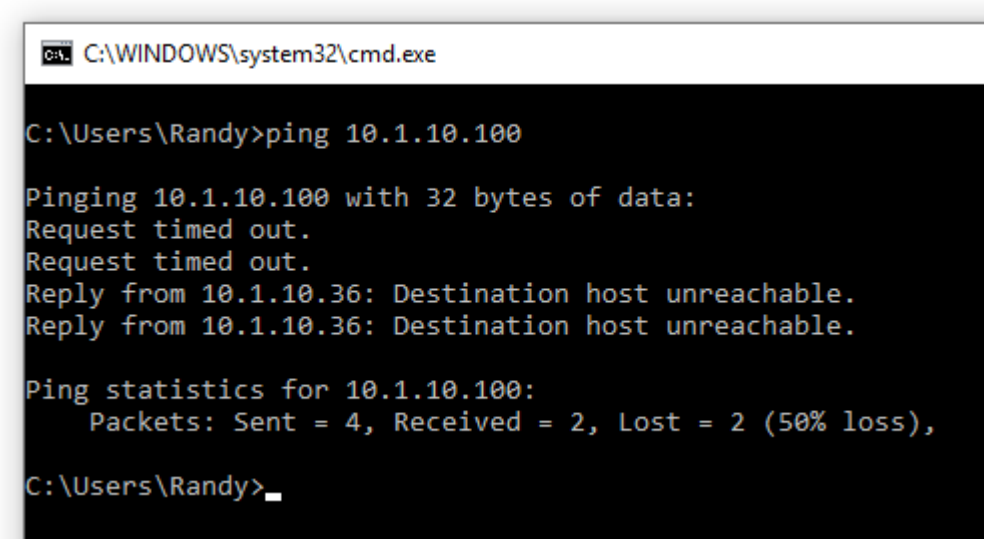
Type the command "ipconfig /all" follow by enter:



There may be a number of Ethernet adapters listed. Look for one that is not disconnected showing an IPv4 Address (indicated above by the red rectangle).

Confirm a Candidate IP Address Has No Conflicts

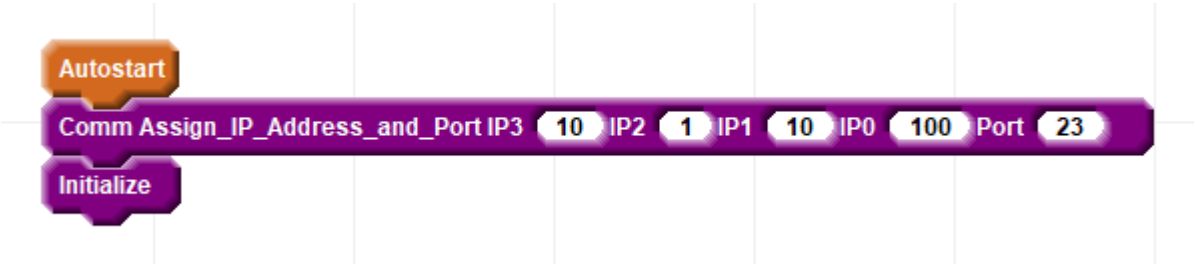
The IP address that will be used in the controller will share the same three initial numbers as the PC but have a unique fourth number. Pick a fourth number that is different from the number shown (which belongs to the PC). Let's choose, in this example, "100" for the fourth number. Combined with the first three numbers seen in ipconfig the candidate IP address is "10.1.10.100". There's the chance that some other device on the network already has this address which would create a conflict. To insure this new number will not cause a conflict use the "ping" command and see if any other device responds:



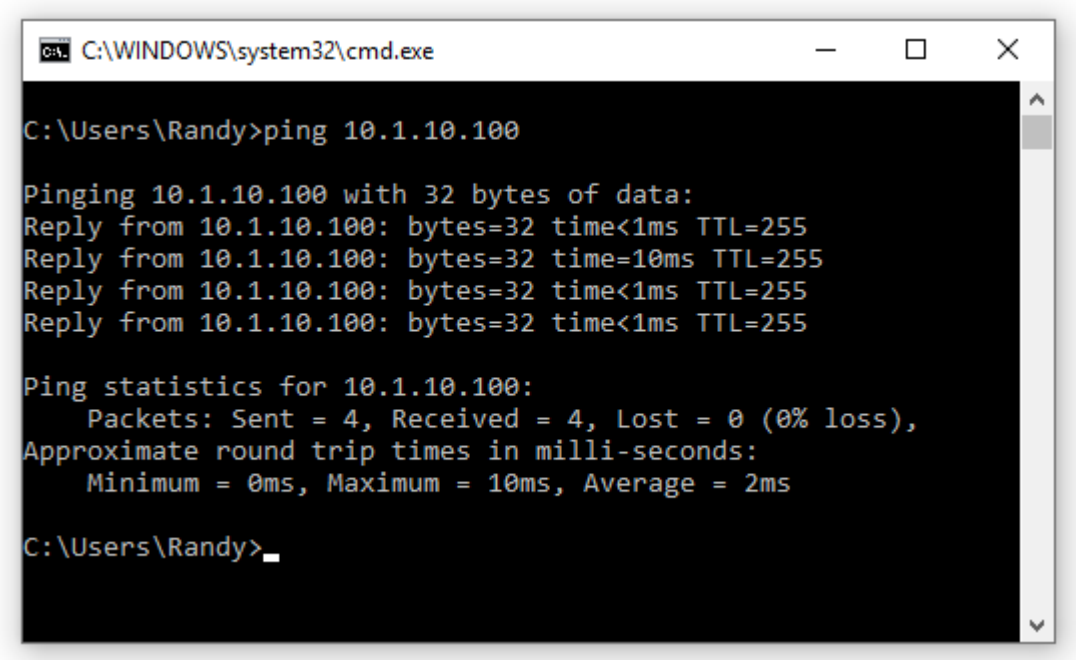
In this case there are several different responses, none of them positive, including "Request timed out." and "Destination host unreachable.". The statistics indicate that 2 packets were received and that there was only 50% loss. However that really just means that the packets made some progress moving about the network. What was received was not from 10.1.10.100 but rather from the PC itself indicating no success. This type of response indicates there is no conflicting device using 10.1.10.100.

Assign the Candidate IP Address to the Controller and Run the Project

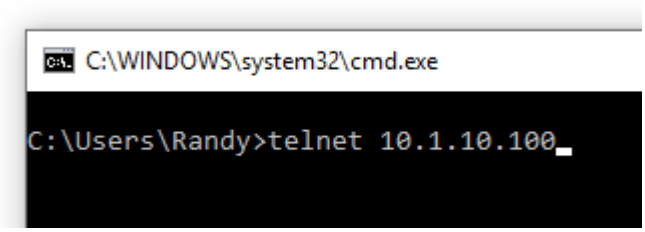
Now follow the instructions in chosen the Method Of Use project to set the IP address of the controller with Snap2Motion. For the Ascii Commands Method of Use the IP address is set in this Ascii Commands Autostart block"



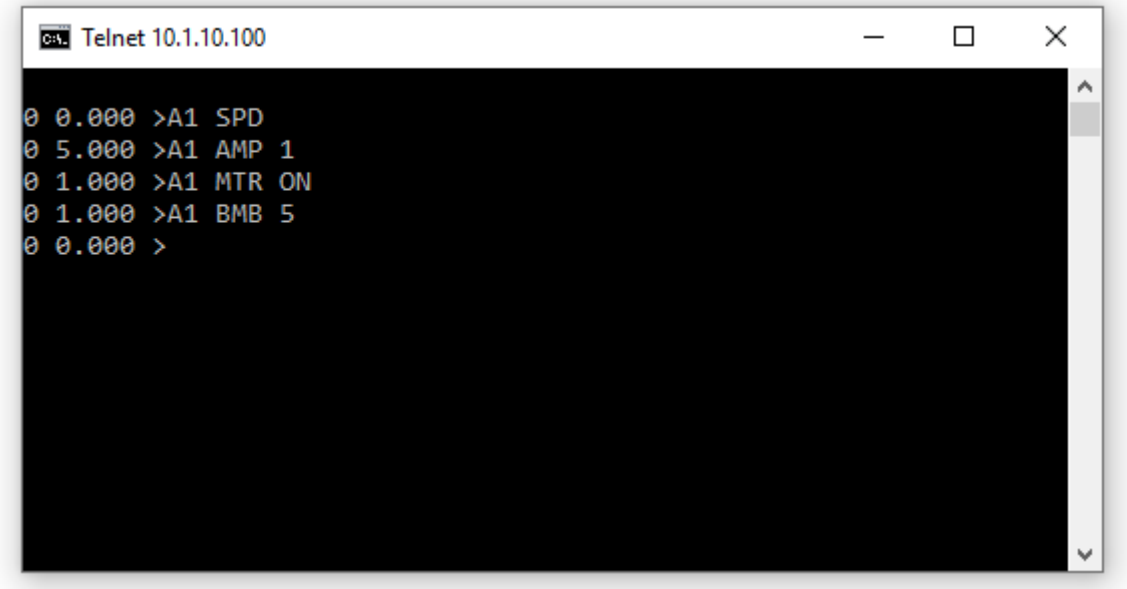
Now run the project by click the green "play" button and wait for the project to start. Return to the command window and use the ping command again (use the up-arrow to recall the last command issued).



This is a different and much more favorable response indicating that there is indeed a device listening and responding (the controller) that received 4 packets and lost nothing. This indicates a successfully chosen IP address for the controller. The next step would be to continue with Method Of Use instructions relating to the use of the connection. In the case of Ascii Commands the next step is to start Telnet:



Telnet can then be used to send Ascii Commands:



Troubleshooting

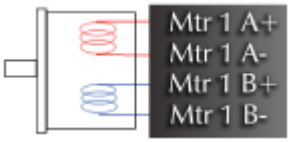
Symptom	Remedy
Only Ping Works	Confirm that controller IP address is different from PC IP address
No Ping Response	Confirm all network elements are on the same subnet

First Motion

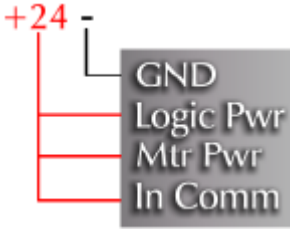
Follow these steps to setup the controller and achieve initial motion.

Wiring

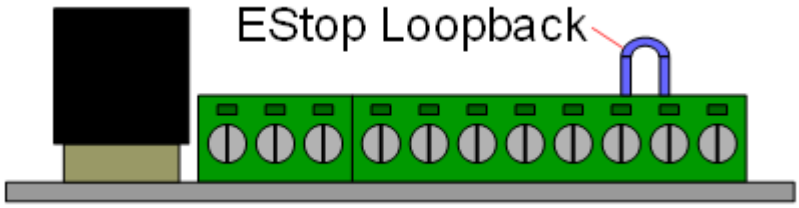
Connect the motor leads from your bipolar motor as shown on the label for Mtr 1. The Mtr 1 plug is the lower plug.



Wire a 24 volt power supply to the controller. The power plug is the upper plug. Supply return goes to the GND pin. Positive 24 volts goes to Logic Pwr, Mtr Pwr, and In Common.



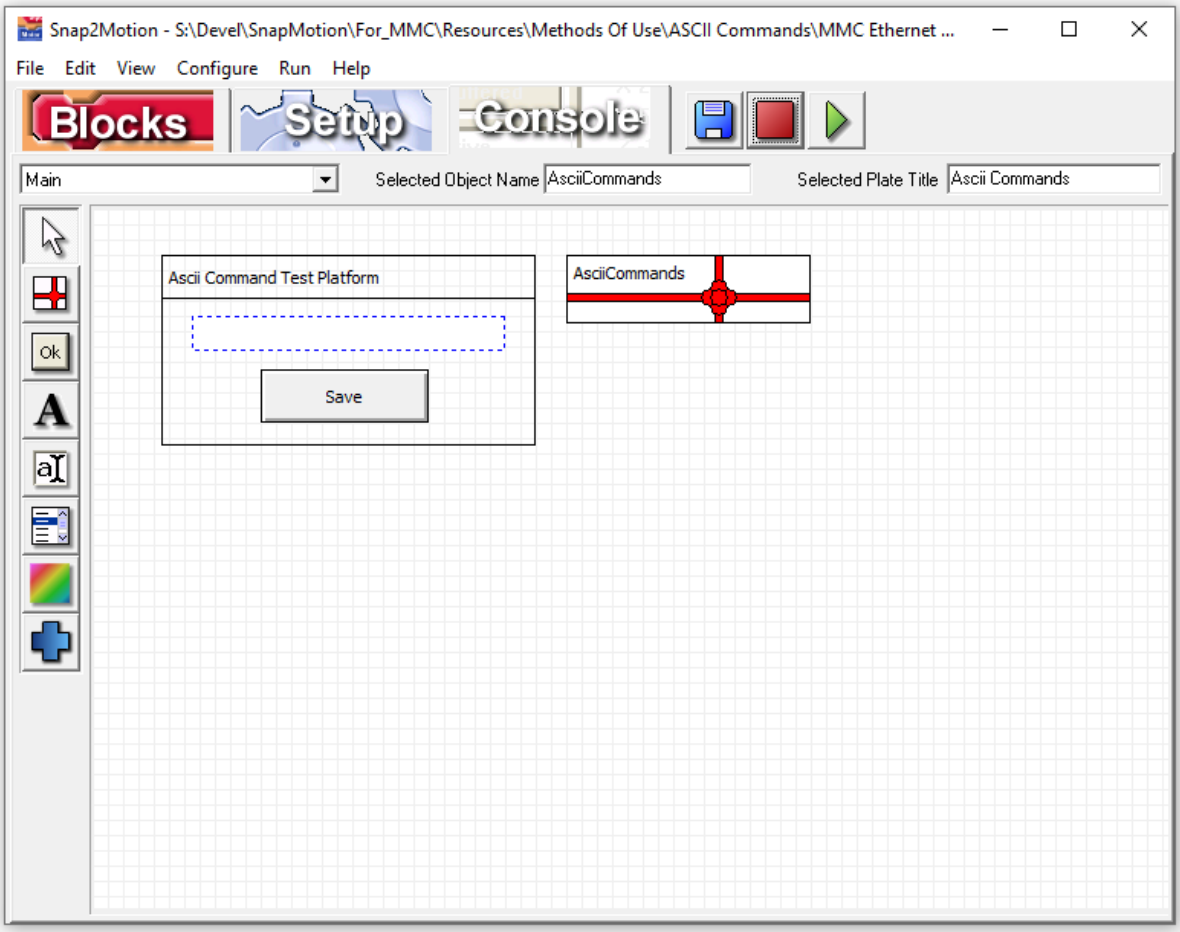
Place a loopback cable as shown on the side connector. This satisfies the EStop circuit which is necessary to permit motion.



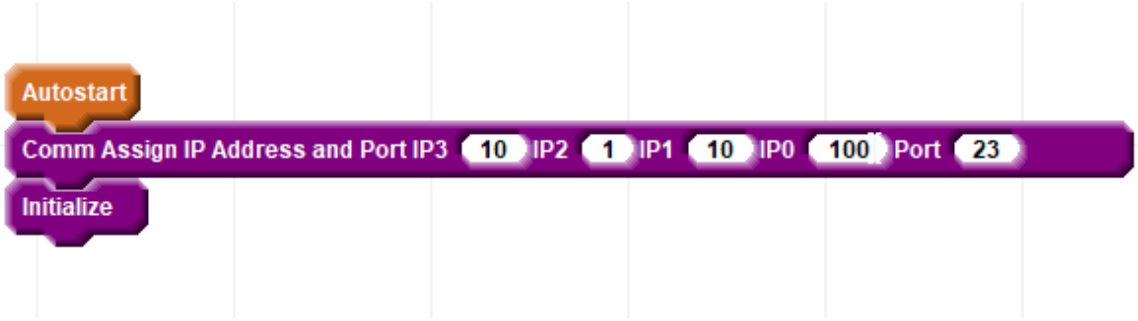
Apply power to the controller. The green "heartbeat" LED should start blinking after about 3 seconds . The heatbeat LED needs to be blinking before the controller will respond to software. If you encounter a problem the first question support will ask will likely be "Is the heartbeat LED still blinking? Was there any deviation to the blink pattern during the problem?".

Set Communication Parameters

Select "File | Open Software Components..", go into the Methods Of Use directory and identify the type of Ascii Command Interpreter needed. Below is the view of the TCP/IP version:

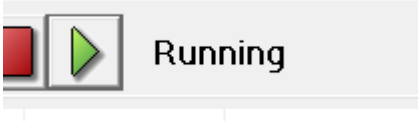


Click on the "ASCIICommands" package (ribbons and a bow) to select the package. Then click on the "Blocks" tab. This will show you the following where the IP address can be set:



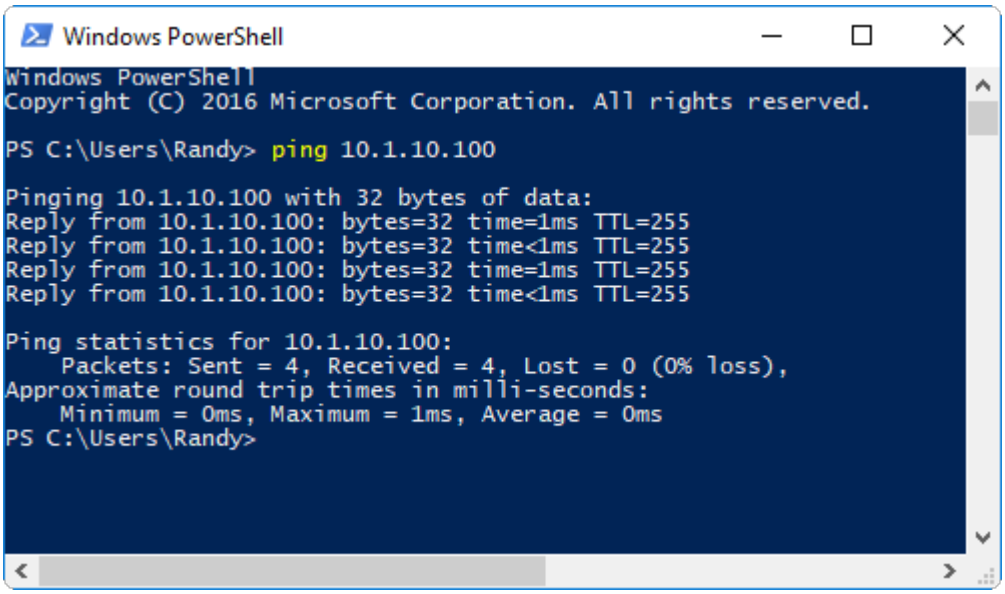
In this example Telnet will be used which has a default port of 23. Other port numbers can be chosen.

Click the "Play" button and wait for the status to indicate "Running":



Use "Ping" to Confirm Network Configuration

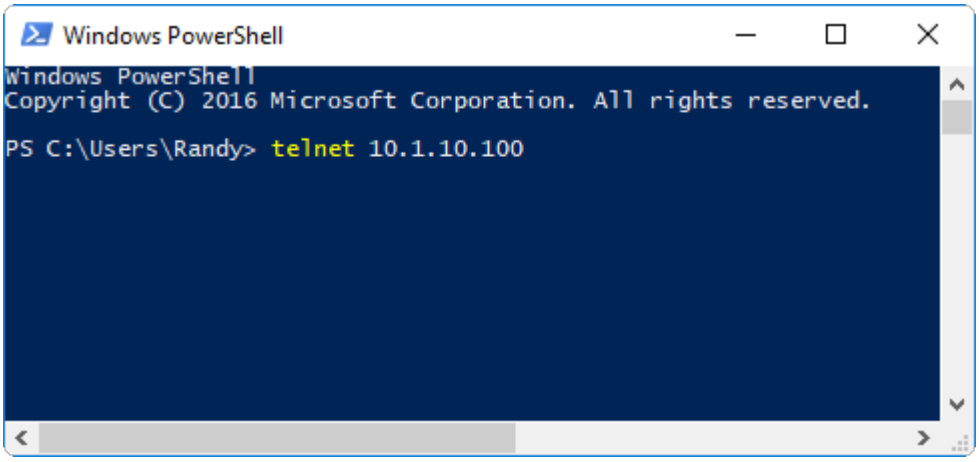
Once the program is running we can use the "ping" command to confirm that the PC is able to connect to the controller over TCP/IP. Open a command line or PowerShell and type in the ping command:



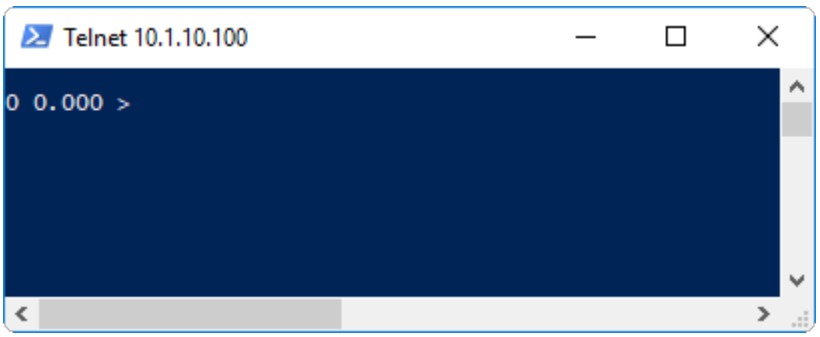
If you do not get a ping response confirm that you have a cable plugged in and are seeing the green activity lighto on the Ethernet connector. Confirm that the PC is on the same subneet with a non-conflicting IP address.

Use Telnet to Issue Commands

Confirm that you have your computer's IP address on the same subnet as what was chosen for the controller's IP address. Plug in an Ethernet cable and confirm that you see the characteristic green blinking light indicating Ethernet traffic. Open a command or Power Shell window and initiate a telnet session by providing the IP address:

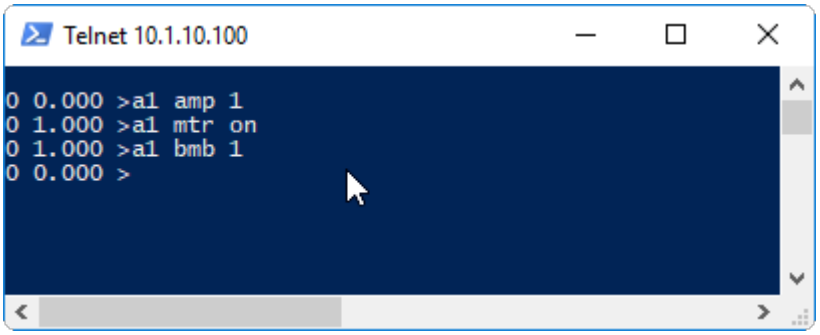


Press "Enter" and the following response should be seen:



The first "0" indicates that no error occurred in handling the command. The second floating point 0 is the value of the response which in this nop command case is 0.

Now type in the following 3 commands and the motor should turn one rotation:



The first command means "Axs 1 set current to 1 amp". The second command means "Axis 1 set the motor to be on". The third command means "Axis 1 begin move by 1 rotation". Additional commands and parameter details are the command reference section of the manual

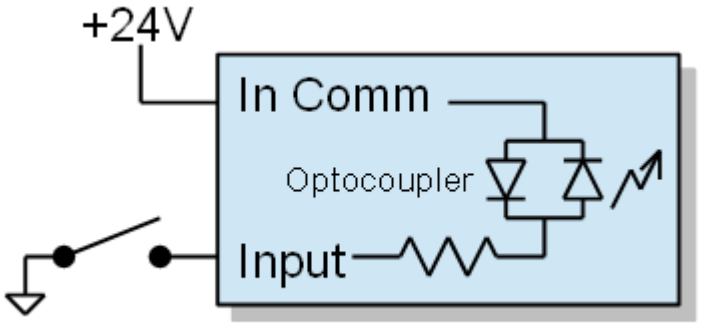
Reading Digital Inputs

Follow these steps to learn how to read an input signal.

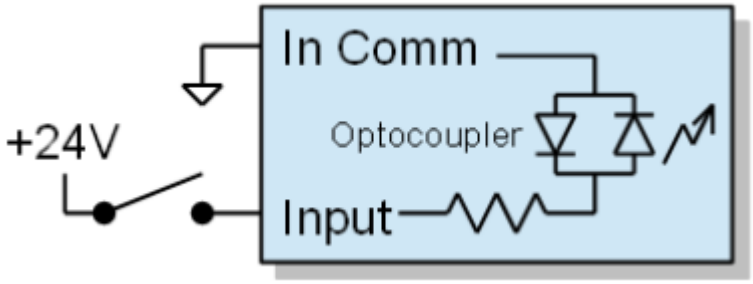
Configuration

Inputs can respond to sourcing signals (PNP, switch closure to +24) or sinking signals (NPN, switch closure to ground). Examine your sensors and determine if the controller inputs need to respond to sourcing or sinking type. It is simplest if the sensors are all the same type. If the sensors are not a all the same type contact support for additional information to support your particular sensor configuration.

If you have sinking inputs then wire "In Comm" (Input Common) to 24 volts and your sensor to the input:



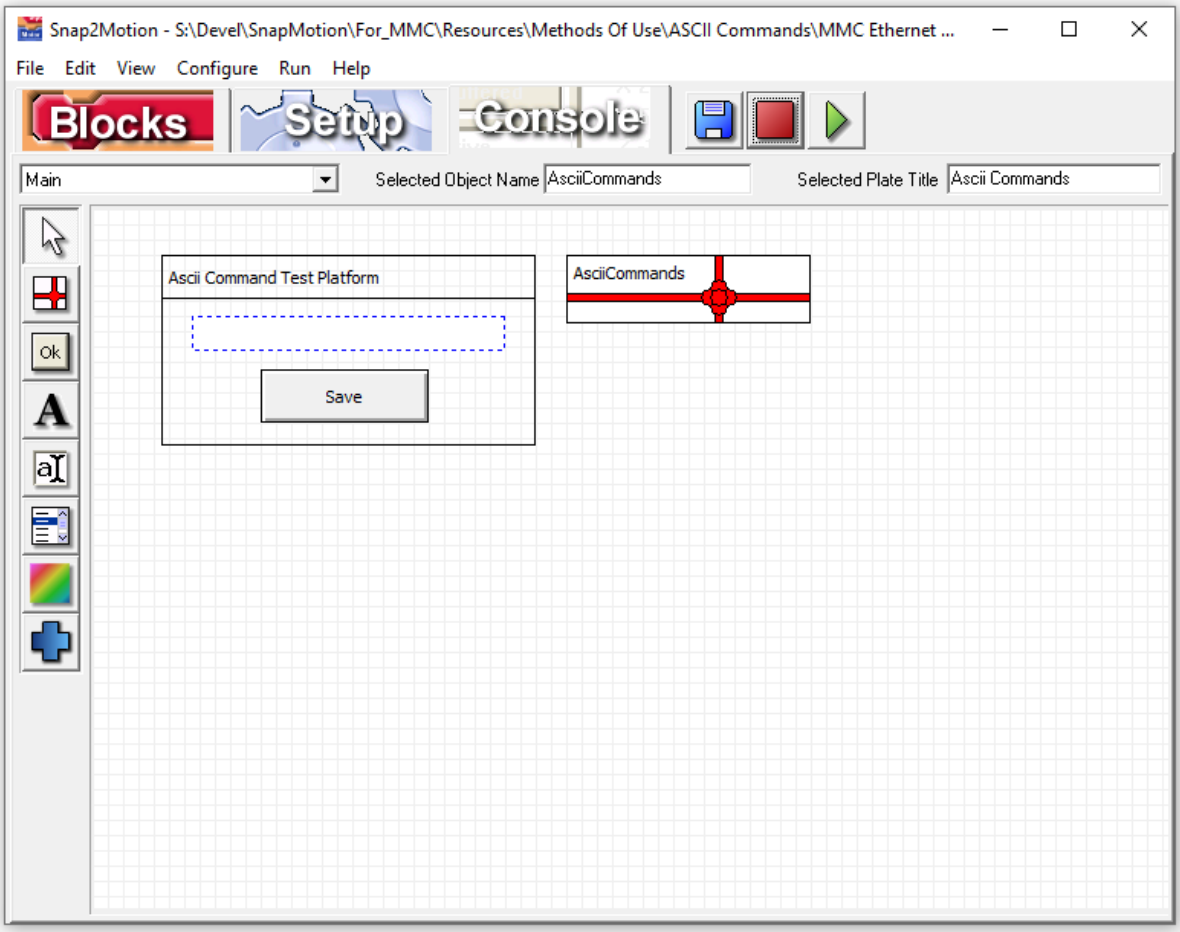
If you have sourcing inputs then wire "In Comm" (Input Common) to return and your sensor to the input:



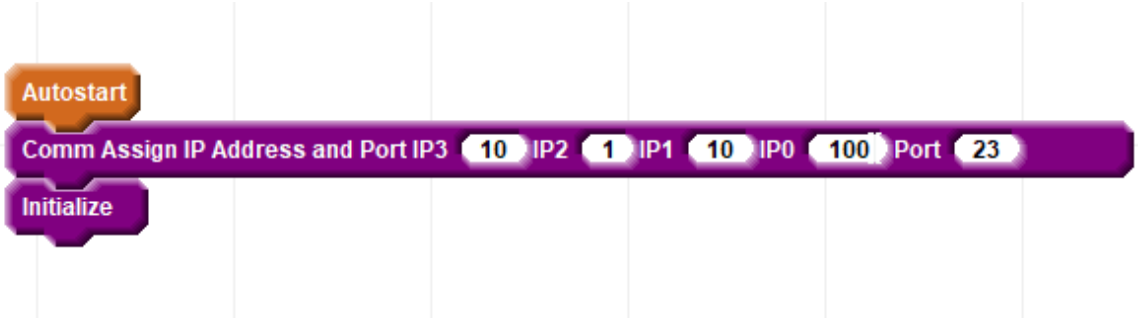
The EStop signal in the left side connector always acts like a sourcing input regardless of the wiring of Input Common. EStop must be active to permit motion.

Set Communication Parameters

Select "File | Open Software Components..", go into the Methods Of Use directory and identify the type of Ascii Command Interpreter needed. Below is the view of the TCP/IP version:

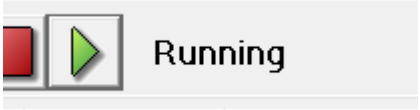


Click on the "ASCIICommands" package (ribbons and a bow) to select the package. Then click on the "Blocks" tab. This will show you the following where the IP address can be set:



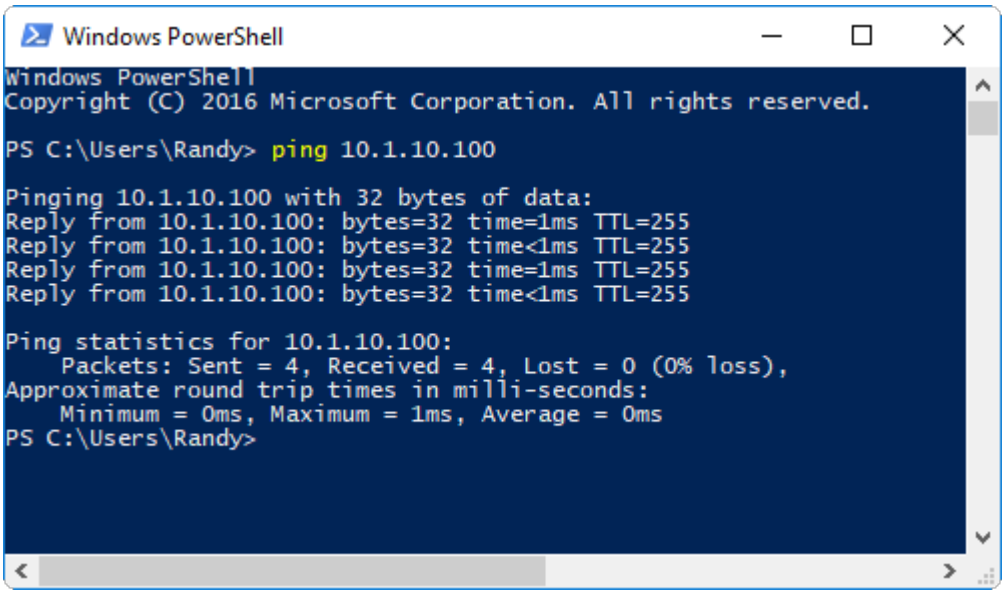
In this example Telnet will be used which has a default port of 23. Other port numbers can be chosen.

Click the "Play" button and wait for the status to indicate "Running":



Use "Ping" to Confirm Network Configuration

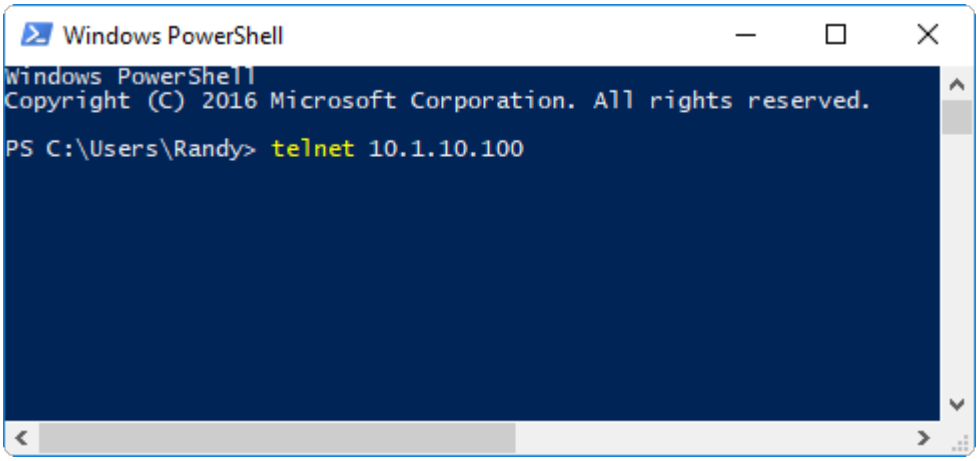
Once the program is running we can use the "ping" command to confirm that the PC is able to connect to the controller over TCP/IP. Open a command line or PowerShell and type in the ping command:



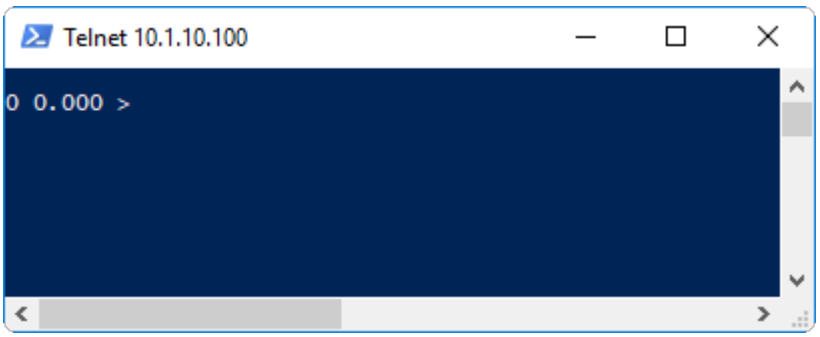
If you do not get a ping response confirm that you have a cable plugged in and are seeing the green activity lighto on the Ethernet connector. Confirm that the PC is on the same subneet with a non-conflicting IP address.

Use Telnet to Issue Commands

Confirm that you have your computer's IP address on the same subnet as what was chosen for the controller's IP address. Plug in an Ethernet cable and confirm that you see the characteristic green blinking light indicating Ethernet traffic. Open a command or Power Shell window and initiate a telnet session by providing the IP address:

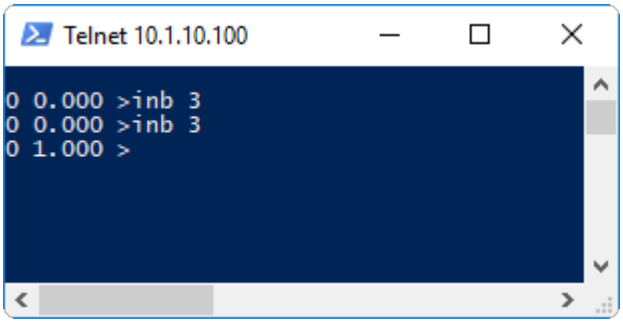


Press "Enter" and this is seen:



The first "0" indicates that no error occurred in handling the command. The second floating point 0 is the value of the response which in this nop command case is 0.

Now type in the command "INB 3" with signal unasserted followed by the signal being asserted.



If the signal is not present the return result is 0 (no error) and 0.000 (no signal). If the signal is present the first response is 0 (no error) followed by 1.000 (signal present). Additional commands and parameter details are the command reference section of the manual.

Controller Description

Controller Description

This describes the controller itself.

Specifications

Controller Description Categories

The specifications of the motor controller

Communication Ports

Description	Type	Connector
Programming Port	USB	USB Standard B
General Purpose Serial	RS232 or RS485 Run-time configurable	3 pin 3.81 mm Euro Screw terminal
General Purpose Ethernet	TCP/IP server supporting 8 concurrent ports	Single RJ-45

Controller Power

Description	Value	Units
Logic Input Voltage Range	12-48	volts DC
Logic Input Power, no outputs active, no 5V load, single module	3	watts
Motor Bus Voltage Range	12-48	volts DC
Motor Drive Current Per Phase	6	amps
5V Out	750	milliamps

Inputs and Outputs

Resource	Number	Voltage	Description
Configurable Digital IO	2	5--48	2 amps sourcing output current optoisolated input is sourcing or sinking as group
Dedicated Digital Input	4	5-48	Optoisolated sourcing or sinking as a group
Analog Input	1	0-20	0-20 volt measurement range. Clamps permit voltage up to 48 volts on pin. 16 bit resolution
Analog Output	1	0-5	12 bit resolution
Encoder Inputs RS422 Differential Receivers	3	5-24	A, B, and Index channels can be used for encoder input or general purpose inputs

LED Indicators

LED	Description
Green Heartbeat	During normal operation this LED should blink at a rate between 1 Hz and several Hz reflecting the controller sample rate. A hesitation or disruption of the steady heartbeat pattern reflects a possible hardware or software issue. The first diagnostic question support will ask is “Is the Heartbeat LED blinking?”.
Yellow Application	This indicator is under application program control and has no intrinsic meaning. The Intermodule Slave program, loaded onto controllers that snap onto a left-most master controller, rapidly blink the Yellow Application LED several times during program startup when the controller is isolated. This pattern indicates that the controller is prepared to snap onto another module.
Ethernet Green Link Active	The link active light indicates that there is traffic on the Ethernet cable and that both ends are connected. If there are any issues with Ethernet first confirm that the link is active.
Ethernet Yellow 10/100	This indicator shows the connection speed

Motion System

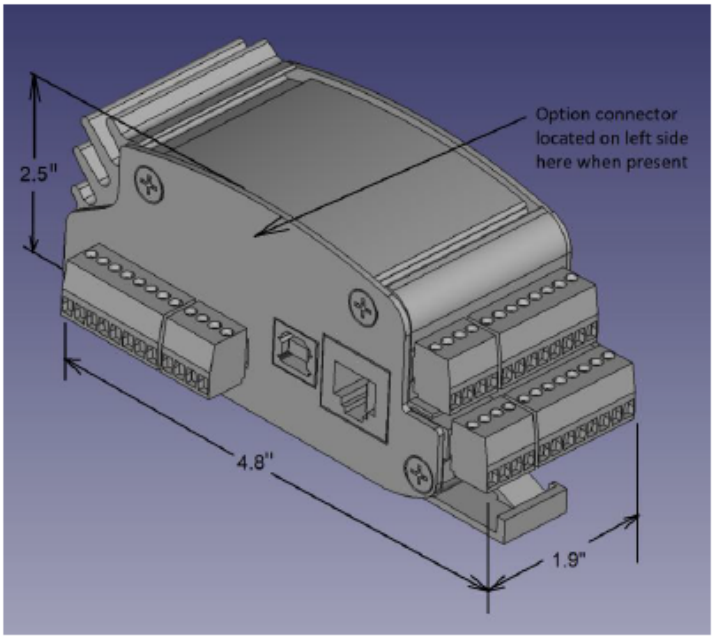
Property	Value
Microcontroller	Arm Cortex
Sample Rates	Configurable from 1 kHz to 8 kHz
Native Motion Capabilities	Concurrent Motion Coordinated Vector Motion
Application Motion Capabilities	Electronic Gearing Electronic Camming Encoder-Replaces-Time PVT Kinematics Conveyor Tracking Arbitrary Closed Form Equations
Position Range	32 bit
Maximum Encoder Count Rate post quadrature	2 MHz
Maximum Step Rate	2 MHz
Hardware Position Capture	Accurate to an individual count

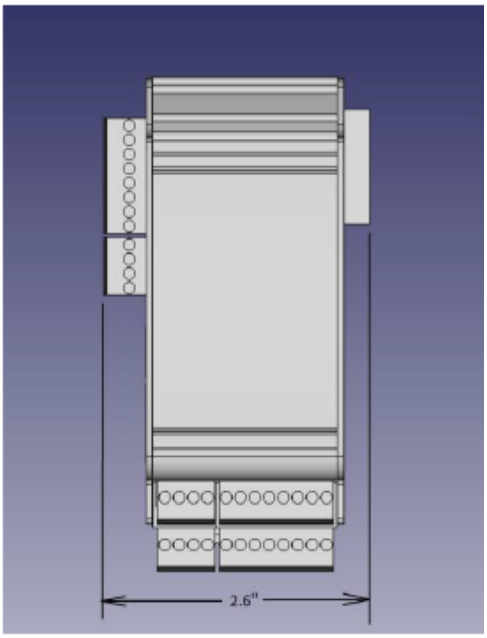
Mechanical

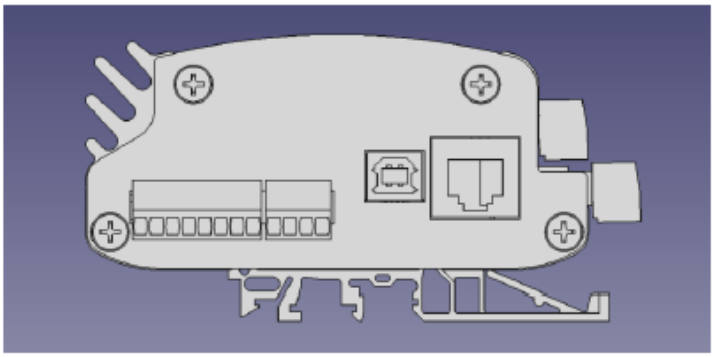
Controller Description Categories

The mechanical layout of the motor controller

Mechanical







Clearance required for right side connector. Insure clearance is provided for airflow around heat sink fins when operating at higher current levels. Avoid mounting controller inverted as this traps air in the heat sink fins. Insure clearance on left side for wiring, USB, and Ethernet connectors.

Connectors

Controller Description Categories

The connectors on the motor controller

Power Connector

Front Side Top 4 Position

MMC	MMC-TC	Description
Gnd	Gnd	Use for power supply return connection
Logic Pwr	Logic Pwr	Used for power supply "+" connection. It is recommended that Logic Power not be interrupted by safety disconnects so as to sustain program operation and communication with host
	Enable	

Motor Pwr		The MMC-T uses this point for Motor Power to provide voltage to the internal drive. Safety standards that require disconnecting drive power can be satisfied by disconnecting this position. The MMC-TC uses this point as an active high enable output. This output will apply the Logic Pwr voltage the this point when the drive is enabled. The enable output can source 40 ma of current.
In Common	In Common	Input Common is used to configure inputs as sourcing or sinking. If signals provide voltage then connect In Common to Gnd. If signals act like switch closures to Gnd then connect In Commmon to Logic Pwr.

Motor Connector

Front Side Bottom 4 Position

MMC-T	MMC-TC	Description
Mtr A+	Step+	Connect to one end of first motor phase for MMC-T or to Step+ drive signal input for MMC-TC (RS422 style +5V drive)
Mtr A-	Step-	Connect to opposite end of first motor phase for MMC-T or Step- drive signal input for MMC-TC. (RS422 style +5V drive)
Mtr B+	Dir+	Connect to one end of second motor phase for MMC-T or Dir+ drive signal input for MMC-TC (RS422 style +5V drive)
Mtr B-	Dir-	Connect to opposite end of second motor phase for MMC-T or Dir- signal input for MMC-TC (RS422 style +5V drive)

Encoder Connector

Front Side Top 8 Position

Signal	Description
Gnd	Used to power encoder
5V Out	Used to power encoder. If external drive is providing signals this signal might not be required
Enc A+	Encoder A+ channel if differential or A channel if single ended
Enc A-	Encoder A- channel if differential or disconnected if encoder is single ended. Internally pulled to 2V. Do not connect to Gnd
Enc B+	Encoder B+ channel if differential or B channel if single ended
Enc B-	Encoder B- channel if differential or disconnected if encoder is single ended. Internally pulled to 2V. Do not connect to Gnd
Enc I+	Encoder Index+ channel if differential or Index channel if single ended
Enc I-	Encoder Index- channel if differential or disconnected if encoder is single ended. Internally pulled to 2V. Do not connect to Gnd

IO Connector

Front Side Bottom 8 Position

If an input is needed and a dedicated input is available it is best to use the dedicated input and preserve the more versatile DIO positions for outputs or spares

Signal	Description
DIO 1	Digital Input Output 1 can be configured as an input or an output
DIO 2	Digital Input Output 2 can be configured as an input or an output
DIN 3	Digital Input 3 is a dedicated input
DIN 4	Digital Input 4 is a dedicated input
DIN 5	Digital Input 5 is a dedicated input
DIN 6	Digital Input 6 is a dedicated input
AIN	Analog Input
AOUT	Analog Output

EStop Connector

Left Side 8 Position

The first three positions of the EStop connector are for application use. The remaining positions are used for communication to snapped-together modules. No connections should be made to the remaining 5 positions.

Signal	Description
Gnd	Provides Gnd to snapped-together modules. May be used as a general purpose Gnd connection position
Logic Pwr	Provides Logic Power to snapped-together modules. May be used as a general purpose Logic Power position and is often jumpered to the adjacent EStop position to satisfy EStop
EStop	This hardware EStop input must be satisfied to permit motion. The signal is satisfied by being connected to 5-48 volts. This connection usually incorporates normally closed EStop buttons so that the opening of a button, or breakage of a wire, performs a shutdown

Serial Connector

Left Side 3 Position

The serial connector can support RS232 or RS485 based on software configuration. When using RS485 a termination resistor is recommended to insure failsafe signal management features operate properly. For noisy environments an external failsafe bias circuit is recommended.

Signal	Description
Gnd	Common to all Gnd positions on controller this is used to provide a common Gnd to the serial device
Rx/+	RS232 receive signal should be connected to transmitter of serial device. When configured for RS485 this is the Data+ signal
Tx/-	RS232 transmit signal should be connected to receiver of serial device. When configured for RS485 this is the Data- signal.

Intermodule Plug Connector

Right Side 8 Position

The plug style connector on the right side of the controller is only used to provide power and communication to snapped-together modules. No connections should be made to this plug connector.

Options

Controller Description Categories

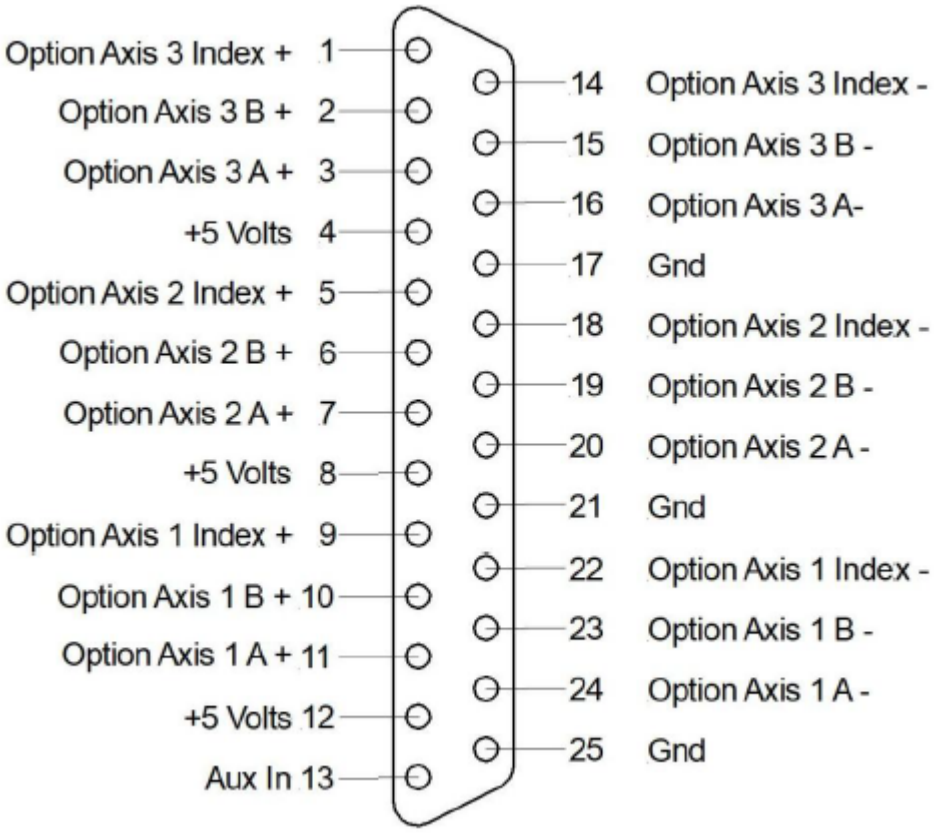
The options for the motor controller

3ENC Option

Description

The 3ENC Option board provides support for three encoders, each with RS422 differential inputs for A, B, and Index signals from a 3 channel encoder. These additional encoders can be used for purposes such as master position references, dual loop, and hand wheel inputs. The 25 pin DSub connector is presented on the left side of the controller above the USB connector. Power for the encoders on this option board are part of the controller?s +5V current budget.

Pin Assignments



Option Specifications

Description	Value	Units
Input Signal Voltage Range	5-24	volts DC

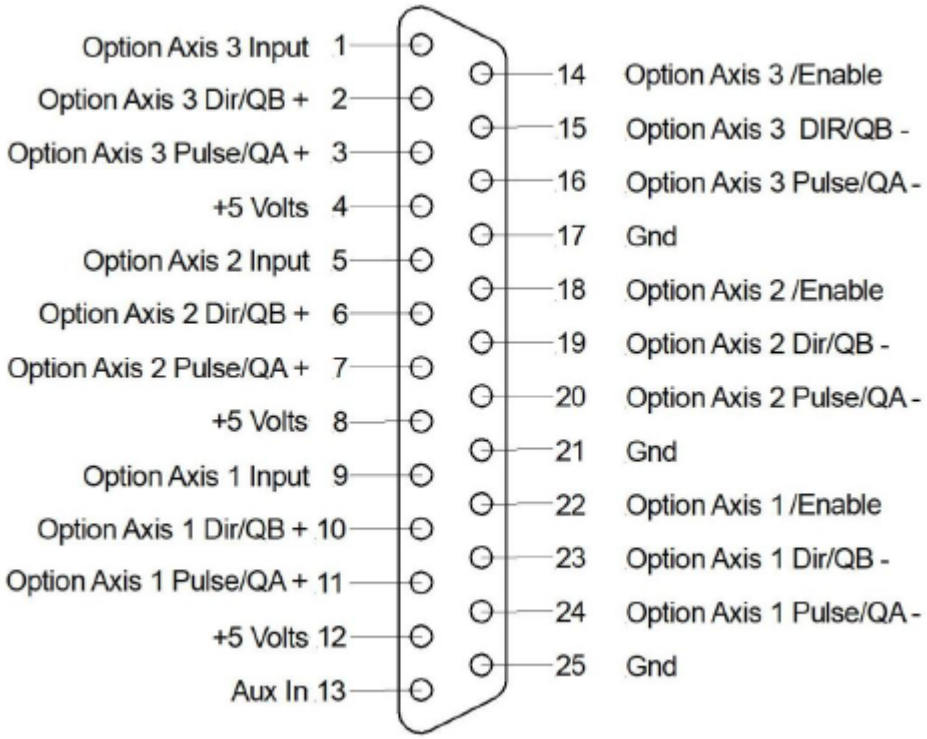
3PD Option

Description

The 3PD Option board provides support for additional motion axes expressed as signals for use with external drives. These signals can be configured as Pulse/Direction signals or as driven quadrature signals for use with external stepper motor drivers or servo motor drivers that can accept external pulse/direction or external quadrature signals that the drive follows. Motion signals are RS422 differential outputs. If available, the quadrature format is desirable as it is more noise immune than pulse/dir however with good wiring practice either format is reliable. An input for each axis is provided and can be used for drive status or as a homing input. This input is the □+□ side of a differential receiver where the □-□ side is tied to a 2 volt internal reference.

Pin Assignments

Connector on controller has female pins



Option Specifications

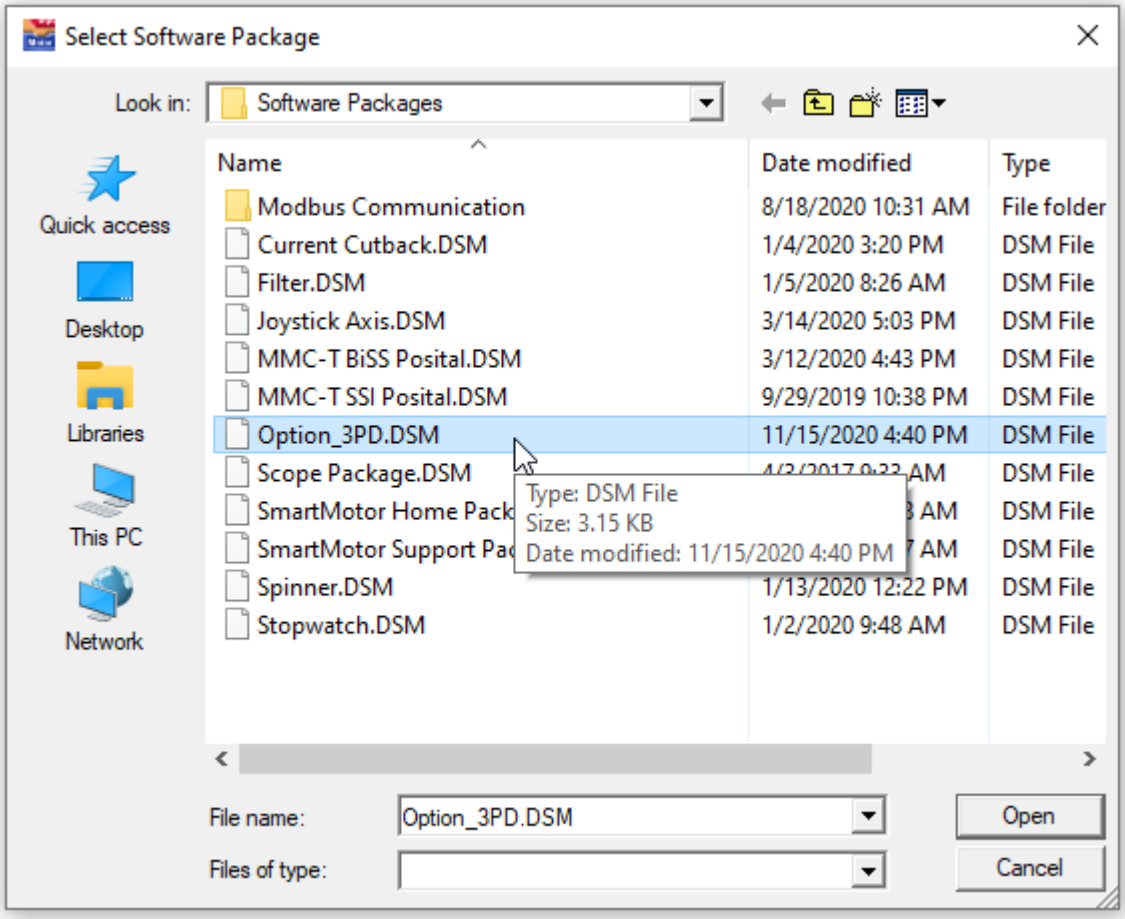
Signal Name	Description
Option Axis Input	Available to be driven by external driver or sensor. This signal has a 4.7K pullup resistor to 5 volts. It is simplest for this signal to be driven low by a sinking style output. The applied voltage should not exceed 24 volts.
Option Axis Step/QA+	This is the "+" side of a differential driver. The "Set Step Width" block can be used to adjust the with of the step pulse. If the source is single ended for step, with only one wire, it should be connected to this input.
Option Axis Step/QA-	This is the "-" side of a differential driver. This signal is connected to a 2 volt reference with a 1K ohm resistor. If the source is single ended for step, with only one wire, this pin should be left disconnected. Do not connect this signal to GND.
Option Axis Dir/QB+	This is the "+" side of a differential driver. The motor reverses direction when this signal changes. If the source is single ended for direction, with only one wire, it should be connected to this input
Option Axis Dir/QB-	This is the "-" side of a differential driver. This signal is connected to a 2 volt reference with a 1K ohm resistor. If the source is single ended for direction, with only one wire, this pin should be left disconnected. Do not connect this signal to GND.
Option / Enable	This output connects to GND when the axis is enabled and is open when the motor is disabled. The output can tolerate 35 volts when in the open condition. When closed to Gnd a maximum of 30 milliamps can be driven.
+5	This pin provides +5V for use with external devices that may require power to provide a signal. It is not necessary to connect +5V to this pin.
Gnd	This pin provides Gnd for use with external devices that may require power to provide a signal. It is not necessary to connect Gnd to this pin.
Aux In	

	One additional general purpose input is available on the option connector. This signal has a 4.7K pullup resistor to 5 volts. It is simplest for this signal to be driven low by a sinking style output. The applied voltage should not exceed 24 volts.

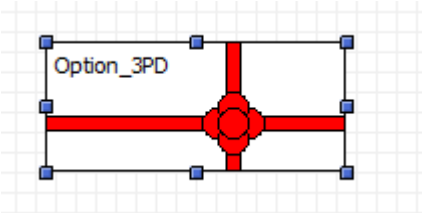
Software Packages

The 3PD option board is supported by the Option_3PD software package. From the console tab select the "Add Software Package" tool:

Select the Option_3PD package:



This places the Option_3PD software package into the project:



Connection Table for Teknic Clearpath Drives

The following connections can be used to control a Teknic Clearpath drive configured for following mode. The High Level Feedback Signal (HFLB) is read through the axis input to obtain drive status. The "+" side of the Teknic inputs is connected to +5V in order to satisfy Teknic's voltage requirement. Connecting the "+" side of the Teknic input to the "+" side of the option's differential driver and the "-" side of the Teknic input to the "-" side of the option's differential driver would not provide sufficient voltage (as indicated in the Teknic documentation). Use the Set Step Width block to indicate step/dir format with a parameter of at least 2 microseconds.

Option Axis 1

Clearpath Pin Number	Clearpath Wire Color	Clearpath Signal Name	Option Pin Number	Option Signal Name
1	Green	HLFB+	9	Option Axis 1 Input
2	Black	Input B+	12	+5V
3	White	Input A+	12	+5V
4	Blue	Enable+	12	+5V
5	Red	HLFB-	25	Gnd
6	Yellow	Input B-	24	Option Axis 1 Pulse/ QA-
7	Brown	Input A-	23	Option Axis 1 Dir/ QB-
8	Orange	Enable -	22	Option Axis 1 / Enable

Option Axis 2

Clearpath Pin Number	Clearpath Wire Color	Clearpath Signal Name	Option Pin Number	Option Signal Name
1	Green	HLFB+	5	Option Axis 2 Input
2	Black	Input B+	8	+5V
3	White	Input A+	8	+5V
4	Blue	Enable+	8	+5V
5	Red	HLFB-	21	Gnd
6	Yellow	Input B-	20	Option Axis 2 Pulse/ QA-
7	Brown	Input A-	19	

				Option Axis 2 Dir/ QB-
8	Orange	Enable -	18	Option Axis 2 / Enable

Option Axis 3

Clearpath Pin Number	Clearpath Wire Color	Clearpath Signal Name	Option Pin Number	Option Signal Name
1	Green	HLFB+	1	Option Axis 3 Input
2	Black	Input B+	4	+5V
3	White	Input A+	4	+5V
4	Blue	Enable+	4	+5V
5	Red	HLFB-	17	Gnd
6	Yellow	Input B-	16	Option Axis 3 Pulse/ QA-
7	Brown	Input A-	15	Option Axis 3 Dir/ QB-
8	Orange	Enable -	14	Option Axis 3 / Enable

Related Topics

IOH and IOL Option

Description

The IOH and IOL option boards provide additional digital inputs and outputs. IOH provides 24 volt inputs while IOL provides 5 volt inputs. The inputs are optically isolated can be configured for sourcing or sinking. These inputs can operate at several hundred kHz making them suitable for externally driven step/dir signals from a PLC.

Electrical Specifications

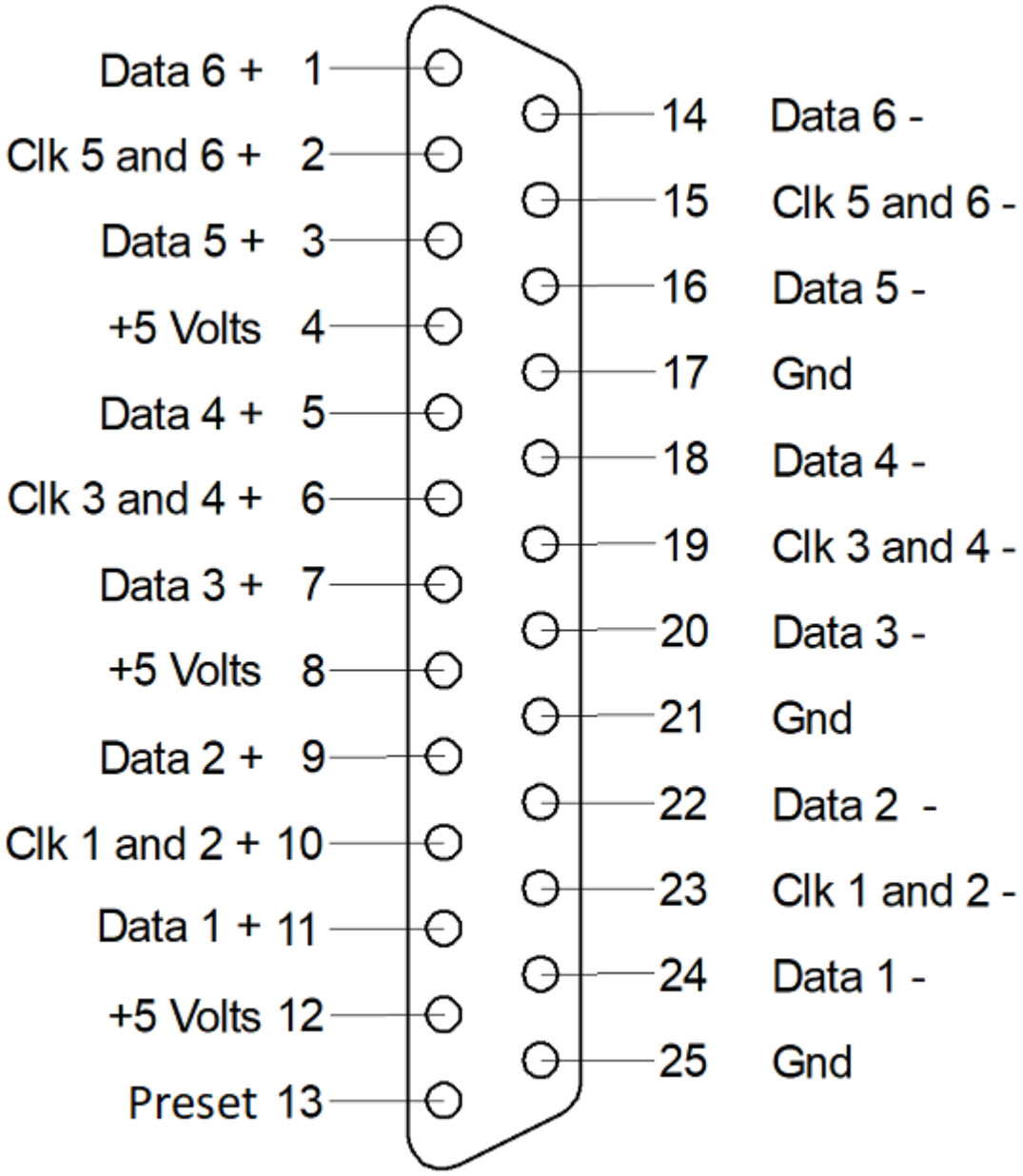
Property	IOH	IOL	Units
Max Input Voltage	24	5	Volts
Max Output Voltage	24	24	Volts
Max Output Current	100	100	ma
Max Incoming Pulse Rate	500	500	kHz

6ABS Option

Description

The 6ABS Option provides support for six BiSS encoders. The encoders are organized into 3 groups of two encoders each. The two encoders in a group share a common RS422 differential driver clock signal, +5V, and GND pins. Each encoder has its own RS422 differential receiver. The remaining pin provides a Preset output that can be applied to all of the axes. The 25 pin DSub connector is presented on the left side of the controller above the USB connector. Refer to the Option_6ABS software package to access 6ABS features.

Pin Assignments



Female DSub Connector On Controller

Option Specifications

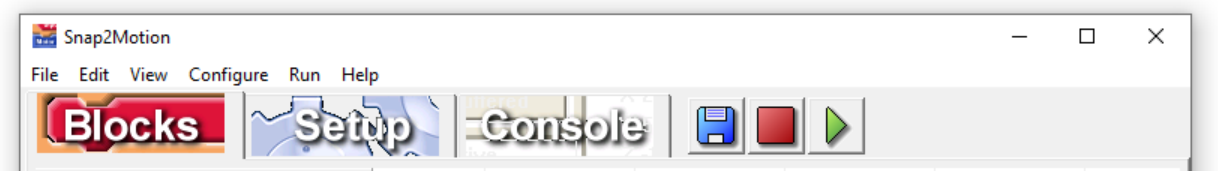
Signal	Description	Rating
+5 Volts	Encoder Power	Max Current 600 ma Total For All Encoders

Data+/-	RS422 Differential Receivers	Max Voltage 24V (5V typical)
CLK+/-	RS422 Differential Drivers	Current Rating 20 ma

Menu Reference

Menus, Tabs, and Buttons

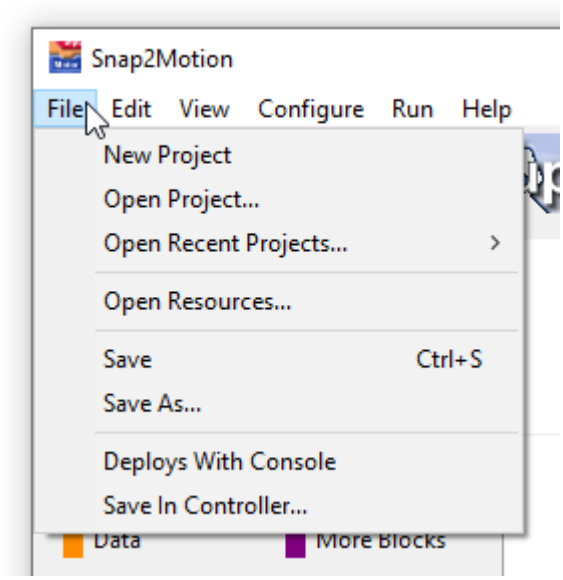
This section describes the Snap2Motion Menu System, Tabs and Buttons.



File

Menu Description

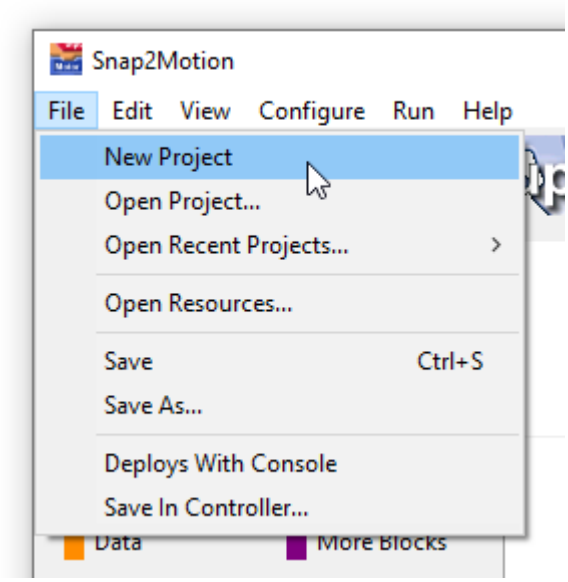
The File Menu relates to saving and loading project information to the PC and controller



New Project

File Menu

The New Project menu item creates a blank page for a new project.

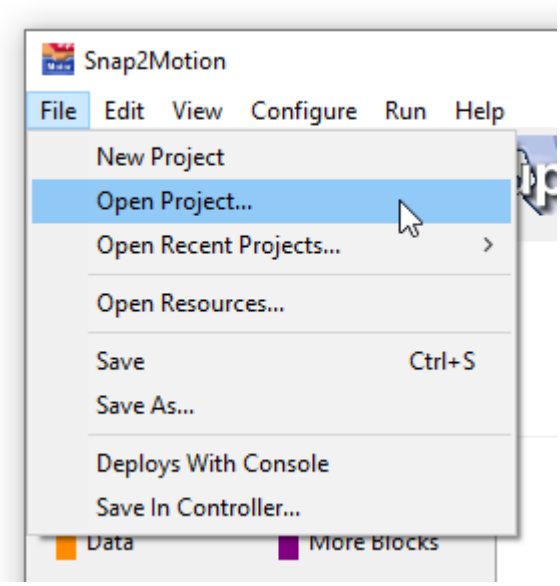


New projects are given default properties including "Compile To Flash" (under Configure) being disabled. For large projects that may need to be turned on but it's best to defer until the extra capacity is needed.

Open Project

File Menu

The Open Project menu item produces a file browser to the current directory for opening up a Snap2Motion project

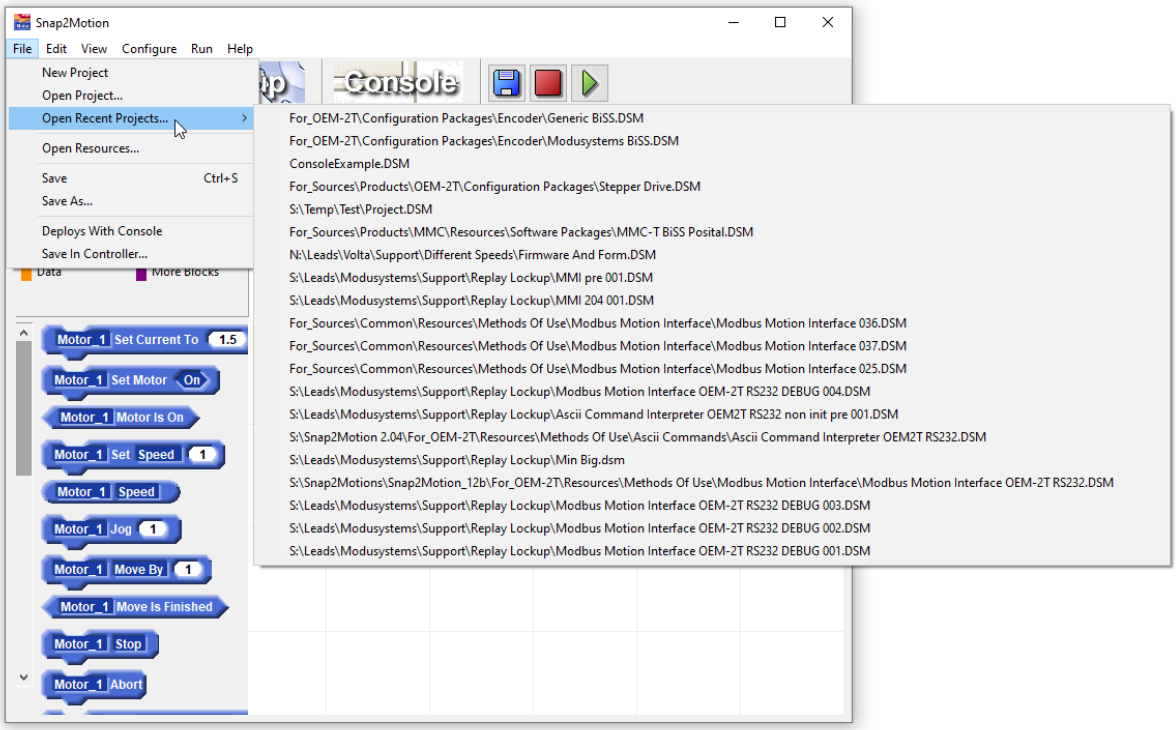


Newer version of Snap2Motion should always be able to open older projects. However if an older Snap2Motion is being used software components and elements provided for technical support may be more current requiring that the Snap2Motion program be updated from our website. Blocks in an older project that are no longer available in newer software will still be load, render, and perform.

Open Recents

File Menu

The Open Recents menu item produces a list of the 20 most opened projects:

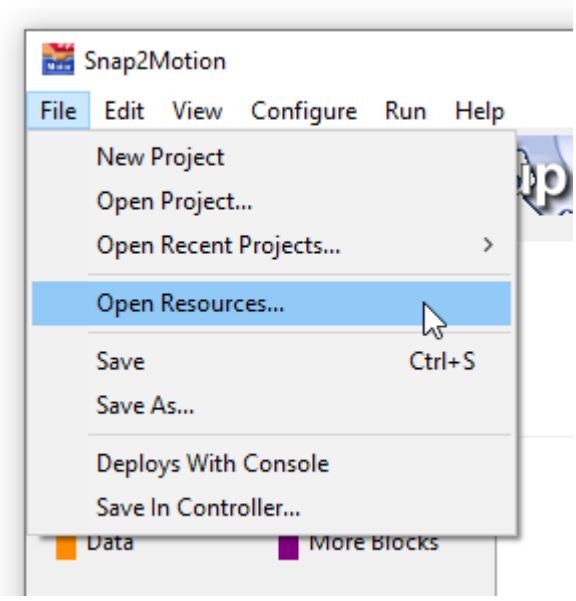


The current project is not show on the list as it is already opened. To revert to the last saved version of the current project select File | New Project and then return to File | Recents and the project will be available. Note that this list is not controller specific. If a number of controller types are being used the projects for all the controllers will be in the list. There is no assurance that a project for another controller type will be useful on the controller currently connected.

Open Resources

File Menu

The Open Resources menu item leads to a directory structure of controller specific examples, software packages, and utilities:

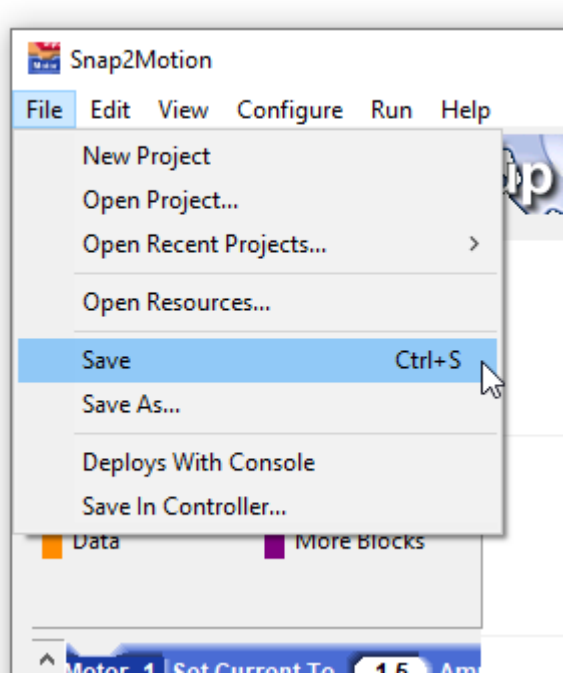


Software Packages opened in this way are presented in "scaffold" projects that illustrate their use. Remember to save opened scaffolds or examples in new work areas so as to leave the resources in their factory stock condition.

Save

File Menu

The Save menu item stores the current project on the PC under the already chosen name. If no name has been chosen a file browser is presented to select a name.

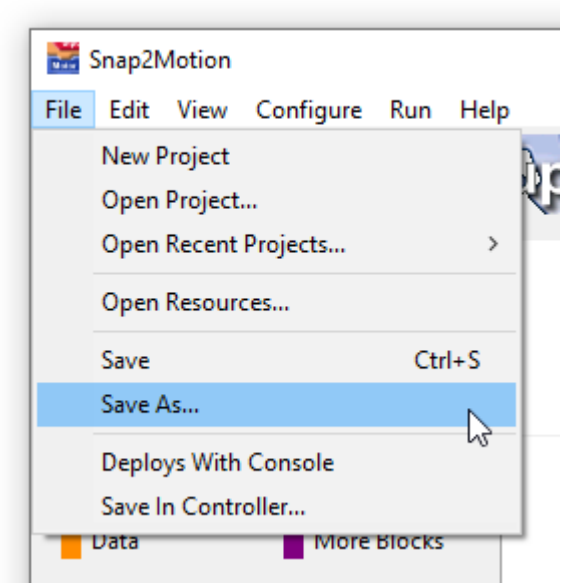


The filename chosen to open a project is its Safe filename. If projects are being opened from example directories or other resource directories make sure to rename them with Save As to keep the resources directory factory stock. The blue disk Icon is a shortcut to File | Save:

Save As

File Menu

The Save As menu item stores the current project on the PC under a new name. The project assumes the new name moving forward.

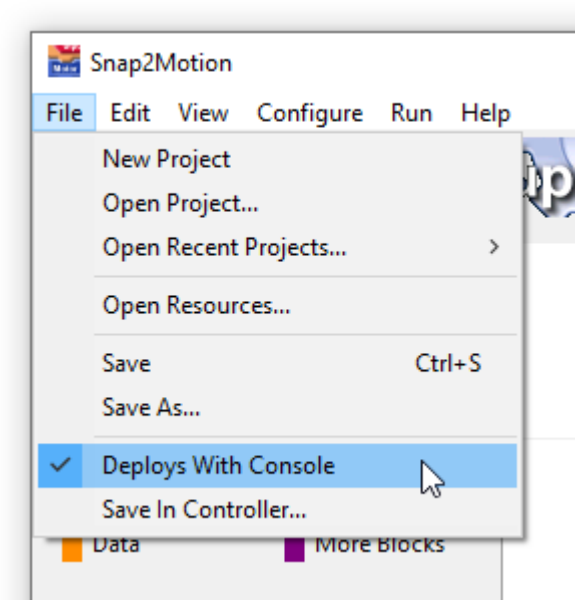


The filename chosen will be the default filename for any future "Save" operations performed either through menu selection, the Control-S key, or the blue save icon:

Deploys With Console

File Menu

The Deploys With Console menu item is a checkbox that indicates if the program is to run standalone without a Snap2Motion console (unchecked) or with a Console (checked)



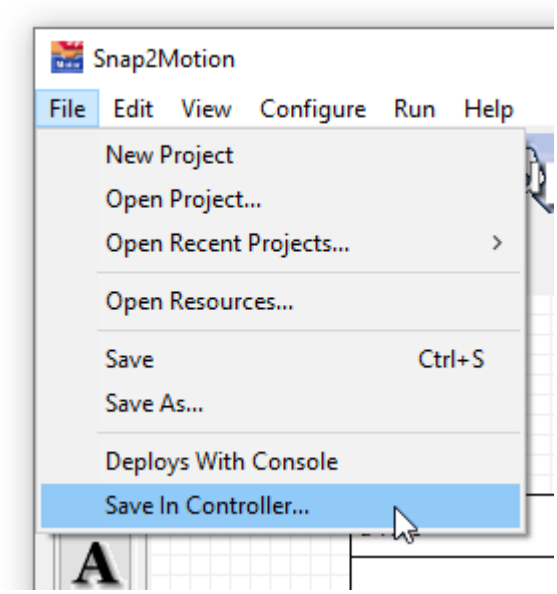
Checking Deploys With Console allows for a much more rapid application startup for deployed systems. Deploys With Console works in conjunction with Save in Controller. If a project is saved in the controller, and the project Deploys With Console is checked, Snap2Motion will confirm that the saved program and saved application match. If they match then startup is immediate as there is no need to download into the controller a program that is already present.

To insure that the program saved in the controller and the one saved on the PC match it is best to first save the application, then run it, and then Save in Controller. If the application on the PC is subsequently changed then the process needs to be repeated to insure both controller and PC programs match.

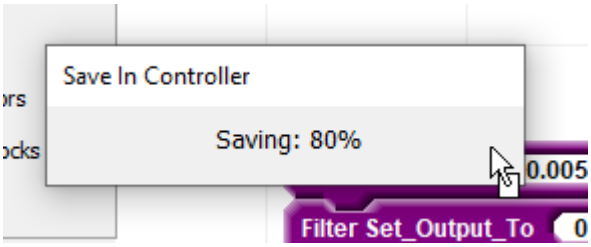
Save In Controller

File Menu

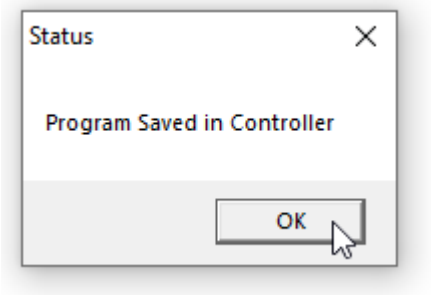
The Save In Controller menu item stores the current project in the controller where it will autostart on the next power cycle.



The project must already have been run. It is best to stop the project if it is currently running and then perform the Save In Controller operation. A successful save shows a progress box:



followed by a prompt indicating completion:



Program storage into the controller is only one way - from the PC into the controller. A project in the controller cannot be retrieved. Make sure projects are saved in the PC in ways that will last as long as the projects being deployed. Note that when a program runs standalone, without Snap2Motion present, that it is important to not be referring to any of the Snap2Motion control items in the program. Attempts to write to a text display or list, which won't be present standalone, will cause that program in the controller to stop waiting for Snap2Motion (which is not there) to respond. When designing projects for stand-alone operation isolate tasks that view data from tasks that act on the data which is good design practice regardless of deployment form.

There are cases where an autostart program can create a condition that locks up the controller. This creates a cunundrum since the controller might not connect or respond to Snap2Motion in this condition preventing the autostart program from being changed. There is a recovery procedure avalable which involves addressing the

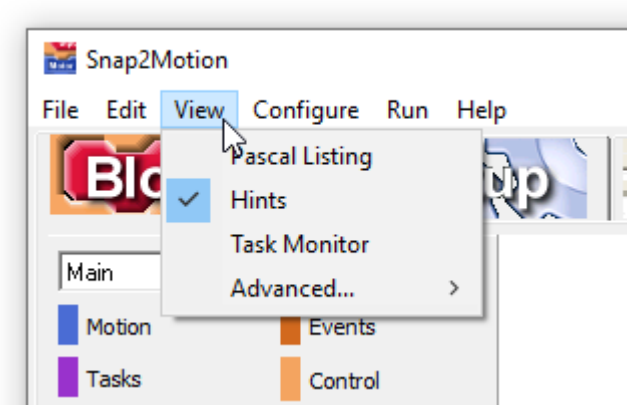
controller from a virtual terminal and sending a "cancel autostart" command while the controller is powering up to prevent the autostart routine from running. If this problem is encountered please contact support for assistance on the specific details for the specific controller involved. Note that once Snap2Motion communication is restored it is best to immediately replace the faulty autostart program with a "blank page" autostart application which effectively cancels autostart.

If the "Compile To Flash" setting is active the program saved in the controller will only be retained if the last operation performed was "Save In Controller". If after saving the program is run again with the Green Play icon then the autostart program is erased and the controller will not autostart. Make sure in this case that the last action is "Save In Controller"

View

Menu Description

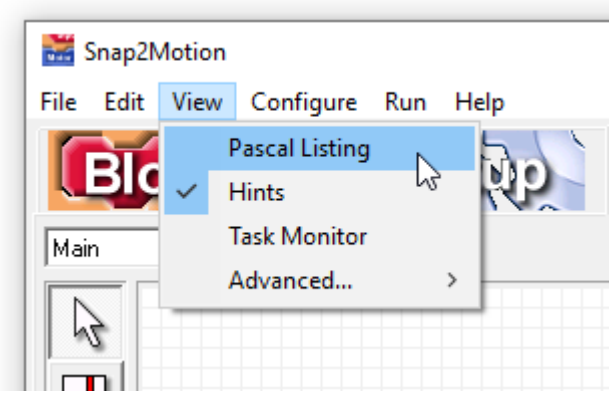
The View Menu relates to content that is displayed and the control of appearance.



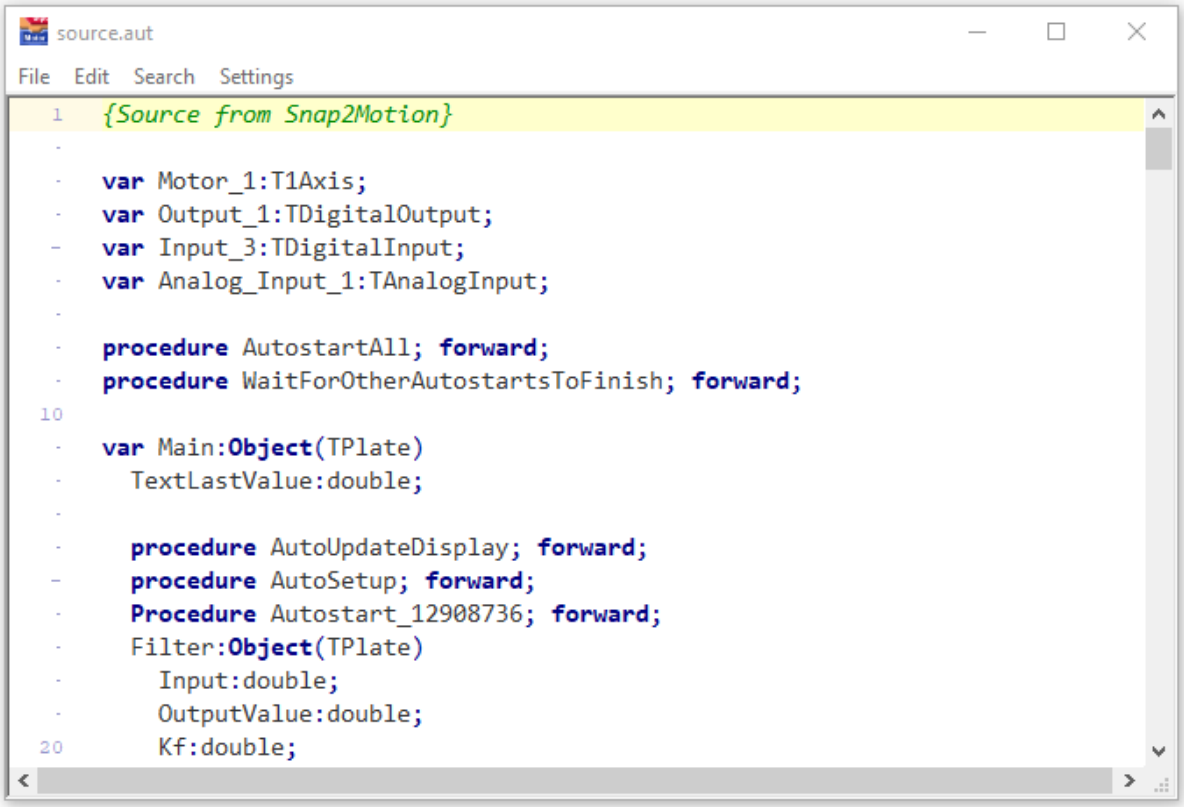
Pascal Listing

View Menu

The Pascal Listing menu item shows the total resulting source program for the project.



The beginning of a listing looks like this:



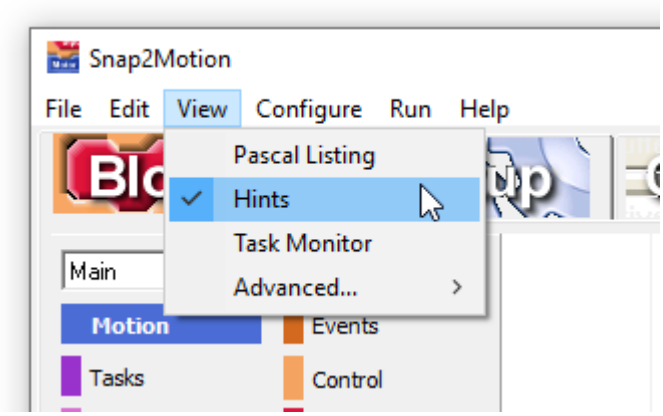
This source listing includes programming expressed in text, blocks, software packages, and additional programming generated by Snap2Motion to sustain display operation and events. This listing is useful for doing global searches throughout the entire project. Note that this is just a listing, Changes made to the program here have no effect on the actual project.

There are some cases where compiler problems are indicated by line number rather than in the programming context of a block workpage or text editor. If an error is indicated in "source.aut" at a certain line and row number then consult the pascal listing to see the problem and gain insight. Errors reported in this way usually have to do with improper naming (i.e. text programming and using a procedure name that has spaces). Note that the problem might be on a line above or below the one indicated.

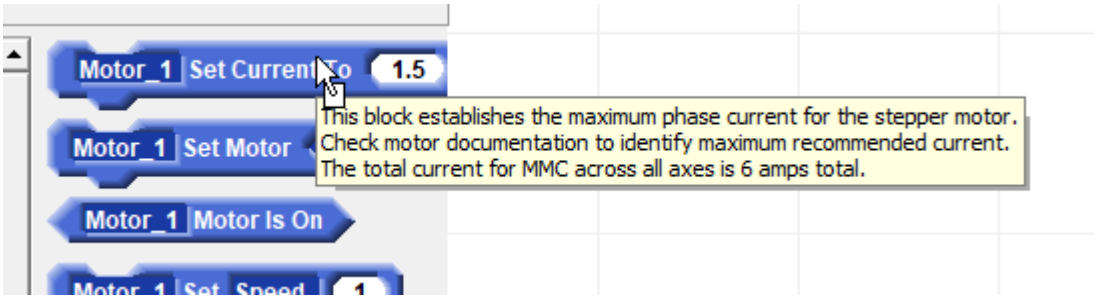
Hints

View Menu

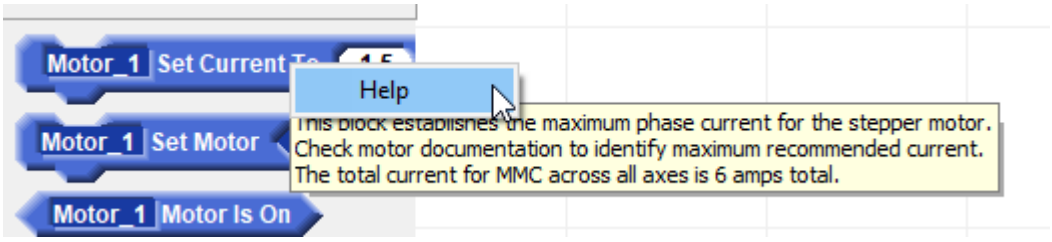
The Hints menu item has a checkbox which can be used to show or hide hover comments on many of the controls and blocks.



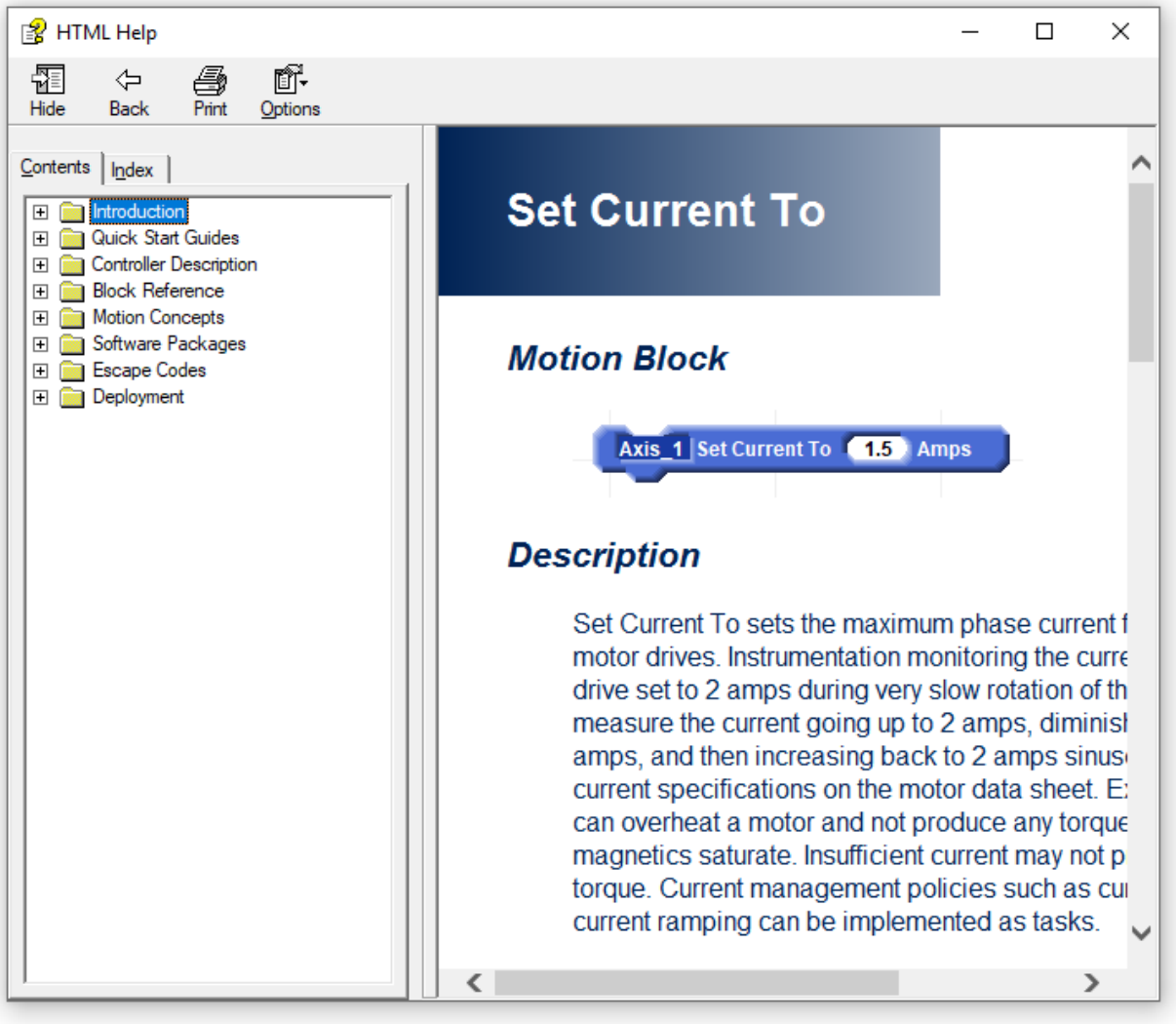
An example of a hint for a motion block is shown here:



Some hints may be verbose enough to fully describe a block or choice. If the information provided is not enough for a block then right clock the block and chose "Help":



This opens up the help system to a page with additional information and related topic links.

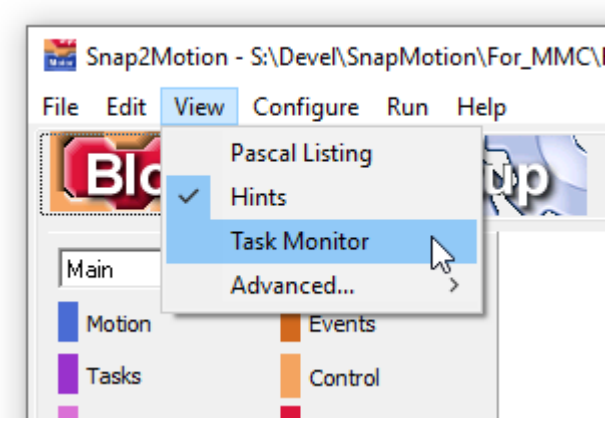


Hints are helpful to get oriented but some experienced users find them annoying. Unchecking the hint checkmark prevents the hints from showing.

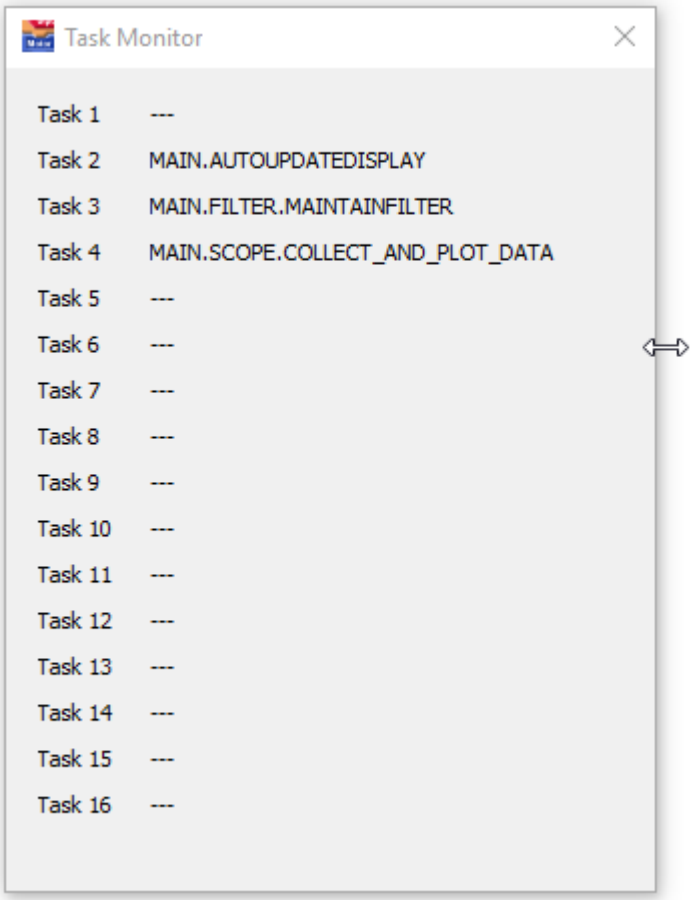
Task Monitor

View Menu

The Task Monitor menu item displays a dialog showing the active tasks currently running in the controller:



The display has 16 "slots" showing task names of active tasks:



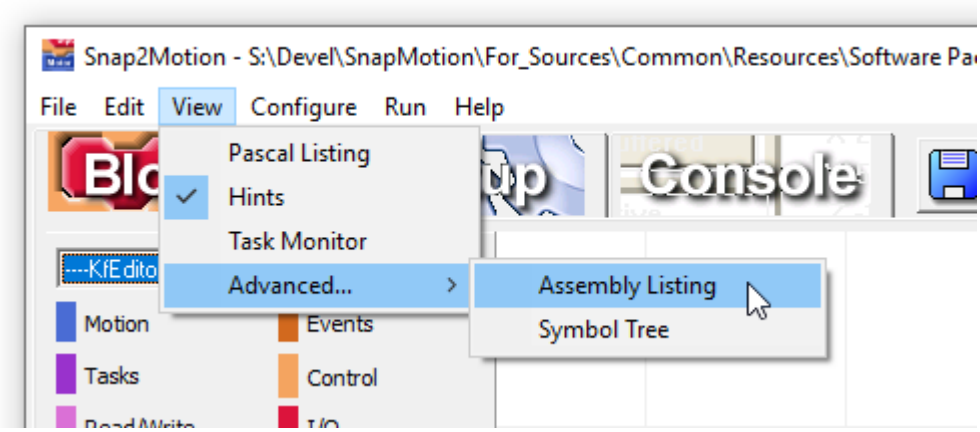
The display can provide insight regarding how many tasks slots remain available for use. Knowing that tasks are active can also help diagnostics. The task monitor can remain open while a program is running showing tasks starting and stopping in the course of application operation.

Tasks operate cooperatively with each task running until a "yield" where control goes to the next task. Tasks make decisions, act on the decisions, and spend most of their time waiting for physical events to occur. Each task has an opportunity to run every controller sample period.

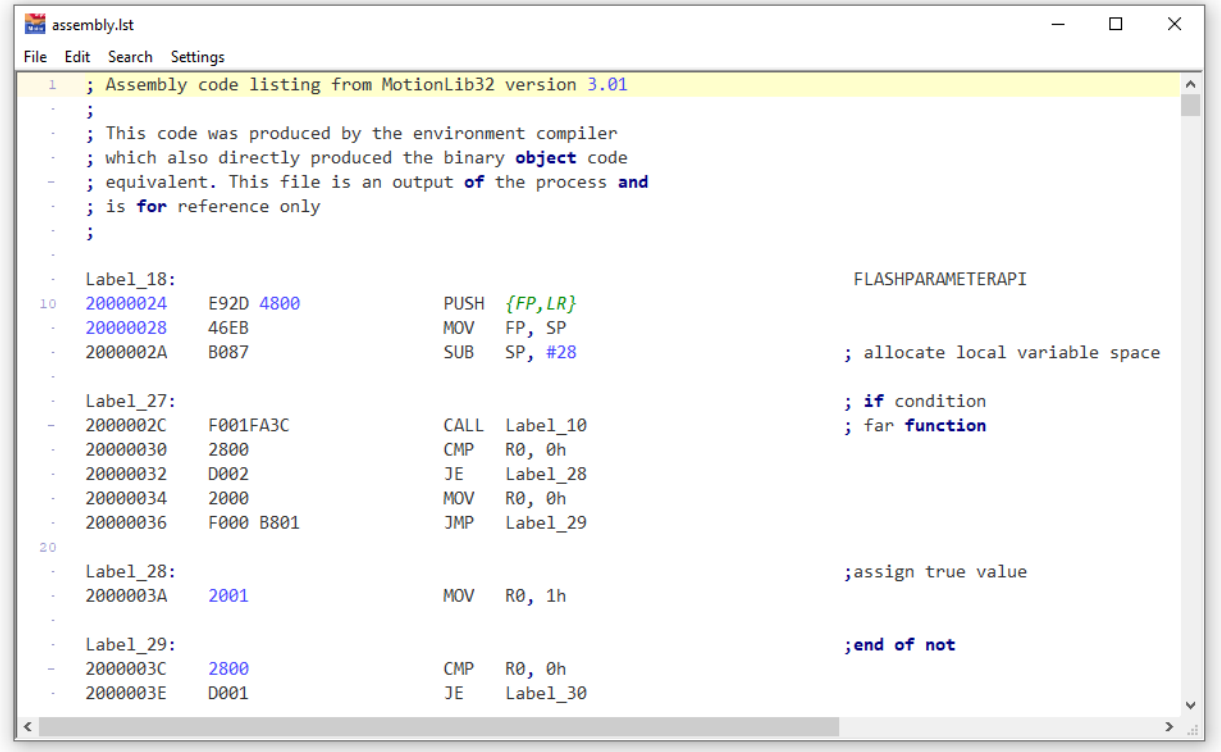
Assembly Listing

View | Advanced Menu

Under the Advanced menu item is Assembly Listing menu item:

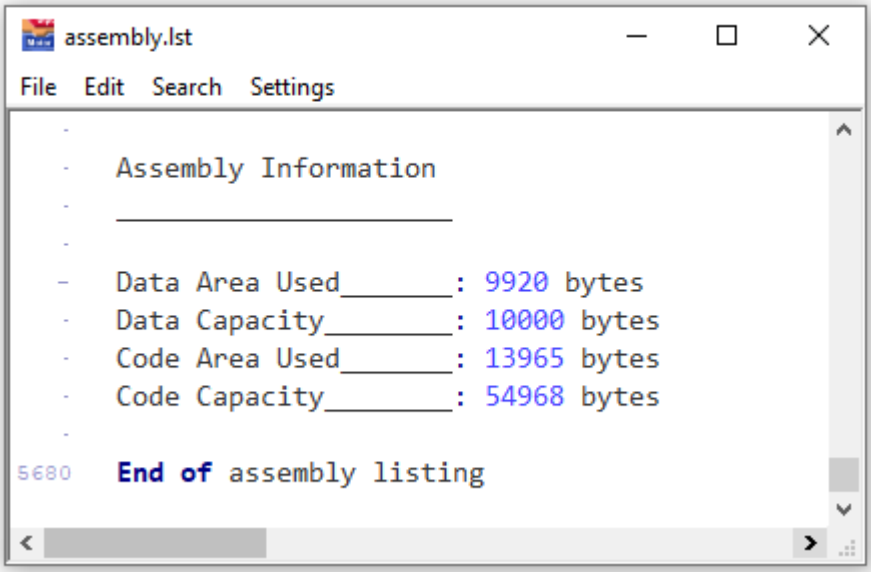


The assembly listing shows the low level description of the project that runs directly on the processor. This is an example of an assembly listing:

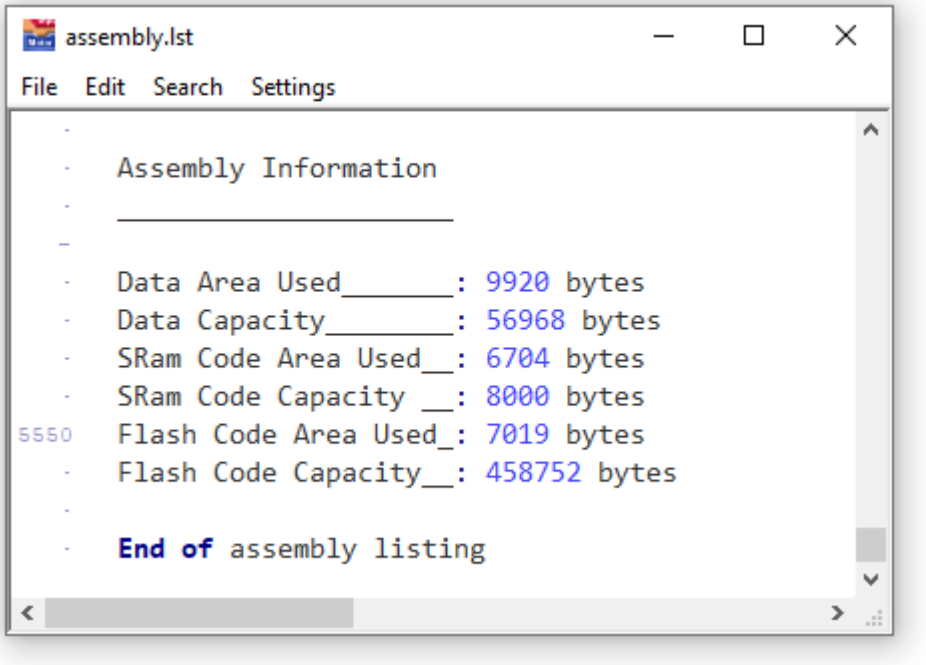


The assembly listing will reflect the last project run, not the last project opened. If no project has been run this error will be shown:

At the end of the assembly listing is a summary of the resources used:



This project was made with "Compile To Flash" off. Most of the data has been used (scope instrumentation in the project) and about 25% of the code capacity. Turning on Compile To Flash and re-running the project produces these results

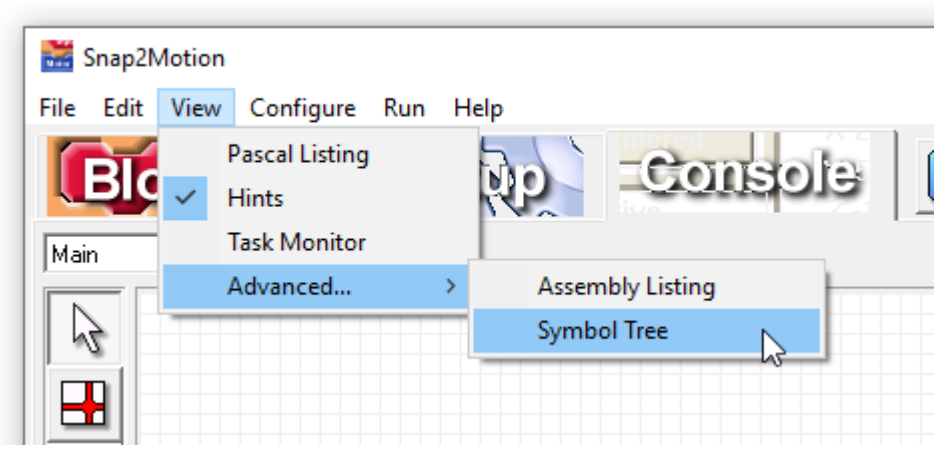


In this configuration only 17 percent of the data capacity and 2% of the code capacity is used. Learn more about the Compile To Flash Menu Item to judge the tradeoffs.

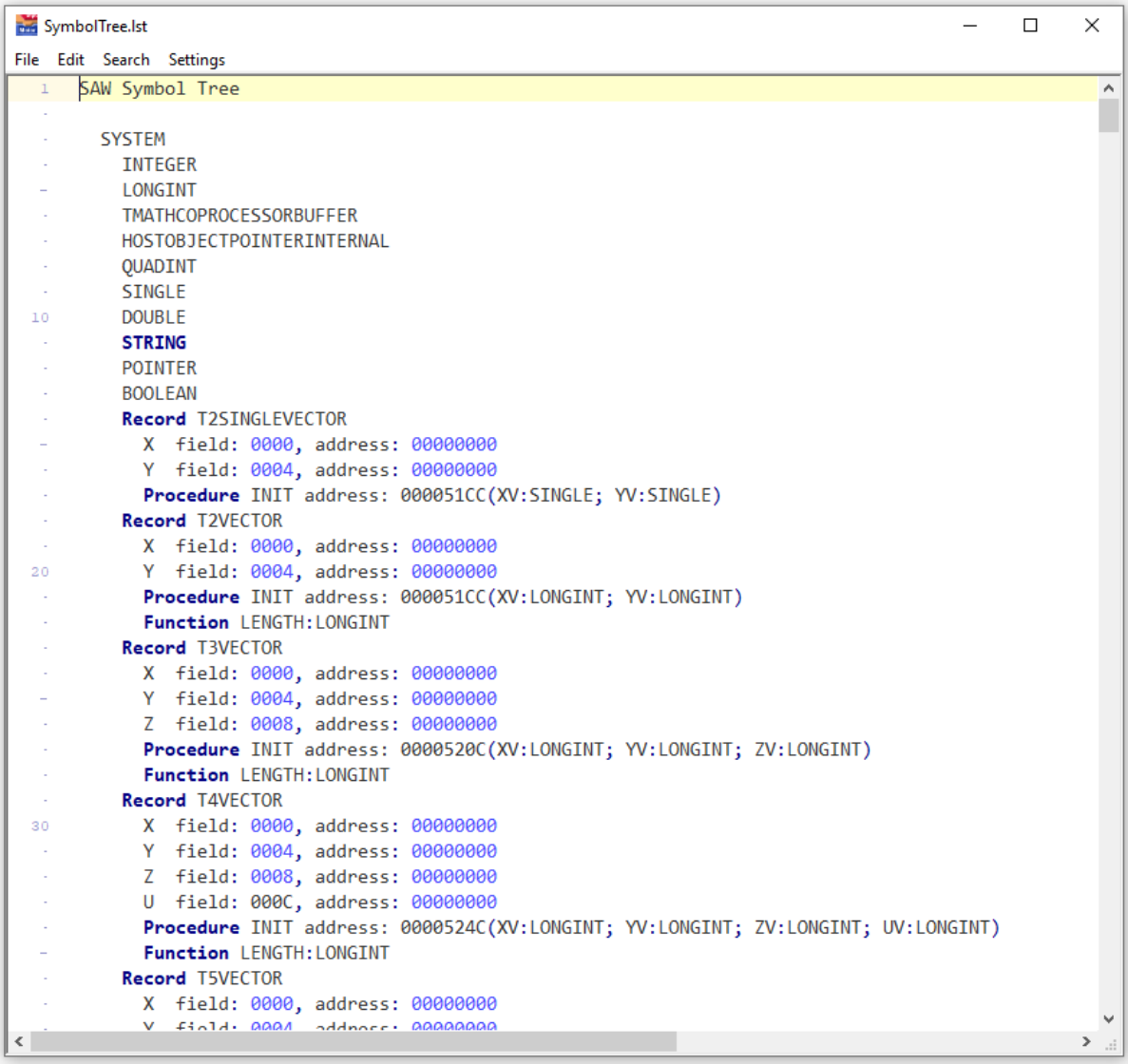
Symbol Tree

View | Advanced Menu

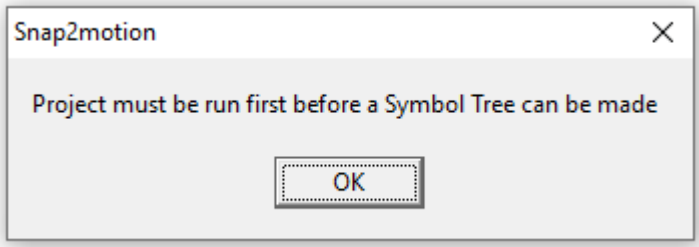
Under the Advanced menu itme is Symbol Tree menu item:



The Symbol Tree is a list of all symbols available to the compiler and present in the project. This list might be consulted when trying to understand why a command is not recognized. The following is an example of a Symbol Tree:



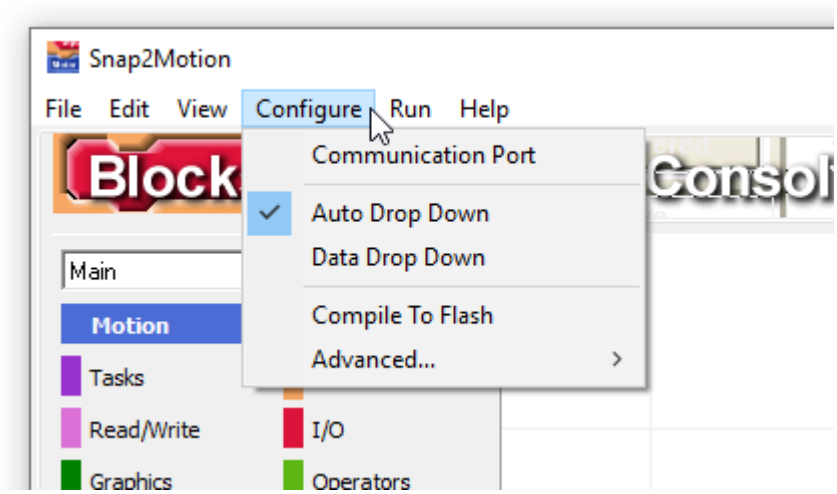
The Symbol Tree is only available after a project has been run. If the project has not been run this error message will be shown:



Configure

Menu Description

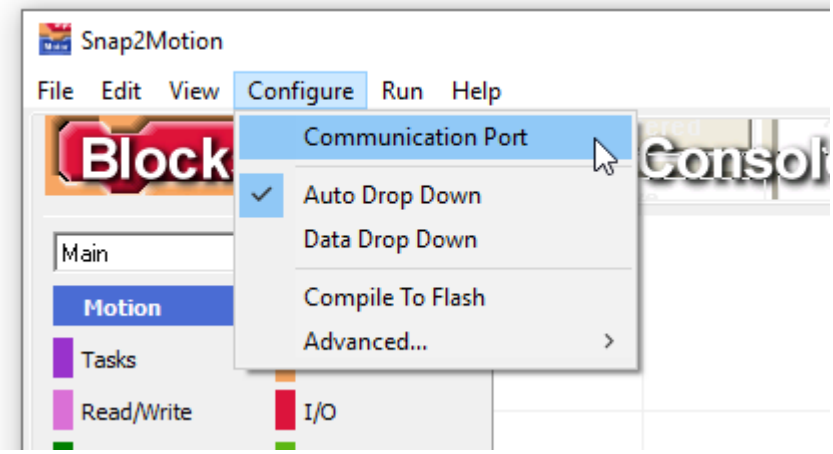
The Configure Menu adjusts project and environment settings



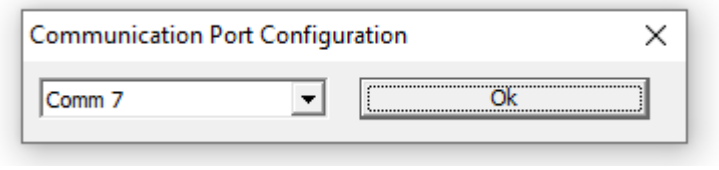
Communication Port

Configure Menu

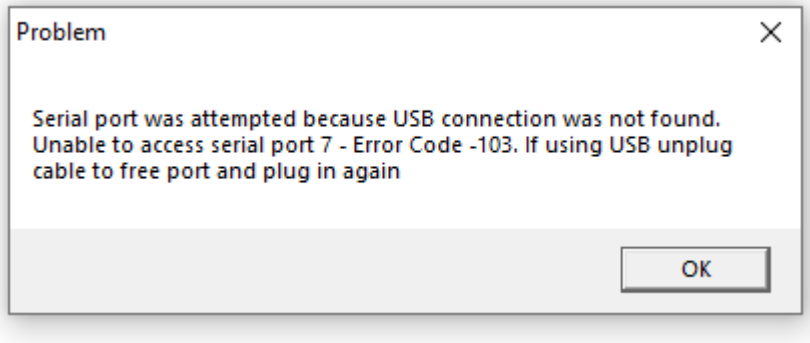
Under the Configure menu item is Communication Port menu item:



Selecting this produces a dialog to identify a PC comm port



Dings Motion products all have USB ports which is the default port. However some legacy products have serial ports. It's also possible on some products to swap the USB port for the AUX RS232 port allowing a host computer to use USB for host communication. To support these possibilities a comm port choice is made avaiable. As the default communication method USB will always be tried first. If a USB cable is not connected to a controller the system will try the port indicated in this dialog. If neither USB nor a Comm port connection is available this message will be displayed:

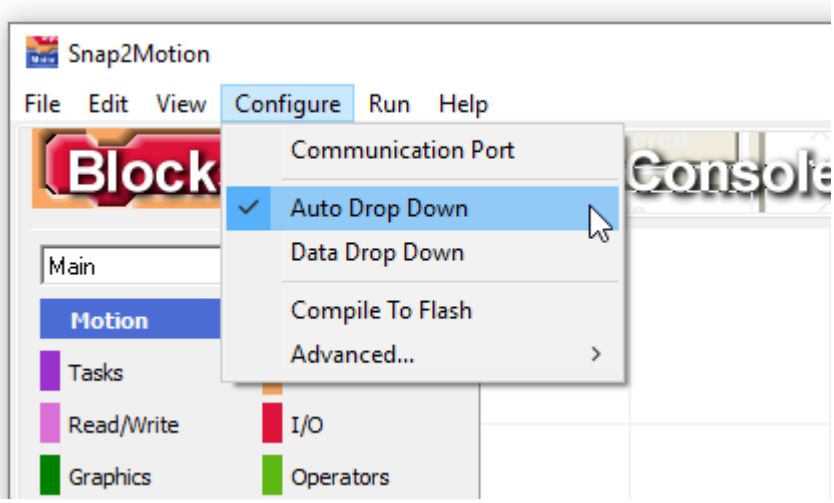


In unusual cases, such as premature termination of the program, the USB port may remain unreleased and unavalailable. Try removing the USB cable, waiting a few seconds, plugging in and trying again.

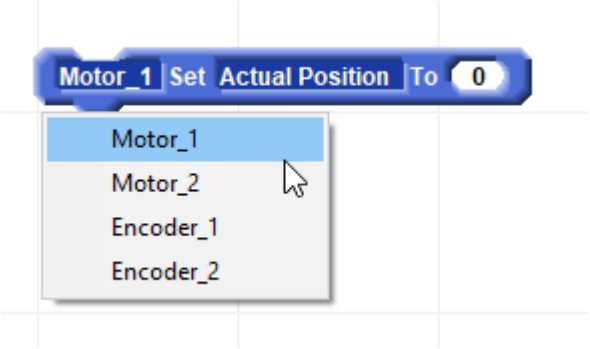
Auto Drop Down

Configure Menu

The Auto Drop Down menu item presents all list choices when placing a new block on a block workpage.

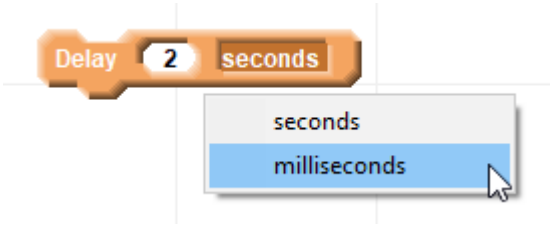


When Auto Drop Down is active a block dragged from the selection on the left onto a workpage will progressively show list choices where choices are available:



After a selection is made the next list will show its choices:

Subsequent duplication of the block will not produce the lists again automatically although they can always be clicked on to revise a choice. Auto Drop Down is useful because it compels the programmer to make a list item selection. It is easy to miss a choice particularly regarding blocks that have units such as the delay block:

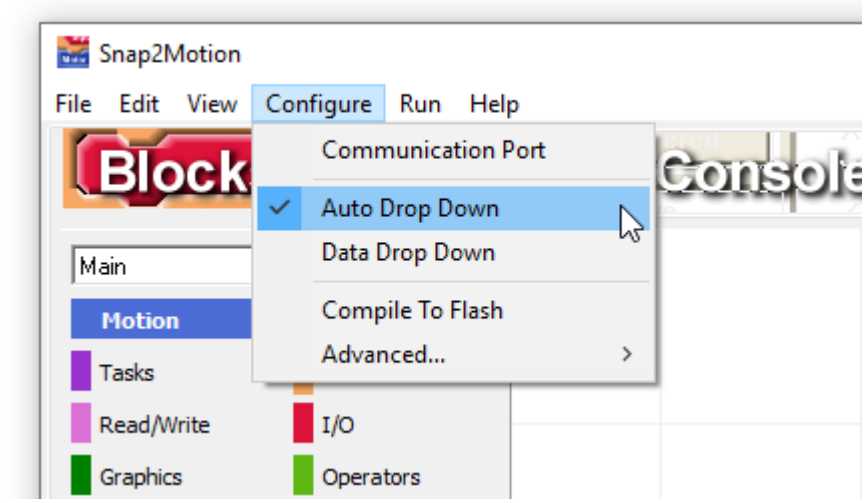


While programming the concern might be on the numeric value and attention not paid to the units field. Time can be lost wondering why no action in a program is occurring only to discover that the time units has been left undistrubed on "seconds" when the programmer's intent is to have "milliseconds". Autodrop down insures a deliberate choice is made. However some find the additional clicks distracting so the feature can be disabled. The Auto Drop Down settings is a property of the development tool and not any particular project.

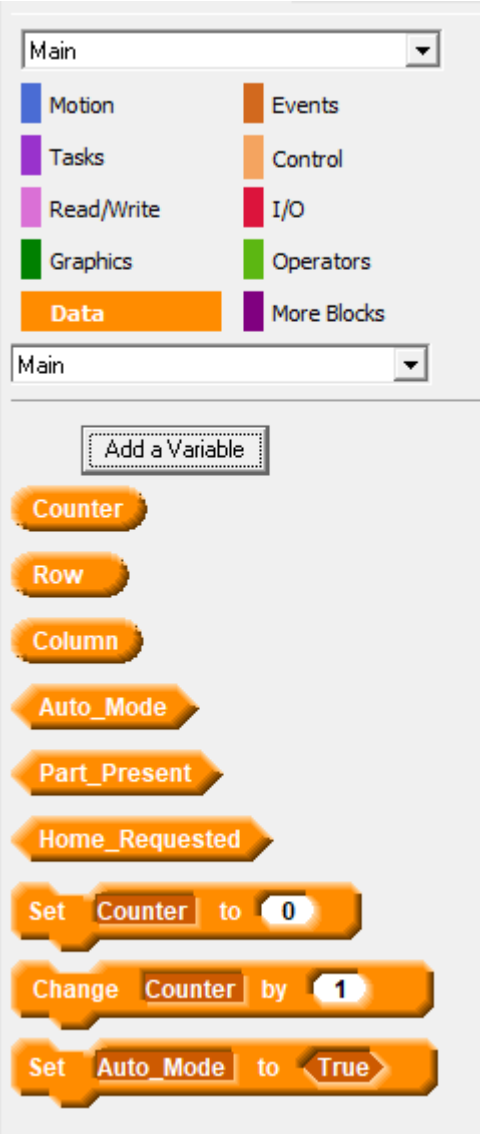
Data Drop Down

Configure Menu

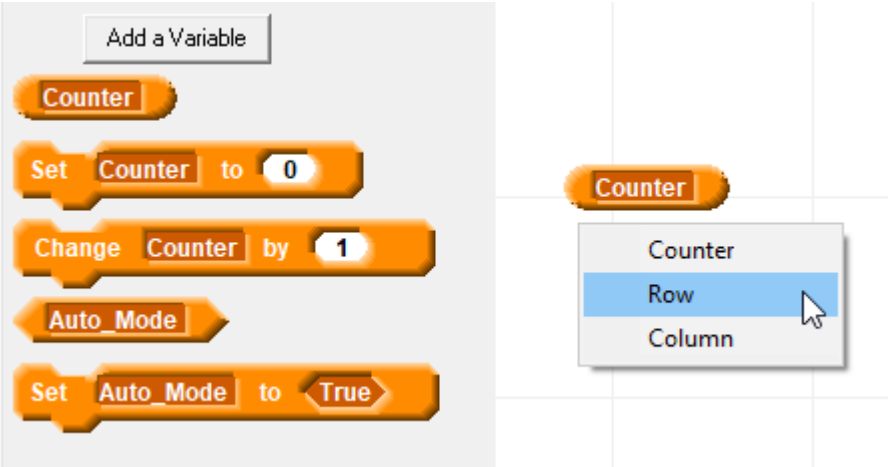
The Data Drop Down menu item provides all variable names as list within a single bubble or diamond instead of as a list of bubbles or diamonds.



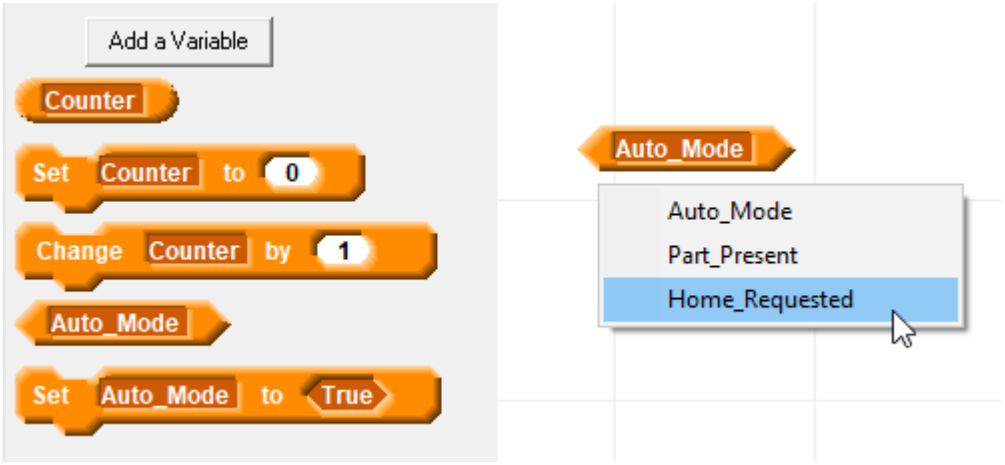
When Data Drop Down is inactive (the default condition) data blocks showon under the Blocks data category are shown individually:



If Data Drop Down is made active the variable names are presented as a list within a bubble for numbers:

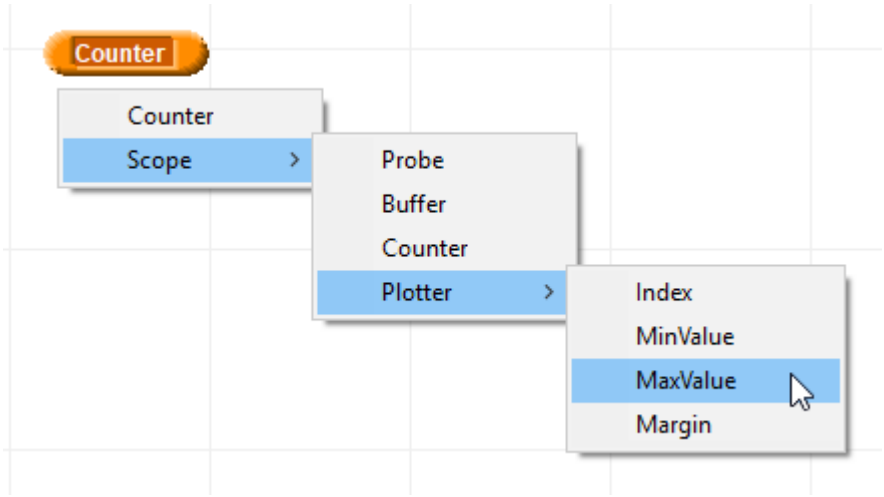


Boolean variables are also presented in a list within a diamond:



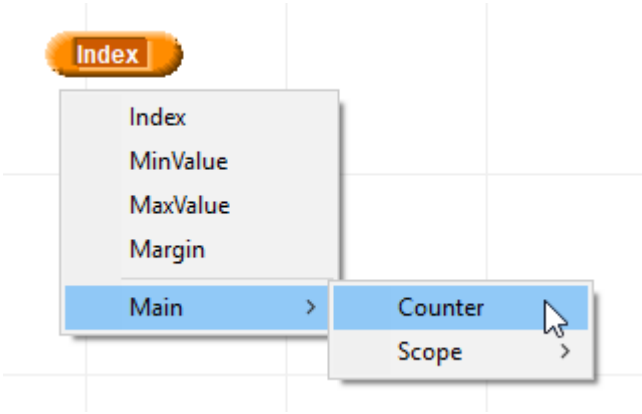
In small projects it's convenient to not have data drop down. The set of variables can be viewed and the desired one chosen in a single drag movement. However for bigger block projects that have many variables it's more convenient to use the Data Drop Down option and not have to scroll through the list of blocks to find the desired variable.

If a project includes software packages that themselves have block accessible variables these can be chosen from the data dropdown by following the software package names:



The resulting bubble or diamond shows the entire variable name path:

If a variable is being added to a child package variables in the parent package are available for selection:

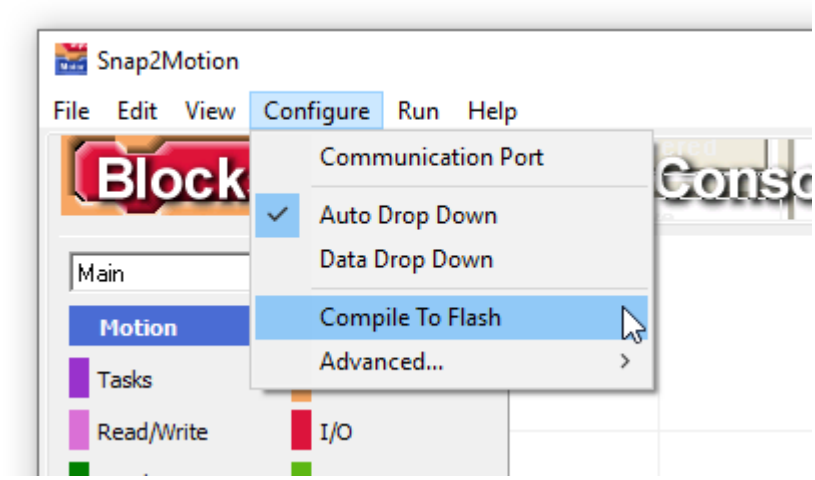


Good design practice suggests that variables should be private to a software package and that access to information handled through a procedures and function interface although this direct access is provided.

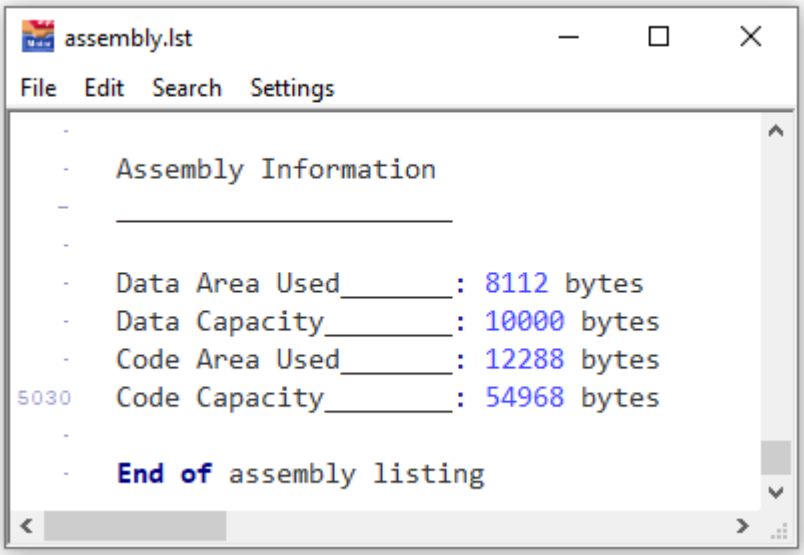
Compile To Flash

Configure Menu

The Compile To Flash menu item allows a choice between saved program retention when other programs are run (Compile To Ram) and program capacity (Compile To Flash).

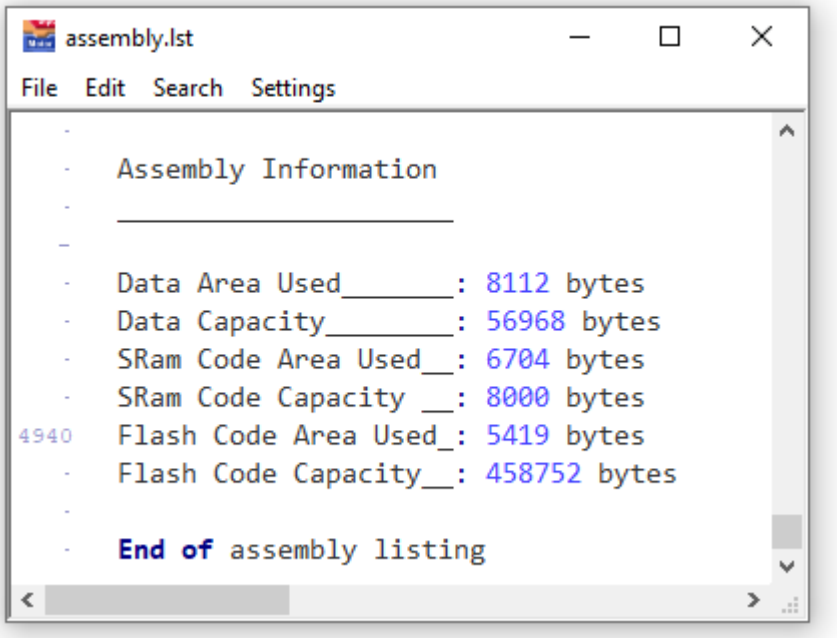


When Compile To Flash is inactive programs are compiled into RAM (Random Access Memory). The menu choice "File | Save In Controller" still saves the program in an area of flash so the controller can autostart. However a saved program with Compile To Flash inactive remains saved even if other programs are run. When the controller is turned off and on again, it will autostart the last program saved in the controller regardless of what other programs might have been run in between. This is convenient since running diagnostic programs does not distrub the configuration of the controller for its deployed purpose. However there is much less RAM available than Flash memory so this only works on smaller projects. This resource listing, viewed by choosing "View | Advanced | Assembly Listing" and scrolling to the bottom shows resources typically available when Compile To Flash is inactive:



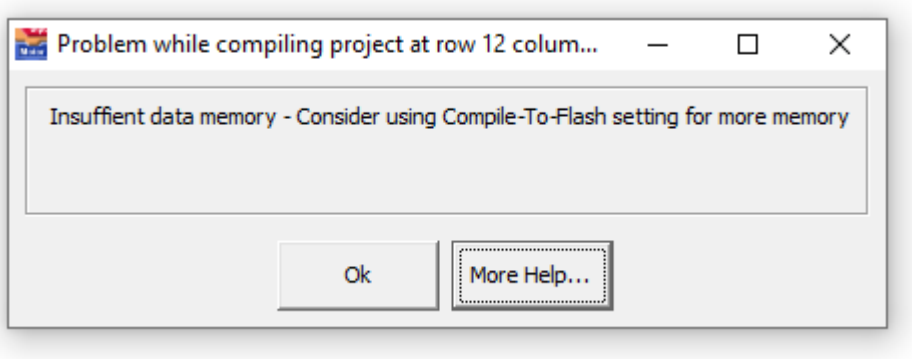
In this particular case there are 10000 bytes of data and about 55000 bytes for program code.

If Compile To Flash is active then RAM can be largely committed to data and the larger Flash memory area is available for the program. The tradeoff for this additional capacity is that a program that is saved in the controller will not be retained if another program is run. The menu choice "File | Save In Controller" must be the last action performed on a controller prior to deployment. If another program, such as a diagnostic program, is run then the deployed application must be run again and saved into the controller again. This resource summary is from the same controller and project but with Compile To Flash active:

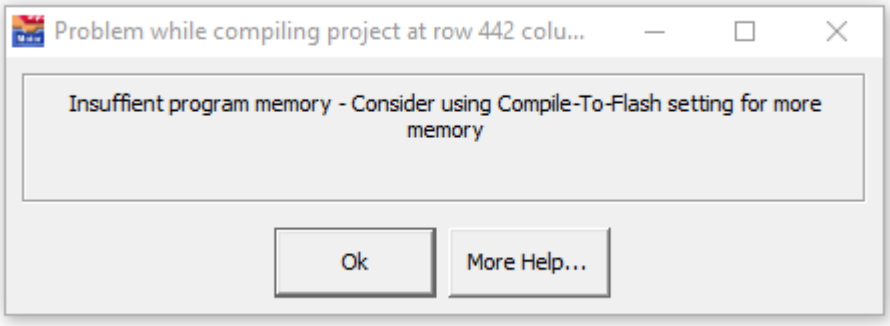


Now there are almost 57000 of data available (a more than 5 fold improvement) and 459000 bytes for the program (about an 8 fold improvement). Although small compared to a PC those resources would generally be sufficient for several man-years of project development.

If there is insufficient data memory this dialog will appear:



If there is insufficient program memory this dialog will appear:

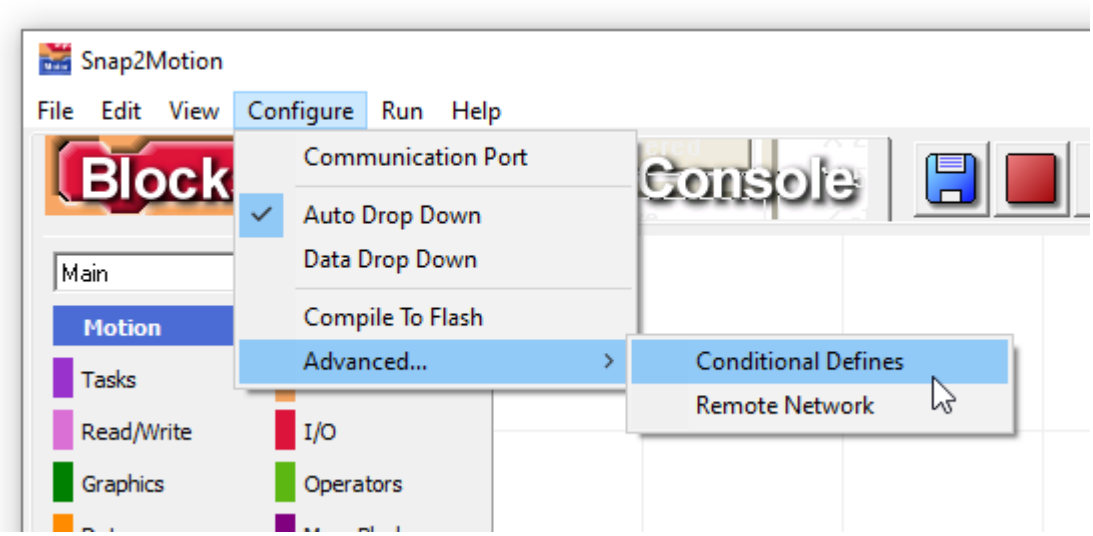


An editor may open trying to show where the program exceeded available memory.

Conditional Defines

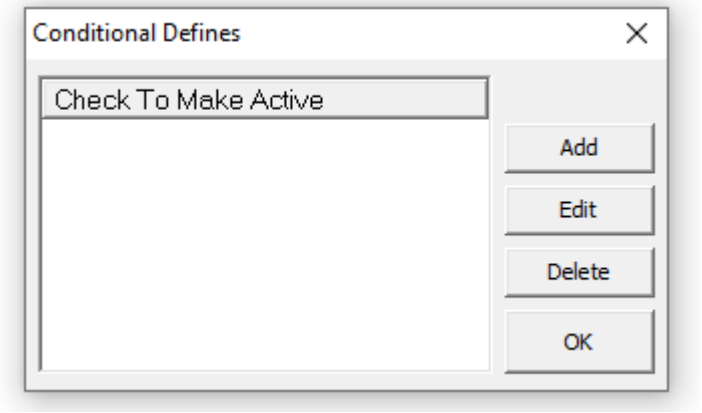
Configure Menu | Advanced

The Conditional Defines menu items is an advanced feature allowing for a convenient way to include or exclude sections of a program during the compilation process.

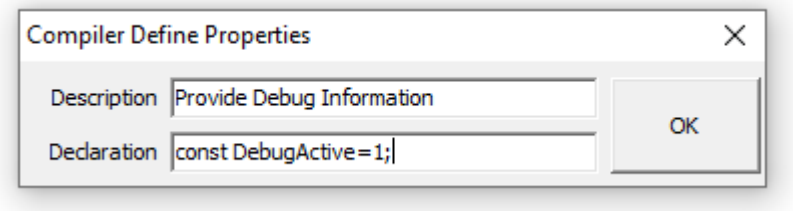


A common use of this feature might to include or exclude debugging statements and mechanisms. Conditional Defines are a property of a project. A saved project will retain the Conditional Define settings when the project is saved and restore them when the project is loaded again.

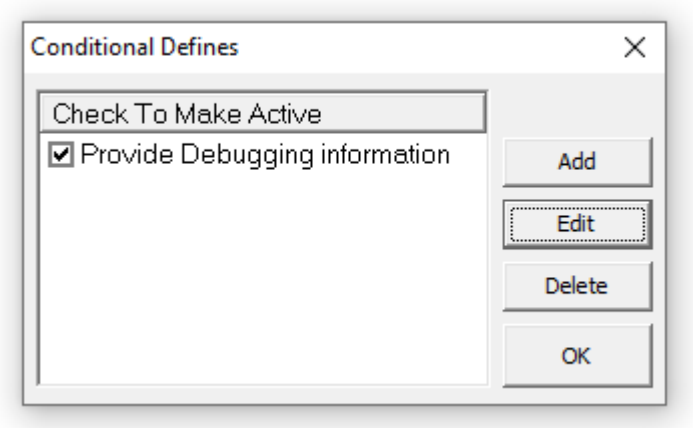
When the Conditional Defines menu item is selected a Conditional Defines dialog is presented:



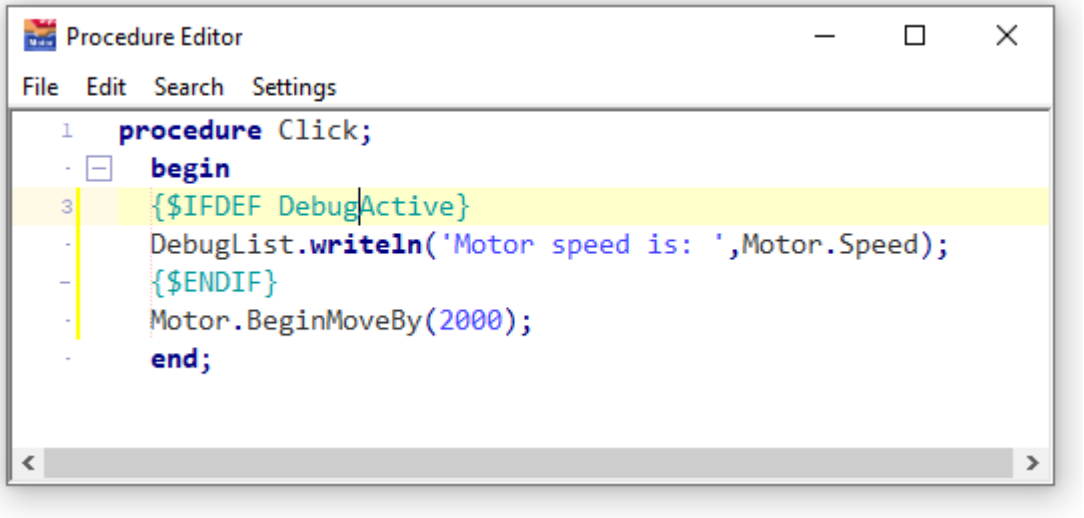
The "Add" button presents a Conditional Define Property Editor:



The Description field represents the Conditional Define in the Conditional Defines Dialog list. The description is free to have spaces. The Declaration field contains a text programming statement that will be compiled before any other part of the program. Conditional compiler directives can then reference the declared symbol to include or exclude program lines. For example consider that debugging statements are needed to track down a problem in text programming. The above Conditional Define looks like this after the "Ok" button is clicked:

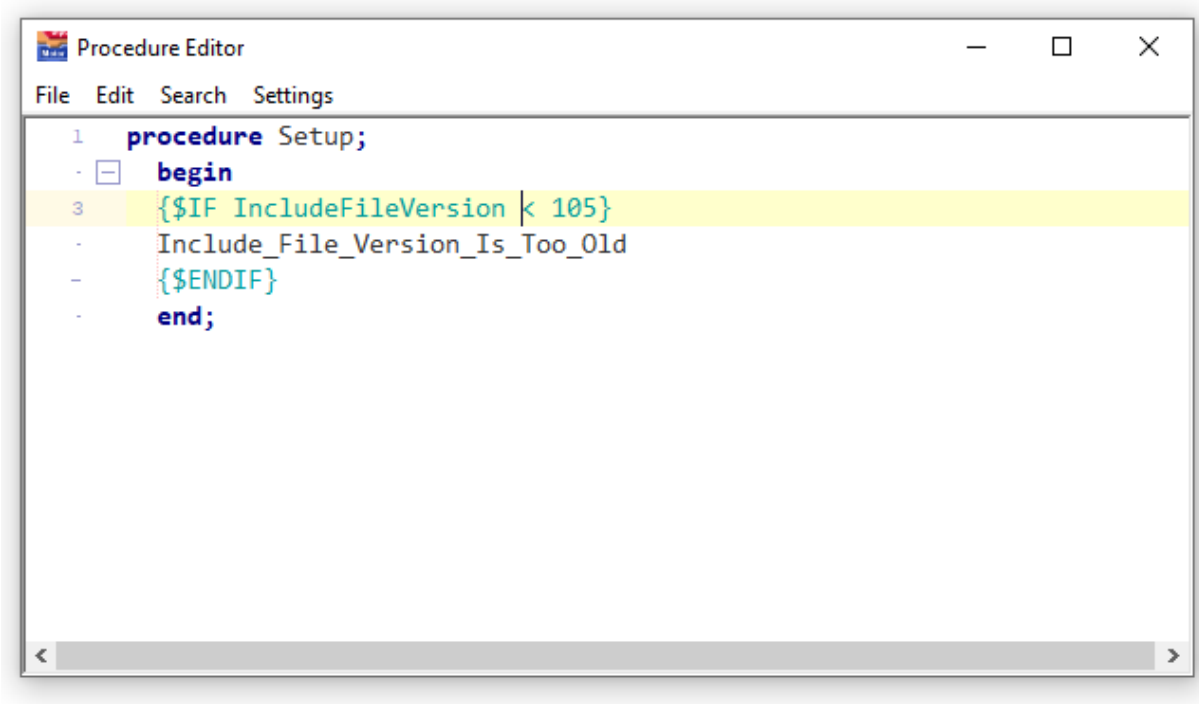


If the checkbox is checked then the associated declaration is included in the compilation. Text programs can then reference DebugActive like this:

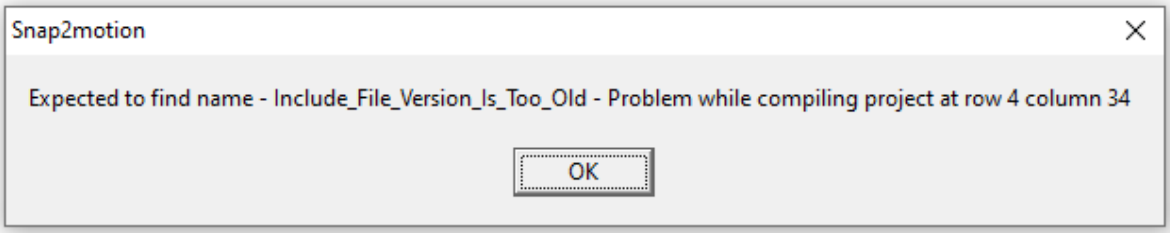


If the checkbox is unchecked then this Conditional Define declaration is not present in the compilation. The conditional program lines will not be included. The lines can be left in the program without any detrimental effect waiting for some time in the future when the diagnostics are re-instated by again checking the Conditional Define. Note that because the conditional is {$IFDEF} - If Defined - the conditional is checking if the symbol itself is defined. No question is being asked of the value in this case.

It is possible to perform conditional compilation based on the value of an expression if that expression can be determined at compile-time by using the $IF construction. Consider a project that uses an include file. The include file as well as the project will undergo revisions. To make sure that the include file being referenced is the correct version for the project a conditional like this could be used (assuming that the include file had an ordinal constant named IncludeFileVersion):

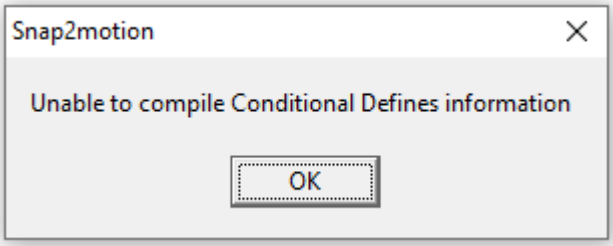


What is conditionally included is an interntional mistake which stops compilation and indicates indirectly the nature of the problem:



Value based comparisons are more restricted than general language syntax. Expressions must be ordinal variables or ordinal literals and there must be a space between each part of the expression.

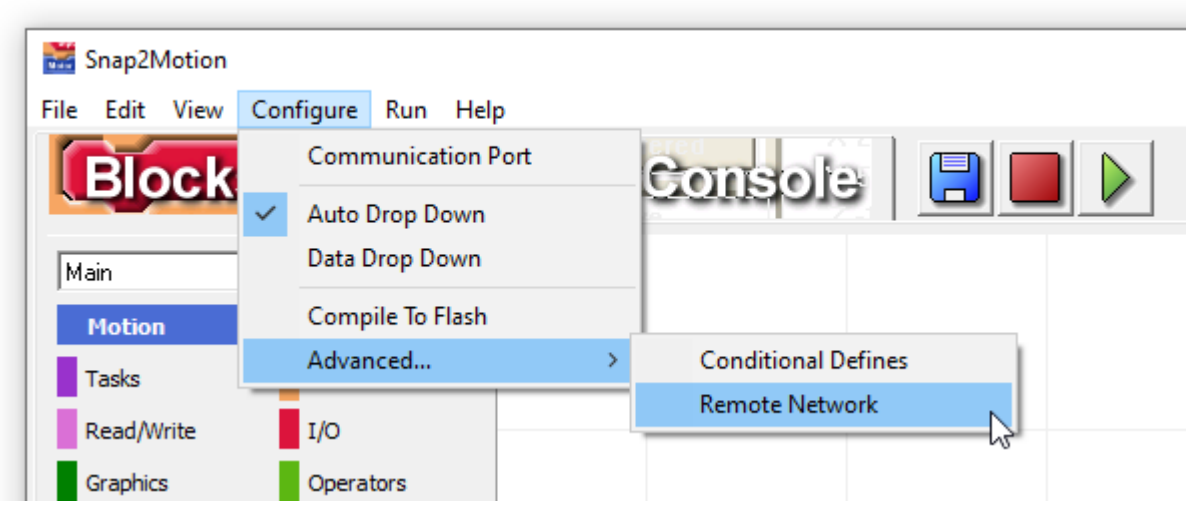
As conditional compilation is an advanced feature it is incumbant on the user to provide a declaration that is accurate. If the declaration provided cannot be compiled this message will appear:



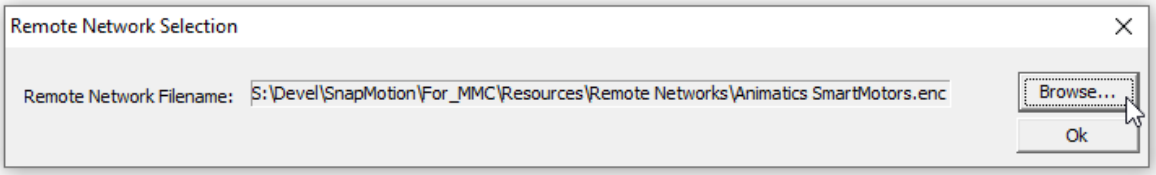
Remote Network

Configure Menu | Advanced

The Remote Network menu item provides a way to choose the type of remote network is used in the systems such as Animatics Smartmotors or the Modbus Remote Controller Protocol.



When this choice is made a display is presented showing the current Remote Network file chosen and a browse button permitting another choice.

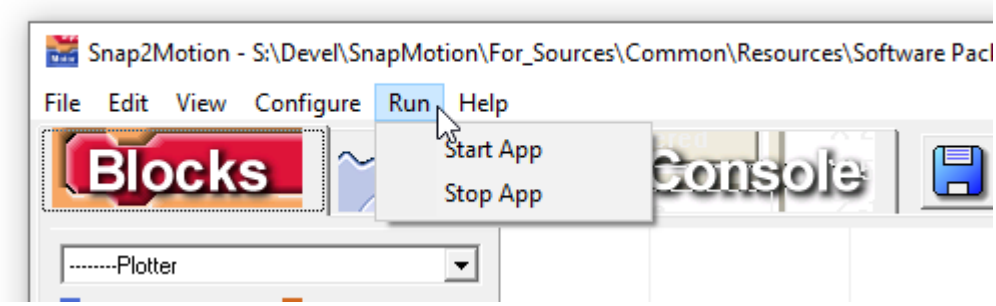


The dialog will also be presented automatically if a choice has not been made and there is an attempt to communicate to remote resources.

Run

Menu Description

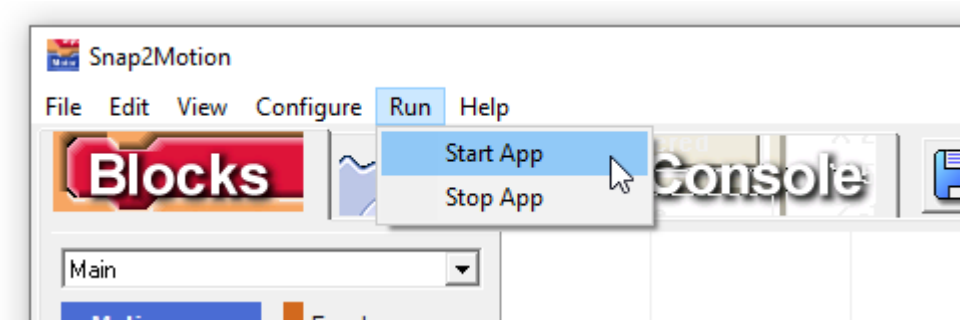
The Run Menu selections start and stop the application.



Start App

Run Menu

The Start App menu item performs the steps to run the application on the controller.



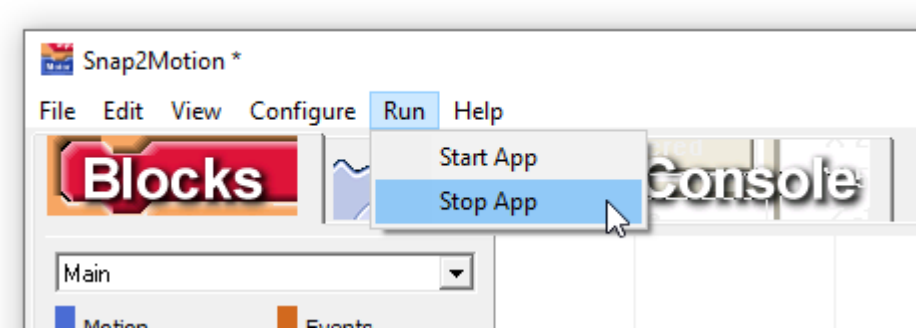
The green play button does the same thing:

Based on what design elements have been chosen additional software components are included along with block and text descriptions of the application. The application is compiled into native object code that directly runs on the controller processor. Communication links are established for the various Windows controls that may be part of a Console described HMI.

Stop App

Run Menu

The Stop App menu item stops the currently running application on the controller:



The red stop button does the same thing:

The tasks running in the controller are stopped. The default behavior is for the motors defined on the Setup tab to abort their motion and turn off.

It's possible to have alternative behavior when the stop button. The alternative behavior is described with text programming. If this is a block programming project feel free to contact technical support for text programming assistance to create an alternative behavior. When "Start App" is performed the system writes the following "SoftwareStop" procedure:

source.aut

File Edit Search Settings

```
Procedure SoftwareStop;
begin
try
  begin
  UserSoftwareStop;

  Motor_1.Abort;
  Motor_1.SetMotor(off);
  yield;

  Motor_2.Abort;
  Motor_2.SetMotor(off);
  yield;

  Encoder_1.Abort;
  Encoder_1.SetMotor(off);
  yield;

  Encoder_2.Abort;
  Encoder_2.SetMotor(off);
  end
recover
  begin
  end;
end;
```

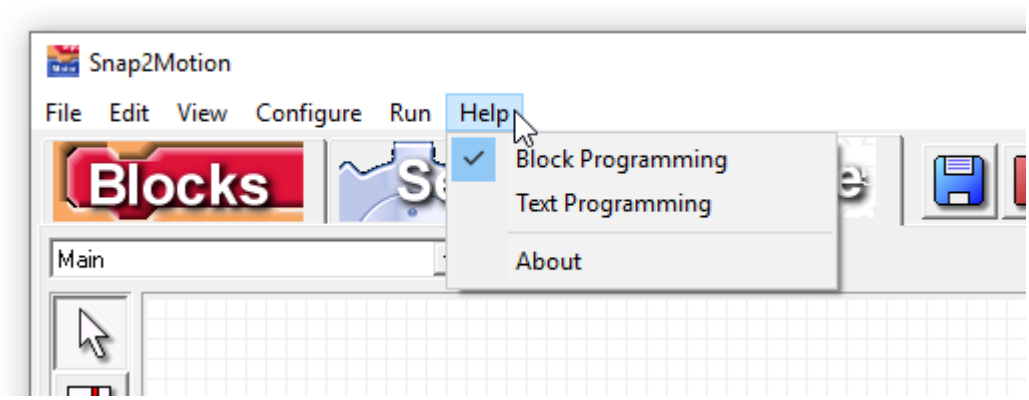
At the top of this routine is "UserSoftwareStop". This routine is defined in the standard.inc file that's part of the software installation. This routine can be changed to do what is required. After the UserSoftwareStop procedure all of the axes defined in Setup have their motion aborted and are turned off. If shutting down

motion and motors is not desired then the UserSoftwareStop routine can conclude by escaping. This will cause the commands in SoftwareStop to be skipped. The standard.inc file is not aware of names used on the setup page but there are other names available to reference the motors. Contact support for assistance if this customization is required.

Help

Menu Description

The Help Menu provides a list of help documents available:

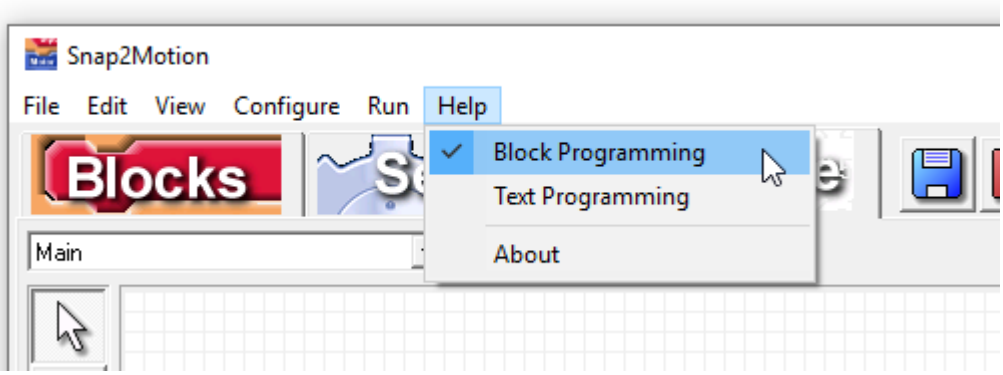


To make the help system more direct help documents are product specific and "Method Of Use" specific. If a different controller is connected the help documents provided will change to the ones that relate to the connected controller.

Block Help

Help Menu

The Block Programming menu item both selects the default method of receiving help and opens the documentation pertaining to block style programming.

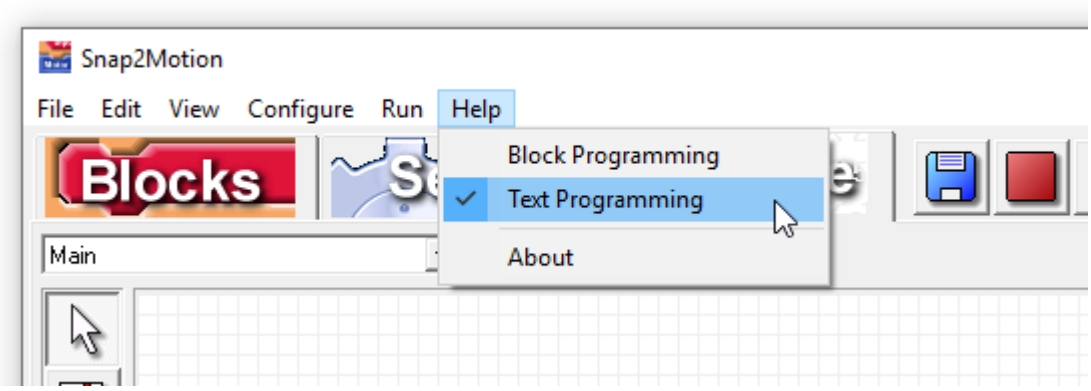


The help system can be entered by clicking on this menu item or at the system's initiative to provide additional information. The explanation will be given in the form indicated by the checkmark.

Text Help

Help Menu

The Text Programming menu item both selects the default method of receiving help and opens the documentation pertaining to text style programming.

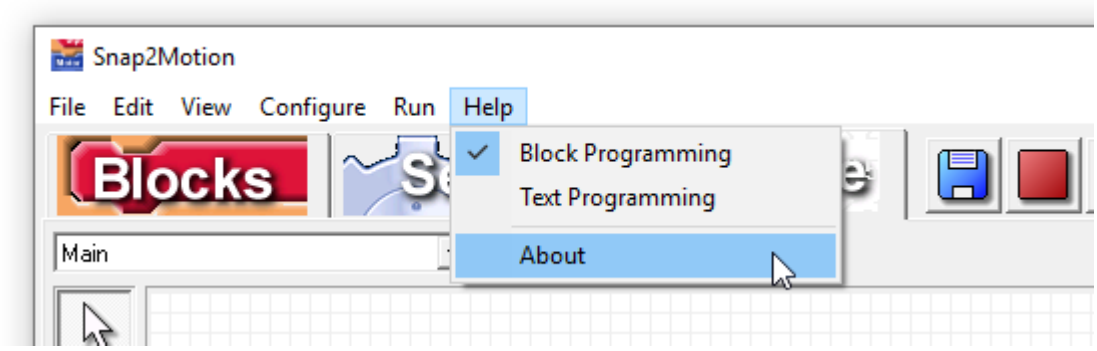


The help system can be entered by clicking on this menu item or at the system's initiative to provide additional information. The explanation will be given in the form indicated by the checkmark.

About

Help Menu

The About menu item provides version information about the software:



If technical support requests the version number of the software it is shown in the resulting dialog.

Protocol Description

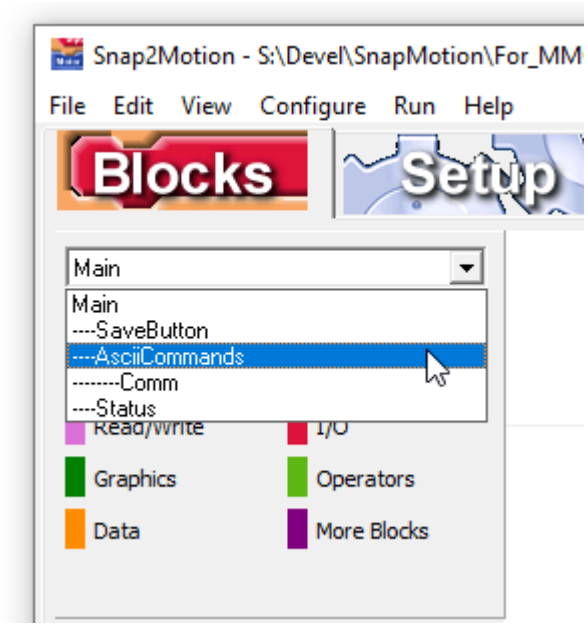
ASCII Command Protocol

This describes how characters are sent to the controller and what command responses to expect.

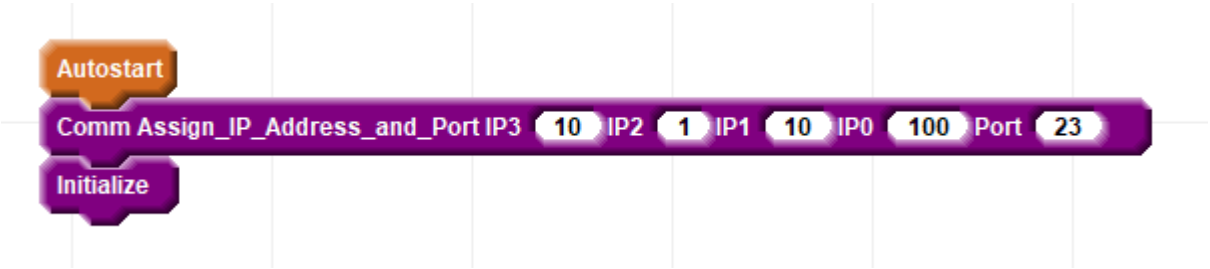
Port Configuration

TCP/IP Configuration

TCP/IP Port Configuration is performed in Snap2Motion. Follow the instructions in Controller Preparation to find the proper program for the Ascii Commands over TCP/IP. Select the "Blocks" Tab and then select "Ascii Commands" from the upper left list:



A block list is provided where an IP address can be set:



Choose an IP subnet that is common with the other Ethernet devices on your network. Port Number 23 is the Telnet port. Telnet is termnal application allowing for manual testing of the controller. Putty, another popular terminal program, can also be used

After making the necessary changes continue following the instructions in Controller Preparation to run and save the project in the controller.

Many times initialization activity occurs in the main form. In this case the initialization for Ascii Commands is being done in the ASCII Commands package itself making the package a completely configured component able to be placed into any project to endow it with the ability to handle ASCII commands.

Procedure Message Format

Description

In general messages involve 3 letter commands. Some commands are in the form

<procedure> <parameters> <CR>

The Ascii Command Interpreter is a Snap2Motion application and can be tailored. If the format or prompt at the end is not desired the project can be edited to produce an alternate response. Check with the factory for details.

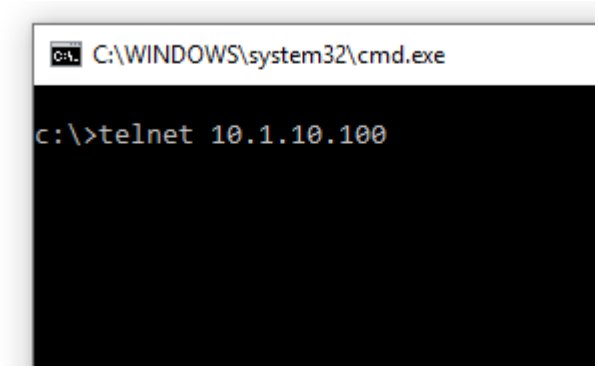
There might be no parameters, one, or several depending on the function. Parameters are seperated by spaces. The space between the procedure and the first parameter is optional. Adding a space im- proves readability. The command is terminated with a carriage return (ASCII 13) character.

After receiving a command, the controller will transmit the following information:

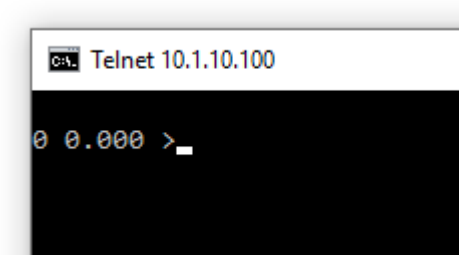
<CRLF> <error number> <return value> <prompt>

The prompt is the "greater than" symbol. The error number indicates if the command was completed successfully. A "0" indicates a success. Other numbers represent errors. If the first number is not 0, the remaining information in the response is not present. After the error number is a space followed by the return value of the command as a floating point formatted value. All commands have return values. Most return values reflect controller state. Some return values confirm information that was sent when no specific return value is associated with the command. The response terminates with the "greater than" symbol as the default prompt.

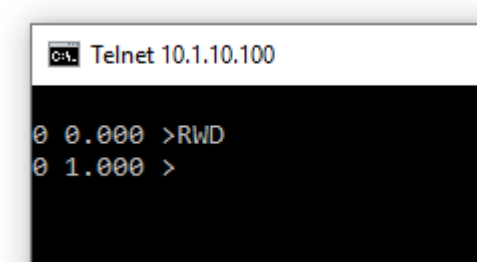
Hooking up a terminal, or using a terminal program on the PC such as Telnet for TCP/IP communication permits exercising the board and viewing this transaction behavior. Telnet comes with Windows but usually involves an additional setup step. Google "install telnet" for additional details on installing Telnet. Telnet is started from the command line with the IP address as a parameter:



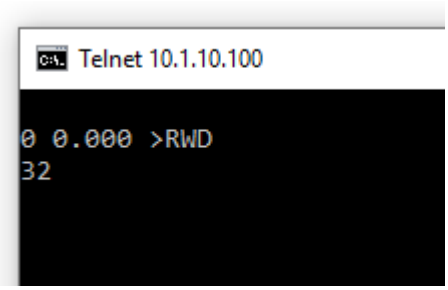
Typing just the "CR" key produces a null command which returns 0 (ordinal) for error and 0.000 (float) for return value:



The command RWD (Reset Watchdog) which has no parameters can be typed with the following example response:

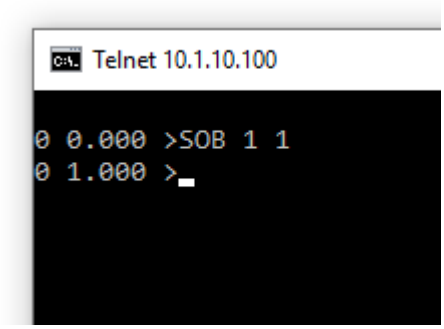


The error is "0" indicating the command succeeded. The value number, in this particular case, reports the watchdog has tripped status that was found prior to the reset. If the EStop connector is removed and the command is performed this is the response:



Error 32 is reported, Watchdog Failed To Reset Escape Code. There is not a value number following because the command failed.

An example of a command which requires parameters would be the output command, SetOutputBit (SOB). The first parameter is the bit number and the second the desired output value.



Parameters are separated by spaces or commas.

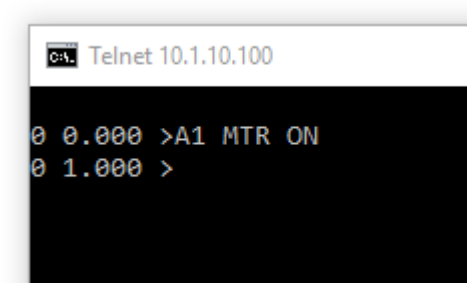
Object Message Format

Description

Another message form reflects the "object oriented" nature of the software and has the following form:

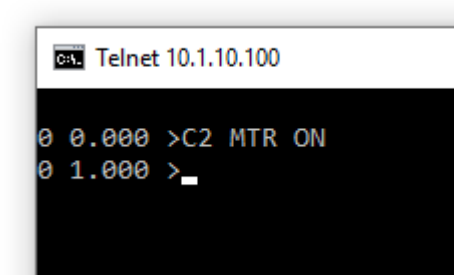
<receiver> <verb> <parameters> <CR>

The receiver can be either a single motor axis or a coordinated group of axes. Individual axes are presented as the "A" array numbered 1 to 16 (A is for "Axis"). An example of an object message to an individual axis would be the MTR command, Motor On command:



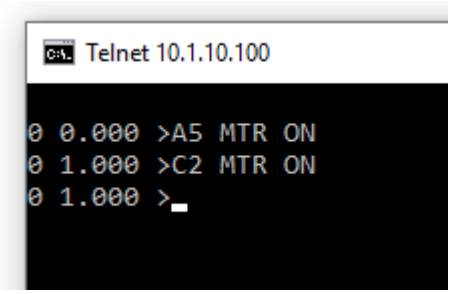
Here the "receiver" is A1, axis 1, the first "element" in the 16 xis array. MTR is the "verb" which acts on the "receiver". ON is a named constant with the value of 1. The parameter could have been a number also.

Groups of axes are represented as the "C" array numbered 1 to 10 ("C" is for Coordinated group). Before a group can be directed, it must be described with the INI command. The INI command takes as parameters the axis numbers that constitute the coordinated group. For example, a two axis coordinated group could be associated with group 1 as follows:



Ten groups are available. Here the first group is being described as running the first and second axis on the controller. The answer, "2" corresponds to the dimension of the group which should be the same as the number of parameters provided.

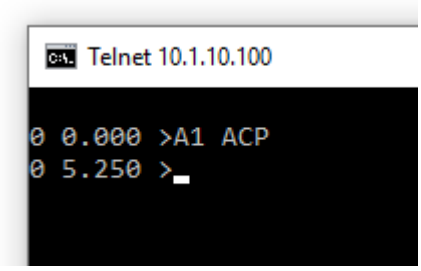
Many commands apply to both individual axes as well as to groups. For example, turning on the motors for axis 5 and for group 1 would be done with the same command, just different receivers:



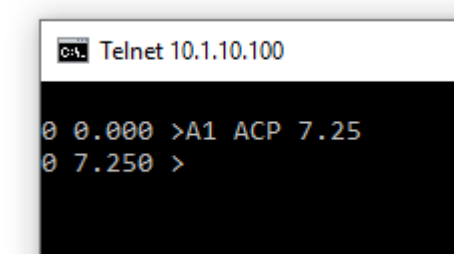
Reading and Writing

Many properties are read and written with the same command. The distinction is determined by whether a parameter has been provided or not. If a parameter has been provided then the command writes the information. If the command is absent a query is performed and the value is not changed.

For example, ACP, Actual Position, can be queried if the position parameter is absent:



Changing the actual position of the axis is accomplished if a parameter is provided:



Note that the result is the parameter value that was sent. This can be used to be assured the value was properly received.

Command Reference

Motion

Command Categories

Motion commands configure motion properties, perform motion actions, and report motion related status.

Abort

Command

```
<axis or group> ABT
```

Description

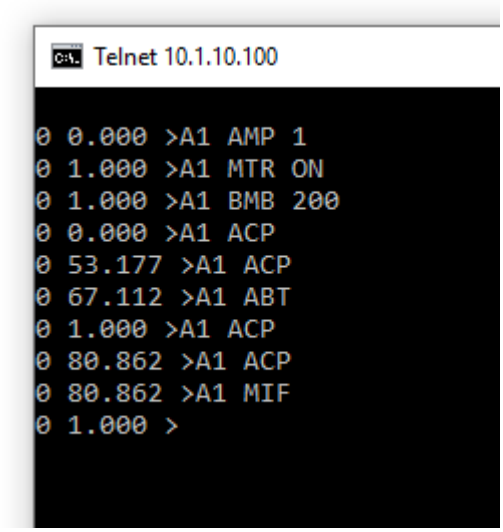
The Abort command immediately and abruptly stops motion without a controlled deceleration. Abort is generally only used at very slow speeds or in a situation where motion progress of any kind is catastrophic and it is necessary to apprehend all motion. A single axis move or a coordinated multi-axis move can be aborted.

Escapes

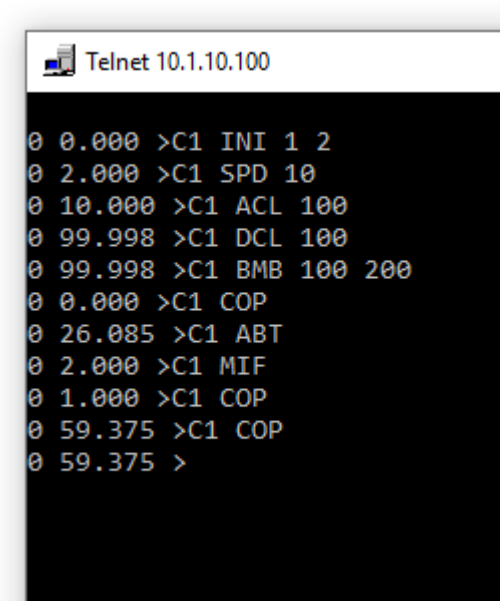
The Abort command does not generate any escapes.

Examples

This command sequence relates to axis 1. The current is set to 1 amp, the motor is turned on, a relative move is started to a destination of 200. The actual position is reported several times showing progress. The move is aborted and the actual position reported several more times demonstrating that the axis has stopped. The Move Is Finished command confirms that the motion has stopped.



This command aborts a two axis coordinated group after starting a vector move:



Related Topics

Stop
Begin Stop
Jog

Accel

Command

```
<axis or group> ACL <value>
```

Description

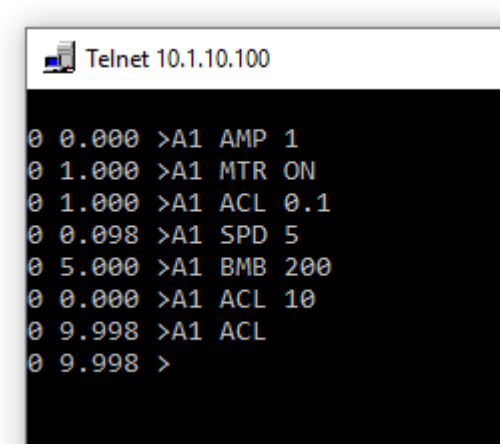
Set Accel is used to set or query the acceleration of a profiled move in User Units per second squared. If the Axis Object indicated is a single axis (T1Axis object type) the acceleration is for the movement of that motor when operating alone. If the Axis Object represents a group, for example a two dimensional group (T2Axis object) or a four dimensional group (T4Axis object), the acceleration applies to the coordinated motion profile of the group. Set Decel may be used to independently set the deceleration.

Escapes

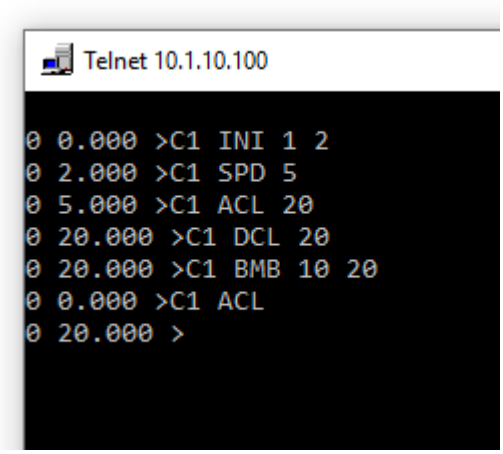
If the Accel setting is 0 or negative an User Accel is 0 Or Negative Escape Code will occur.

Examples

The following list of commands initiates a move and changes the acceleration during the move. The current accel is checked.



The following list of commands configures a vector move.



Related Topics

Accel
Speed
Decel

Actual Position

Command

```
<axis or group> ACP <value> <value> ... <value>
```

Description

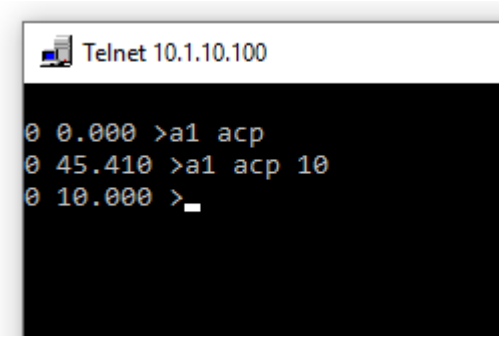
The Set Actual Position commands is used to set or query the current physical position of the axis. The commands takes as many parameters as group size. A single axis requires one parameter. A 3 axis group requires three parameters. Additional parameter ovals are provided when a multiaxis group is chosen. Setting the Actual Position also sets the Commanded Position to be the same value to prevent unexpected motion. The Actual Position of an encoder is the sensed position. The Actual Position of a stepper motor is the number of electrical pulses that have been emitted from the controller going into the stepper motor drive or, if using closed loop methods, into the commutator, indicating the desired position. The Set Actual Position command should not be used while an axis is in motion. The Actual Position can be queries while in motion but it should not be changed while in motion. The Set Actual Position command is most often used in homing procedures which use sensors or mechanical interference to find machine features and establish a coordinate system. Note that the position parameter does not need to be zero. If a machine homes to a position regarded as the end of the range of motion the Set Actual Position block can be used to assign that position a high value.

Escapes

Setting the actual position does not generate any escapes.

Examples

This command queries the actual position of axis 1 and then sets the value to :



Related Topics

Positive Limit
Negative Limit
Commanded Position
Destination Position

Arm Capture

Command

```
<axis> ACT
```

Description

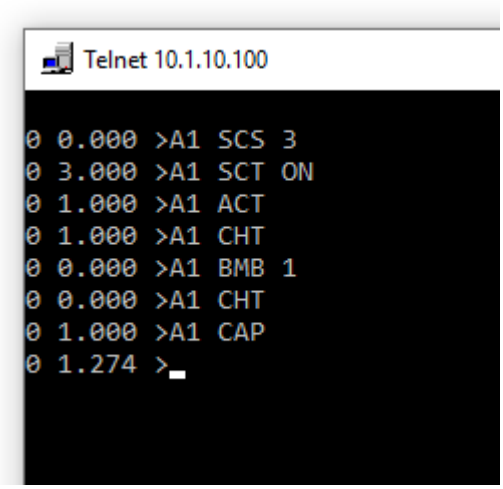
Arm Capture is used to setup the system to record the axis position when a specified input occurs. Arm Capture resets the capture latches for the related axis. When Capture Has Tripped the Capture Position information is valid and can be used by the motion application. The hardware capture latches can record an event with the resolution of the encoder and provide additional precision over software capture methods.

Escapes

The Arm Capture commands does not produce any escapes.

Examples

This list of commands configures a capture event, performs a capture, and reads the result. A capture source is chosen, a capture edge is chosen, and the capture is armed. Motion then occurs. A check is made that the capture event has occurred. When the capture has been confirmed tripped the capture position is valid and can be reported.



Related Topics

Capture Position
Set Capture Trip
Set Capture Source
Capture Has Tripped
Capture Bit

Begin Move By

Command

```
<axis or group> BMB <value> <value> ... <value>
```

Description

The Begin Move By commands initiates a relative move by the specified position parameters but does not wait for the move to finish before continuing. The motion is performed with a trapezoidal velocity profile based on parameters set with the Accel , Decel , and Speed commands. These parameters apply to the vector path motion of the coordinated group rather than to any particular axis. Begin Move By is a nonblocking command. The execution continues immediately allowing other commands to be performed while the motion is occurring. It is often necessary at some point to wait for the motion to finish which is accomplished with the Move Is Finished commands. Begin Move By can be used on an axis that is already in motion. The target will be changed to the current position of the motor when the block is executed plus the parameter value. The Stop, Begin Stop, and Abort commands will end the motion before it reaches the target position.

Escapes

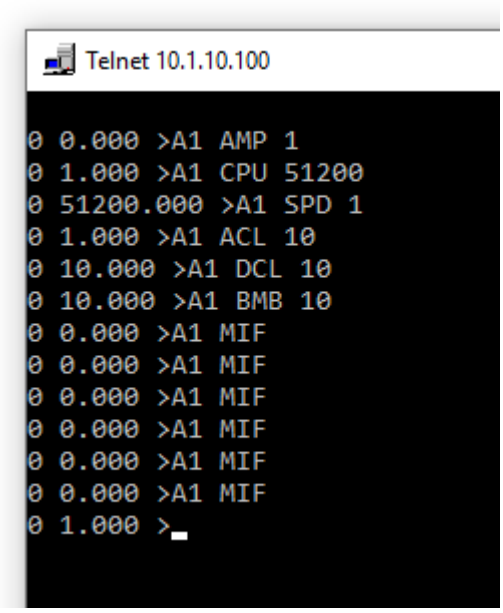
Begin Move By will produce an Axis Is Busy Escape Code if the group is already in motion.

A single axis can be directed to a new destination while in motion as long as the destination is still ahead of the motion and obtainable with the current decel setting. If not a Motion Overrun Escape Code will occur.

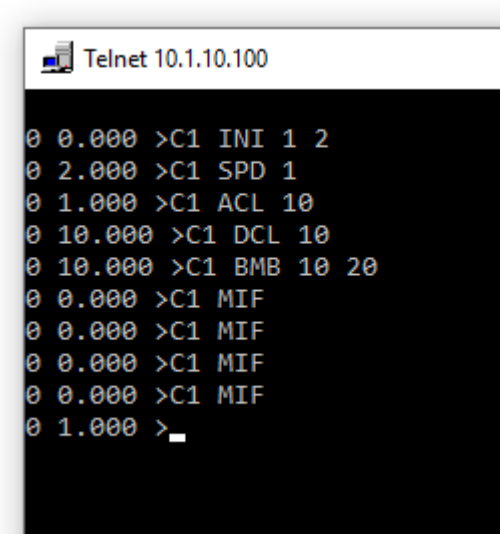
If the destination of the move is beyond the positive and negative software limits, the Begin Move By block will produce a Position Limit Escape Code and the move will not be attempted.

Examples

The following sequence of commands configures a motion profile, begins a move, and checks to see when the move is finished.



The following performs a coordinated move.



Related Topics

Begin Move To
Move By
Speed
Accel
Decel
Move Is Finished

Begin Move To

Command

```
<axis or group> BMT <value> <value> ... <value>
```

Description

Begin Move To initiates an absolute coordinated move to the specified position parameters but does not wait for the move to finish before continuing. The method requires as many parameters as the dimension of the axis group. A single axis has one parameter. A group with 3 axes requires 3 parameters. The motion is performed with a trapezoidal velocity profile based on parameters set with the Accel , Decel , and Speed _BLOCKS. These parameters apply to the vector path motion of the coordinated group rather than to any particular axis. Begin Move To is a nonblocking command. Execution continues immediately allowing other actions to be performed while the motion is occurring. It is often necessary at some point to wait for the motion to finish which is accomplished with the Move Is Finished command. If there is nothing to do during the motion it is simplest to use the Move To blocking form of the command. However this will block the interpretation of other commands form the host. To prevent this communication blackout use non blocking motion forms. Begin Move To can be used on an axis that is already in motion. The target will be changed to the new destination. The Stop, Begin Stop, and Abort commands will end the motion before it reaches the target position.

Escapes

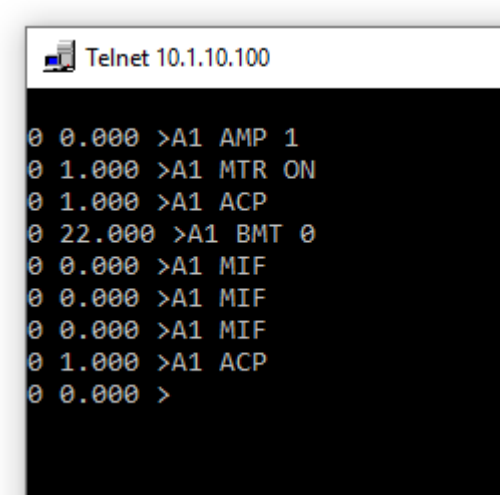
Begin Move To will produce an Axis Is Busy Escape Code if a coordinated group is already in motion.

A single axis can be directed to a new destination while in motion as long as the destination is still ahead of the current position in the current direction of travel and obtainable with the current decel setting. If not a Motion Overrun Escape Code will occur.

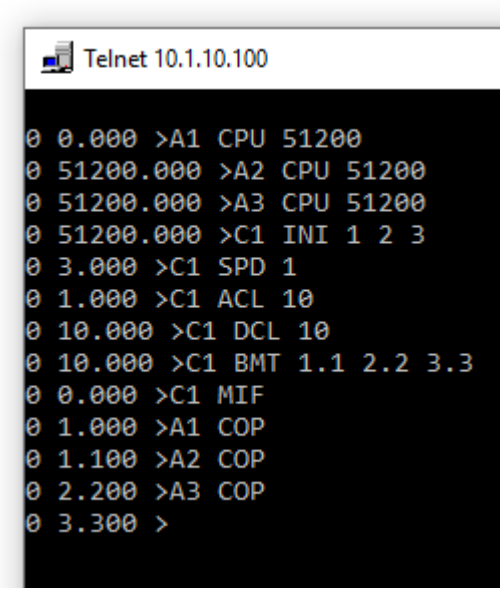
If the destination of the move is beyond the positive and negative software limits, the Begin Move To block will produce a Position Limit Escape Code.

Examples

This command sends axis 1 to absolute coordinate 0:



This command begins a vector move to coordinates 1.1, 2.2 and 3.3:



Related Topics

Begin Move By
Move To
Set Speed
Set Accel
Set Decel
Move Is Finished

Begin Stop

Command

```
<axis or group> BST
```

Description

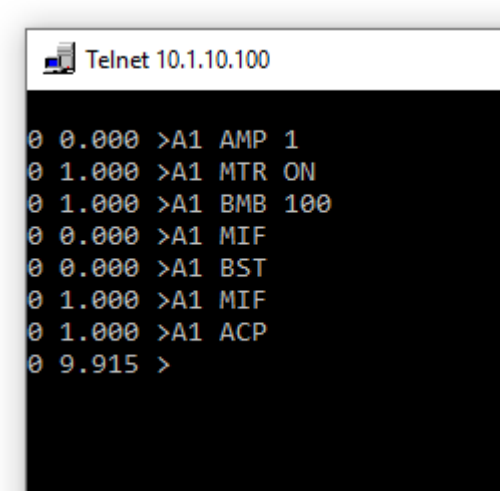
Begin Stop directs the axis or group to slow down at the specified decel rate and stop motion. A coordinated group will remain coordinated during the stop. This is a non-blocking command. The calling program will not wait until after the stop has finished before continuing but will immediately execute the next statement while the axis or group is decelerating. The Move Is Finished commands can be used to confirm when the axis has come to a complete stop.

Escapes

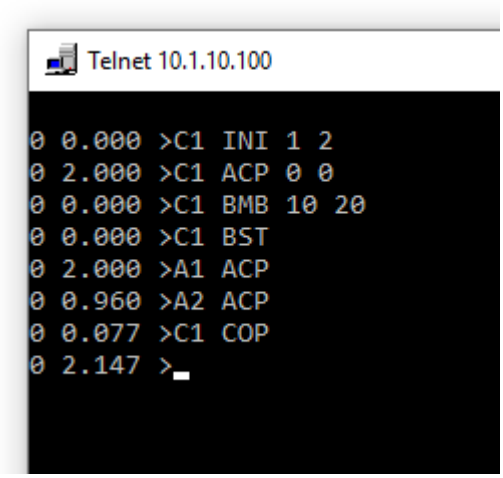
Begin Stop does not generate any escapes.

Examples

This sequence of commands initiates a move and stops it early:



This initiates a coordinated vector move and then stops it early:



Related Topics

- Stop
- Jog
- Abort

Capture Bit

Command

```
<axis> CAB
```

Description

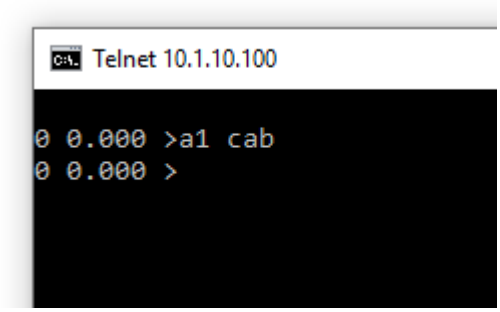
The Capture Bit returns the current value of the signal chosen to be the capture signal for the indicated axis. The signal is selected with Set Capture Source. This signal is used to trigger the capture latches for precisely recording an axis position. This command is provided to observe the state of the capture signal independent of the capture hardware.

Escapes

The Capture Bit commands does not produce any escapes.

Examples

This command shows that the capture bit for Axis 1 is not active:



Related Topics

Arm Capture
Capture Has Tripped
Capture Position
Set Capture Trip
Set Capture Source

Capture Has Tripped

Command

```
<axis> CHT
```

Description

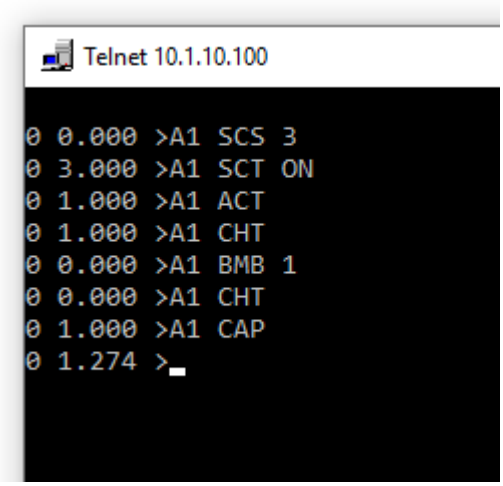
Capture Has Tripped returns true if the capture signal, configured by Capture Source , has occurred. If Capture Has Tripped then Capture Position is valid and contains the position where the event occurred.

Escapes

The Capture Has Tripped commands does not produce any escapes.

Examples

This list of commands configures a capture event, performs a capture, and reads the result. A capture source is chosen, a capture edge is chosen, and the capture is armed. Motion then occurs. A check is made that the capture event has occurred. When the capture has been confirmed tripped the capture position is valid and can be reported.



Related Topics

Capture Position
Set Capture Trip
Set Capture Source
Arm Capture
Capture Bit

Capture Position

Command

```
<axis> CAP
```

Description

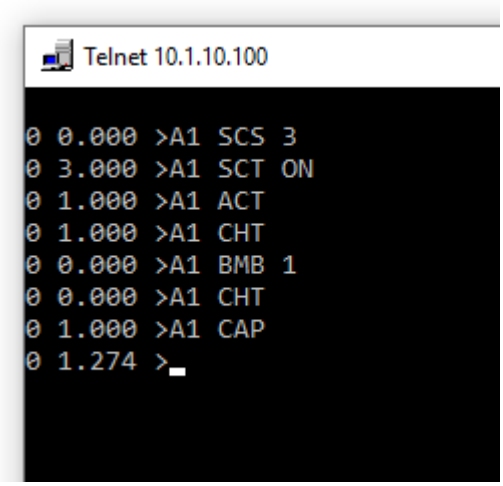
Capture Position returns the position where the capture event occurred. The capture event occurs when the capture is armed and the selected capture signal transitions as sepcified by Set Capture Trip. The Capture Position is only valid if Capture Has Tripped.

Escapes

Reading the Capture Position does not produce any escapes.

Examples

This list of commands configures a capture event, performs a capture, and reads the result. A capture source is chosen, a capture edge is chosen, and the capture is armed. Motion then occurs. A check is made that the capture event has occurred. When the capture has been confirmed tripped the capture position is valid and can be reported.



Related Topics

- Arm Capture
- Set Capture Trip
- Set Capture Source
- Capture Has Tripped
- Capture Bit

Commanded Position

Command

```
<axis> COP <value>
```

Description

Set Commanded Position is used to set the desired setpoint for the motor. During normal profiled moves the commanded position is set by the internal motion profiler whilch calculates a smooth sequences of commanded positions. However there are some situations where the criteria for where the motor should move each sample period is custom, for example electronic gearing. Set Commanded Position bypasses the internal motion profiler allowing the direct assignment of the position setpoint. Note that a discontinuity in setpoint positions will directly map into an attempted discontinuity in motor position resulting in a substantial bump. When working with stepper motors Set Commanded Position must be used along with Set Profile Velocity to accomplish effective motion performance. This command is usually used in a resident program operating at the sample rate frequency necessry to provide a smooth trajectory.

Querying the commanded position reports the intended position for the motor. Virtual axes can only report commanded positions as there is no actual position with a virtual axis. Commanded motion can occur on a physical axis with the motor off.

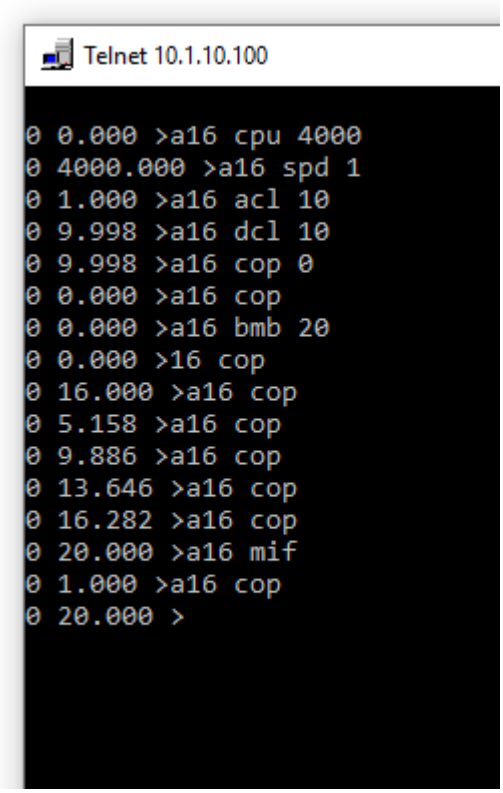
The Commanded Position of a coordinated group is the pathlength along the vector move.

Escapes

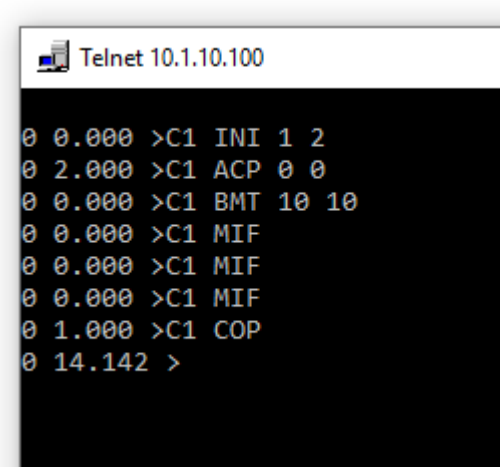
The Set Commanded Position command does not generate any escapes.

Examples

The following commands perform motion of a virtual axis and report the position.



This vector move is shown to have a vector length of 14.142.



Related Topics

Positive Limit
Negative Limit
Actual Position
Destination Position

Coordinate Inversion

Command

```
<axis> CIV <on or off>
```

Description

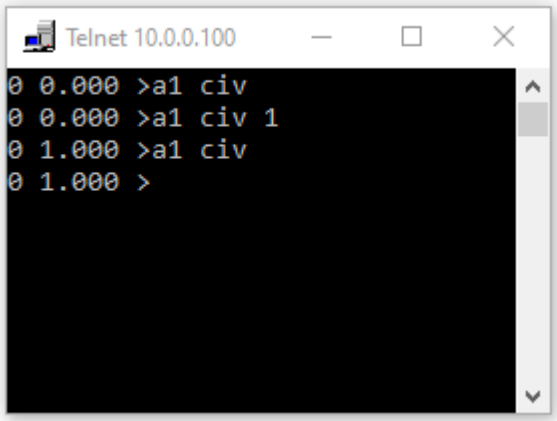
Set Coordinate Inversion is used to change the direction a motor or encoder regards as positive. Axis direction is influenced by mechanical transmission reversals, encoder phase definition, and wiring conventions. If the motor does not move in the direction regarded as positive this command may be used to invert the direction by calling with a non-zero parameter. This command operates in an incremental manner by inverting the coordinate frame about the current Actual Position rather than 0. The best time to use this command is during initial setup before homing has been performed. This is not intended to be used after initialization as it can relocate the origin.

Escapes

Set Coordinate Inversion does not generate any escapes.

Examples

The following commands query the coordinate inversion of axis 1 and find it to be off, set it on, and confirm it is on:.



Related Topics

Set Positive Limit
Motor On

Counts Per User Unit

Command

```
<axis> CPU <value>
```

Description

The Counts Per User Unit command allows motion to be described in convenient units of rotations, degrees, or millimeters instead of motor steps. An explanation of how to identify the proper value for Counts Per User Unit is described in the Introduction topic User Units. This command is used by the Setup Tab set the ratio to the desired value.

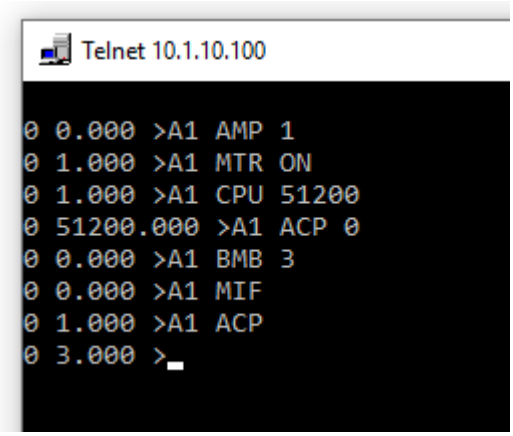
The command can be used with host directed motion to establish a desired counts per user unit regardless of how the controller was initialized.

Escapes

Set Counts Per User Unit does not produce any escapes.

Examples

Stepper motors have 51200 microsteps for a full rotation. This command list establishes a user unit of rotations and then commands Axis 1 to move by 3 turns:



Related Topics

User Units Description
Begin Move By

Decel

Command

```
<axis or group> DCL <value>
```

Description

This command is used to set the deceleration of a profiled move in user units per second squared. If the Axis Object indicated is a single axis (T1Axis) the deceleration is for the movement of that motor when operating alone. If the Axis Object represents a multiple axis group such as a two dimensional (T2Axis) or four

dimensional group (T4Axis), the deceleration applies to the coordinated motion profile of the group. Accel else Set Accel may be used to independently set the Acceleration of the axis or group.

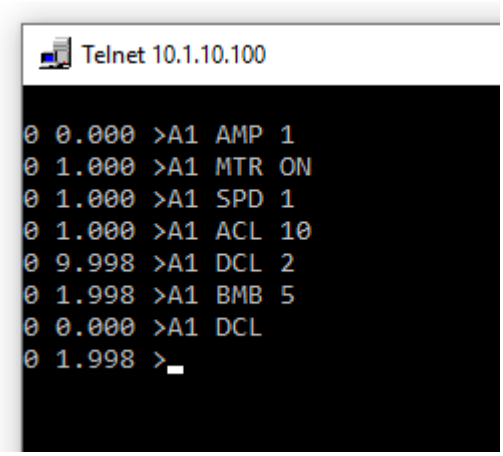
Escapes

If the Decel setting is 0 or negative an User Decells 0 Or Negative Escape Code will occur.

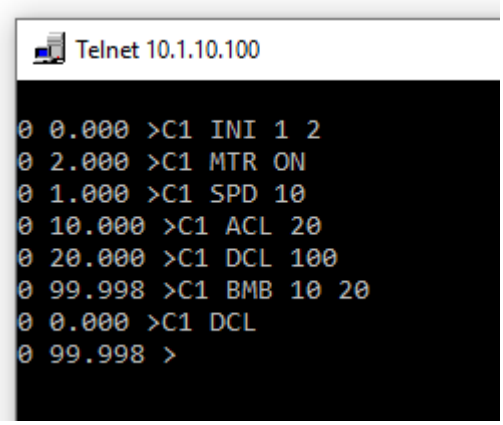
If a new decel value that is issued during a move cannot be achieved with the current direction and speed a Motion Overrun Escape Code will occur.

Examples

This list of commands performs a move with a decel that is more gradual than the accel



This list of commands performs a coordinated vector move with a sharp decel compared to the accel



Related Topics

Decel
Set Speed
Set Accel

Destination Position

Command

```
<axis or group> DEP
```

Description

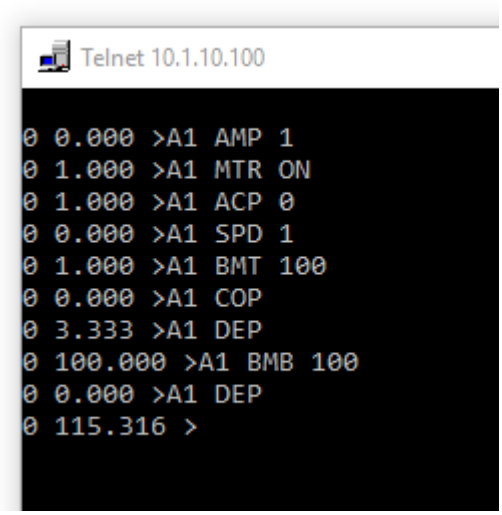
The Destination Position commands is valid during a motion and indicates the current target position of the move as an absolute coordinate. This value is useful for identifying how far an axis has to go before the move is finished by subtracting the Commanded Position from the Destination Position. For multiaxis moves the Destination Position is the length of the hypotenuse of the vector move being performed. The beginning of the move is 0 and the end is the Destination Position. If a move is retargeted with a move by command while the move is in progress the Destination Position is a way to identify the final target position for that axis.

Escapes

The Destination Position commands does not produce any escapes.

Examples

The following list of commands starts a motion and notes the new destination after a mid-move destination adjustment:



Related Topics

Actual Position
Capture Position
Error Position
Move To
Commanded Position

Initialize Group

Command

```
<group> INI <value> <value> ... <value>
```

Description

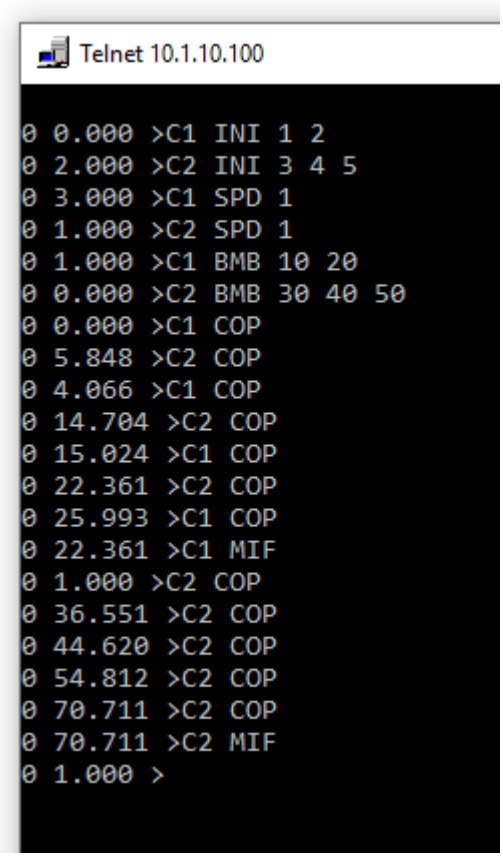
This command describes what axes comprise a multiaxis coordinated group. Coordinated motion is then performed by sending commands to the group rather than the individual component axes. The parameters represent the component axes. The dimension of the coordinated group is established by the number of parameters provided. Two parameters will establish a 2 axis coordinated group. Four parameters will establish a 4 axis coordinated group. The maximum dimension of a coordinated group is 6 axes. Note that the compiler directive MoreThan3Axis must be defined to access coordinated group dimensions greater than 3.

Escapes

If only one parameter is provided a TooFewParameters error will be returned. If more than 6 parameters are provided a TooManyParameters error will be returned.

Examples

The following command list initializes a two axis group named C1 and a three axis group named C2. Both groups start vector moves. Commands are used to monitor progress and determine when the moves are finish:



Related Topics

Begin Move By
Begin Move To
Move By
Move To
Speed
Accel
Decel
Move Is Finished

Jog

Command

```
<axis> JOG <speed>
```

Description

Jog directs an axis to move at the specified speed indefinitely. If the magnitude of the single precision floating point parameter is smaller than the magnitude of the current speed the axis will slow down at the decel rate. If the magnitude is greater it will speed up at the accel rate. It is possible to jog in the opposite direction with respect to the current speed which causes the axis to come to a stop, reverse direction, and accelerate up to the indicated speed. It is possible to jog at 0 speed. In this condition the move is still considered active even though it is not physically progressing. Jog may supersede a move, changing it into a continuous motion. Jogging is terminated by a Begin Stop, Stop, or Abort.

Although it is possible to use jog to produce movement when searching for home switches or other input events, it is generally a better idea to move a distance which should include the event so that the behavior of the machine if the event is not found is to stop rather than to travel indefinitely. Analagous to the notion of a "time out" such a move could be thought of as a "position out". If the expected event does not occur within the expected distance (as distinct from an expected time) then an error must have occurred.

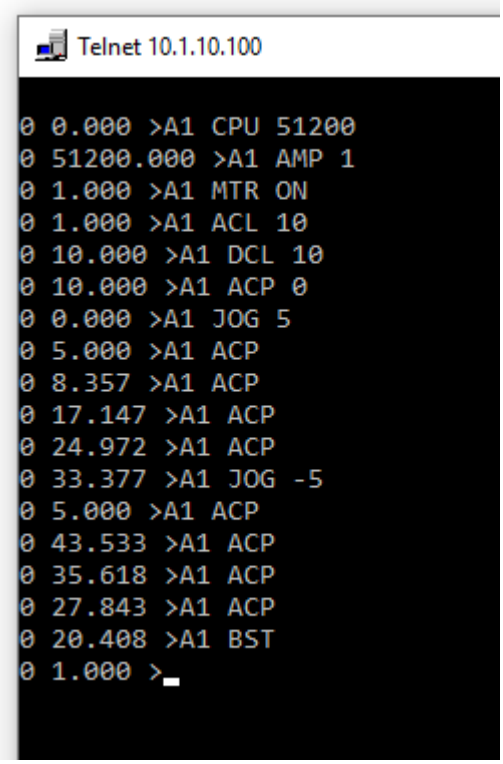
Jogging is NOT protected by positive and negative limits. Jogging, by it's nature, is a continuous and ongoing move. To realize jogging velocity to the edge of a machine movement, begin a move to the positive or negative limit at the required speed.

Escapes

Jog does not generate any escapes.

Examples

The following commands jog the motor:



Related Topics

- Set Speed
- Set Accel
- Set Decel
- Stop
- Begin Stop
- Abort
- Positive Limit

Motor

Command

```
<axis or group> MTR <ON or OFF>
```

Description

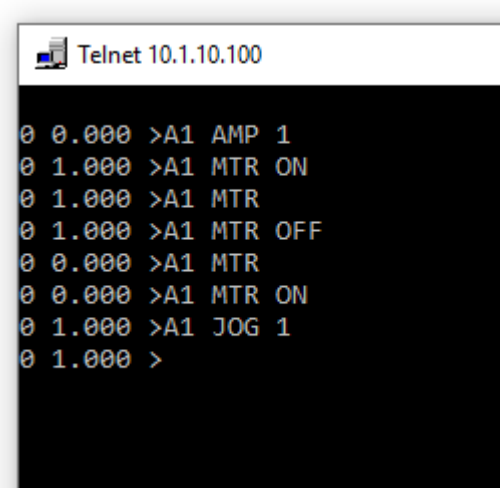
Set Motor is used to turn motor positioning control on and off for an axis or all of the axes in the related group. Called with a parameter value of true enables the amplifier lines. The motor establishes the current Actual Position as the commanded, or desired, position so as to remain still. When called with a parameter value of false the amplifier lines are disabled, the motor command is set to 0 volts (if configured for servo) and no further motor activity occurs. Readability of the program is improved by using the predefined boolean constants On and Off.

Escapes

Set Motor will produce a Watchdog Has Tripped EscapeCode

Examples

This list of commands sets the motor on and off examing its state and then starts jogging:



Related Topics

Motor States

Move Is Finished

Command

```
<axis or group> MIF
```

Description

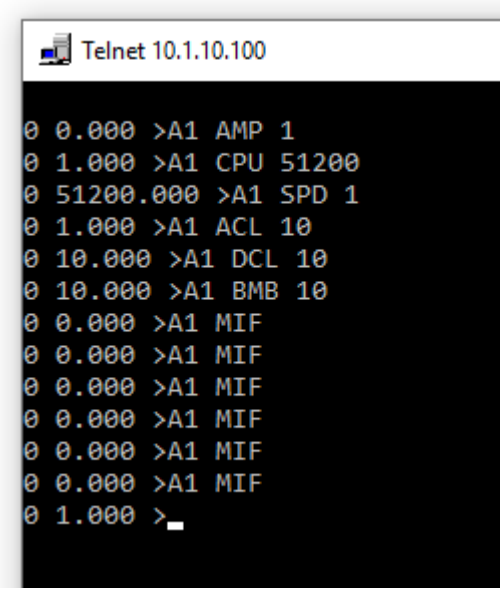
Move Is Finished indicates if the a motor or motor group has completed motion or is still in progress. This command is used with non-blocking Begin Move By and Begin Move Tomotion commands that initiate motion but do not wait for the motion to be comleted. It is useful when multiple concurrent motions are started and it is necessary to confirm that their independent activities have all completed before continuing to the next program step. This can also be used to monitor a motor's condition for purposes such as managing current cutback for a stepper motor drive.

Escapes

Move Is Finished does not generate any escapes.

Examples

This list of commands initiates a move and then watches to see when the move is finished.:



Related Topics

Begin Move By
Begin Move To

Negative Limit

Command

```
<axis> NLT <value>
```

Description

Set Negative Limit establishes a negative-direction boundary for movement. If the commanded position on the axis falls beyond this boundary, a Position Limit Escape Code will occur and the move will not commence. Jogging is not subject to software limits since jogging is used for ongoing motion. Note that the parameter is a signed value. A common mistake when a machine has the origin in the middle is to set the Positive Limit and Negative Limit to the same value. However, the limits are a software boundary measured with an offset from the origin; assigning them the same value places them in the same location which makes all other commanded locations out of range. Negative limits have negative numeric parameters unless the desired range of motion is exclusively positive from the origin. Software limits are always active but can be effectively disabled by being set with large values.

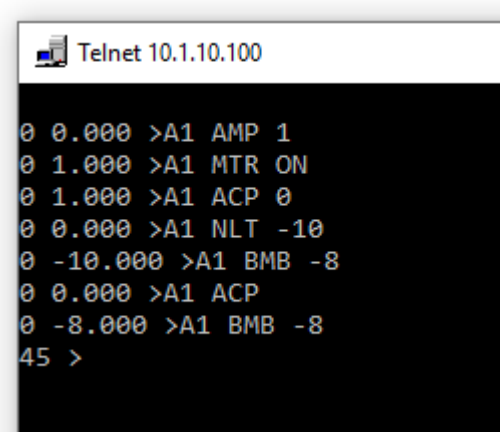
When designing homing routines it is usually necessary to widen the software limits until the coordinate system is established to afford freedom to search for the home sensor. Once the origin of the coordinate system is found the limits can be re-established.

Escapes

The Negative Limit commands does not generate any escapes

Examples

This list of commands sets a negative limit, moves within the limit and then tries to move outside the limit"



Related Topics

Positive Limit
Position Limit EscapeCode

Positive Limit

Command

```
<axis> PLT <value>
```

Description

Set Positive Limit establishes a positive-direction boundary for movement. If the axis is asked to attempt a move beyond this boundary, a Position Limit Escape Code will occur and the move will not commence. Jogging is not subject to software limits since jogging is used for ongoing motion. Software limits are always active but can be effectively disabled by being set with large values.

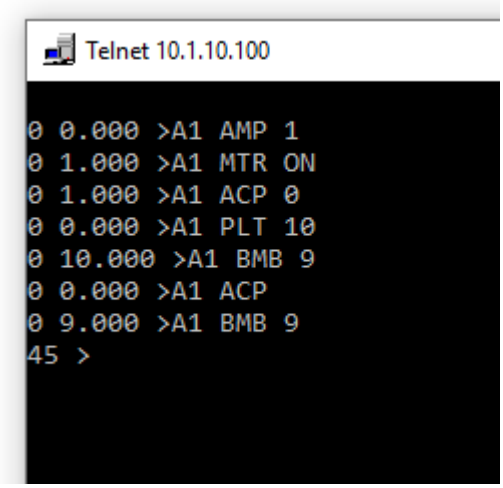
When designing homing routines it is usually necessary to widen the software limits until the coordinate system is established to afford freedom to search for the home sensor. Once the origin of the coordinate system is found the limits can be re-established.

Escapes

The Positive Limit commands does not generate any escapes

Examples

This list of commands sets a positive limit, moves within the limit and then tries to move outside the limit"



Related Topics

Negative Limit
Position Limit EscapeCode

Profile Phase

Command

```
<axis or group> PFP
```

Description

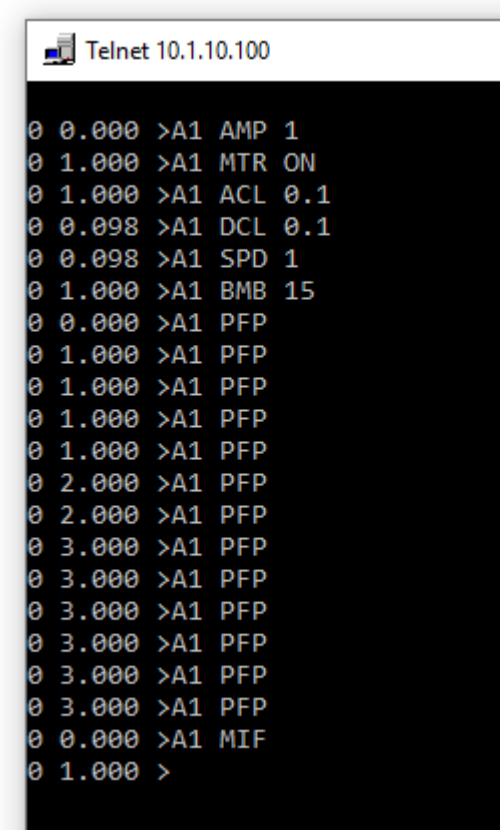
Profile Phase indicates if the trapezoidal motion profile is in the Accel phase (value 1), the Slew phase (value 2) the Decel phase (value 3), or no longer moving (value 0). This is used to identify transition points in the profile and perform operations at those transitions. Note that the Profile Phase function can indicate if the motor is moving or not without a delay. The Move Is Finished command includes a built in yield which delays the answer by a controller sample period. The Profile Phase command can answer the same question immediately by comparing Profile Phase to 0.

Escapes

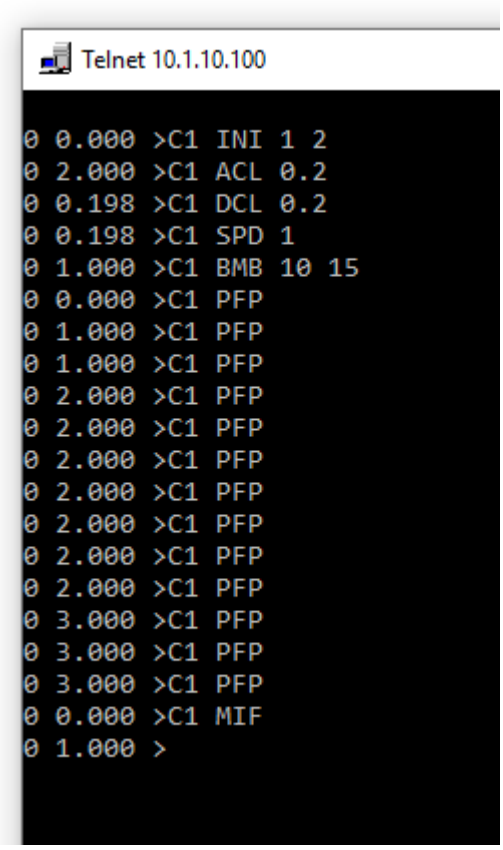
Profile Phase does not generate any escapes.

Examples

This list of commands produces a move with long accel and decel times allowing the profile phase to be inspected in all three phases:



This list of commands examines the profile phase of a coordinated move:



Related Topics

Accel
Decel
Speed
Move Is Finished

Profile Velocity

Command

```
<axis or group> PFV
```

Description

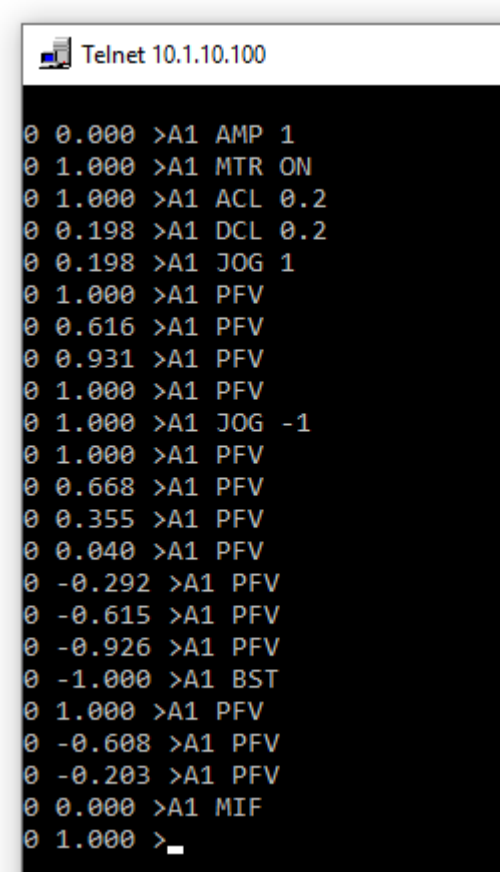
Profile Velocity returns the current commanded speed (signed magnitude) that is being used to generate the trapezoidal motion trajectory. During slew, the magnitude of Profile Velocity is the same as Speed. During Accel and Decel the profile velocity varies linearly with progress through the phase. The value returned is the value of the trapezoidal profile, not the actual motor speed itself. Reading the actual speed requires an application program that differentiates position information and returns a value in a user variable.

Escapes

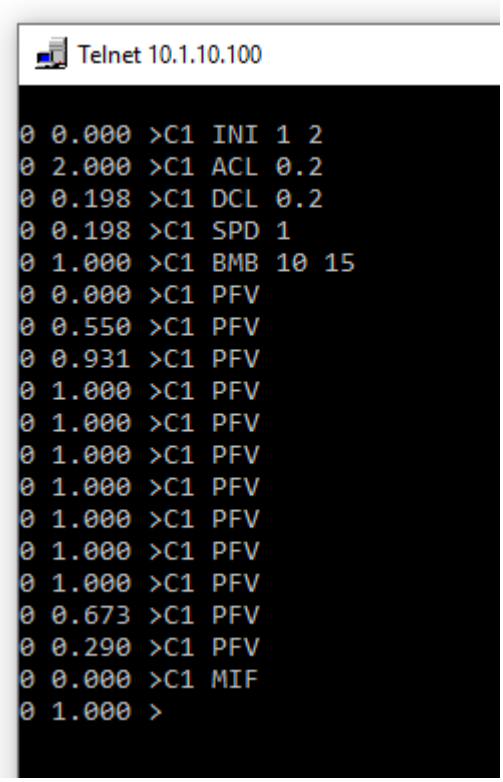
The Profile Velocity commands does not generate any escapes.

Examples

This list of commands initiates a move and examines the profile velocity:



This list of commands initiates a coordinated move and examines the profile velocity:



Related Topics

Set Capture Source

Command

```
<axis> SCS <value>
```

Description

The Set Capture Source commands is used to indicate which input will be used for a capture event.

The tables elaborates the parameter:

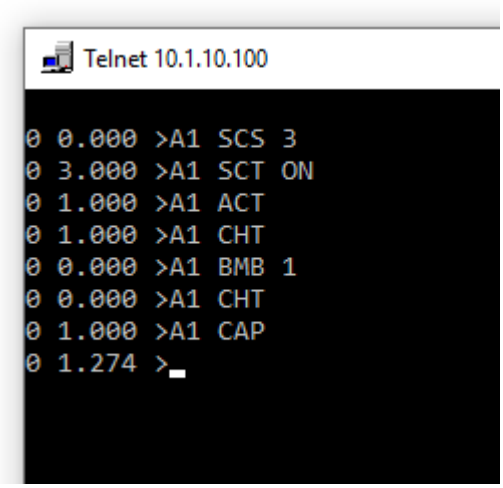
Parameter	Signal Location
1	Front Encoder A
2	Front Encoder B
3	Front Encoder Index
6	Option 1 Index
9	Option 2 Index
12	Option 3 Index

Escapes

The Set Capture Source block does not produce any escapes

Examples

This list of commands configures a capture event, performs a capture, and reads the result. A capture source is chosen, a capture edge is chosen, and the capture is armed. Motion then occurs. A check is made that the capture event has occurred. When the capture has been confirmed tripped the capture position is valid and can be reported.



Related Topics

Capture Position
Set Capture Trip
Capture Has Tripped
Capture Bit
Arm Capture

Set Capture Trip

Command

```
<axis> SCT <value>
```

Description

The Set Capture Trip commands is used to indicate whether the capture event should occur on the rising edge or the falling edge of the capture signal.

Escapes

The Set Capture Trip commands does not produce any escapes

Related Topics

Capture Position
Set Capture Source
Capture Has Tripped
Arm Capture
Capture Bit

Set Current To

Command

```
<axis> AMP <value>
```

Description

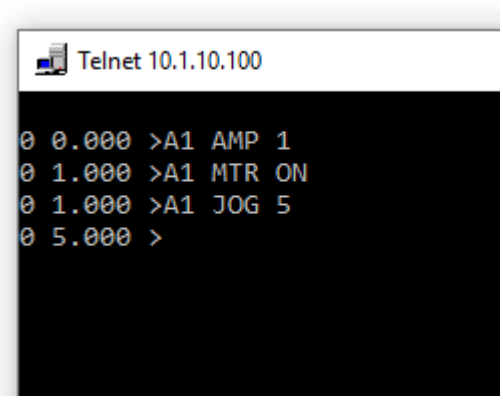
Set Current To sets the maximum phase current for internal stepper motor drives. Instrumentation monitoring the current of an internal drive set to 2 amps during very slow rotation of the motor would measure the current going up to 2 amps, diminishing down to -2 amps, and then increasing back to 2 amps sinusoidally. Note the current specifications on the motor data sheet. If the current specification is RMS then command parameter should be the RMS specification * 1.4. Excessive current can overheat a motor and not produce any torque improvement if the magnetics saturate. Insufficient current may not provide sufficient torque. Current management policies such as current cutback and current ramping can be implemented as tasks.

Escapes

Set Current To does not generate any escapes. If the requested current is above available current the maximum available current will be used.

Examples

The following command sets the current for axis 1 and starts a move:



Related Topics

Set Motor
Move Is Finished

Speed

Command

```
<axis or group> SPD <value>
```

Description

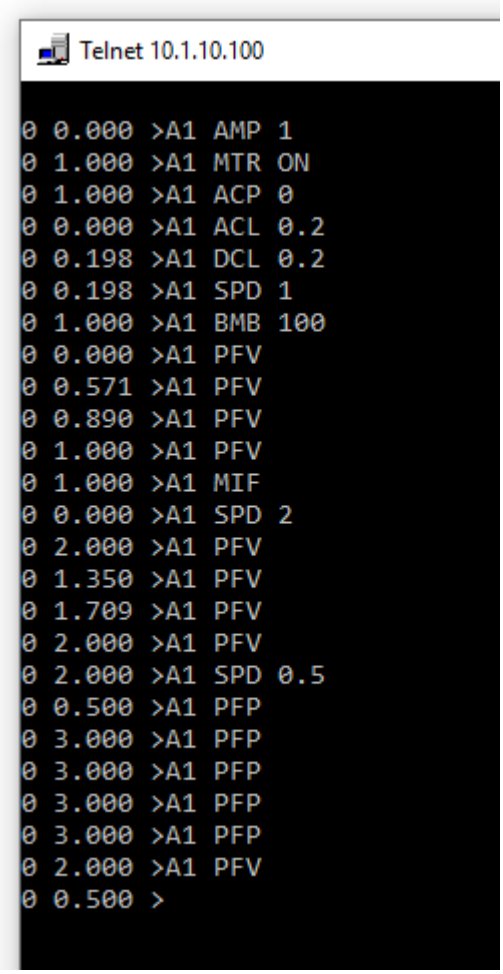
This command is used to establish the slew speed to be used during axis movement. The parameter is in units of User Units per second. The speed of a move may be changed on the fly at any point in a move and takes immediate effect if the motion is in the slew phase. Setting the speed to zero is a way to pause motion activity. The speed value can later be restored to the previous value and motion will resume. For single axis machines this command effects the speed of the axis. For multiaxis groups Set Speed effects the vector speeed of the group.

Escapes

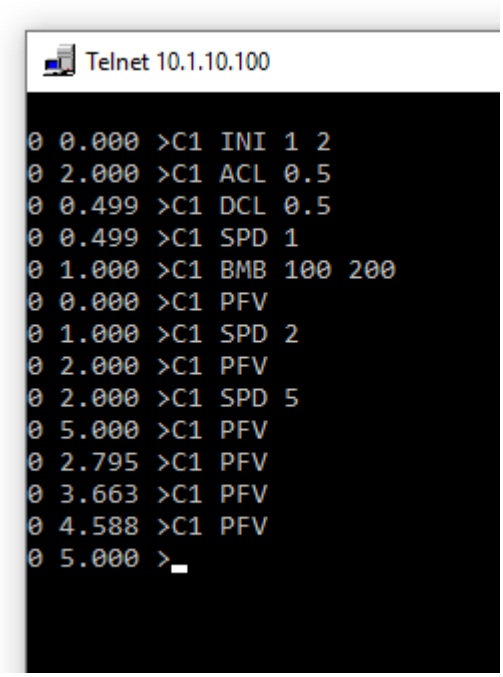
If the speed parameter resolves to a higher step frequency than 2Mhz then a Parameter Out Of Range Escape Code will occur.

Examples

The following list of commands varies motor speed during a move:



This list changes the speed of a coordinated move:



Related Topics

Accel
Decel

Multitasking

Command Categories

Multitasking commands provide access to arrays of shared data types and the means to begin, abort, and monitor controller resident tasks that use that information to perform actions.

User Booleans

Command

```
USB <index>
```

Description

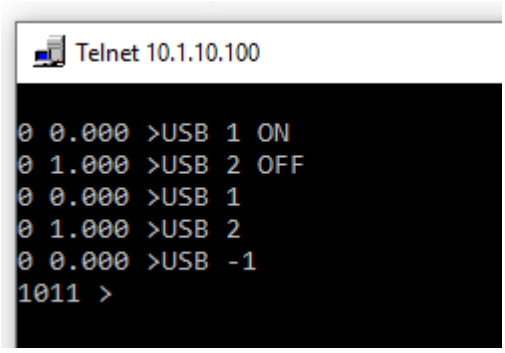
User Booleans can be used to transfer information between the host and resident controller programs. The 100 element array [1..100] of booleans can be read and written from both sides. Providing a parameter performs a write. Providing just the index but no parameter performs a read. There is no established convention for how the values are used. Generally they provide parameters to resident controller routines and return results back. The size of the array is a constant inside the interpreter package that can be adjusted as required.

Escapes

If the array index is out of bounds a Parameter Out Of Range Escape Code will occcur.

Examples

The following list of commands assign and retrieve User Numbers and illustrate an error response to an index being out of bounds:



Related Topics

User Longints
User Numbers

Begin User Task
Abort User Task

User Longints

Command

```
USL <index>
```

Description

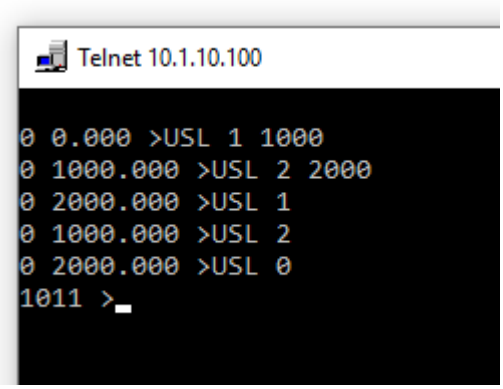
User Longints can be used to transfer information between the host and resident controller programs. The 100 element array [1..100] of 32 bit integers can be read and written from both sides. Providing a parameter performs a write. Providing just the index but no parameter performs a read. There is no established convention for how the values are used. Generally they provide parameters to resident controller routines and return results back. The size of the array is a constant inside the interpreter package that can be adjusted as required.

Escapes

If the array index is out of bounds a Parameter Out Of Range Escape Code will occcur.

Examples

The following list of commands assign and retrieve User Numbers and illustrate an error response to an index being out of bounds:



Related Topics

User Booleans
User Numbers
Begin User Task
Abort User Task

User Numbers

Command

```
USN <index>
```

Description

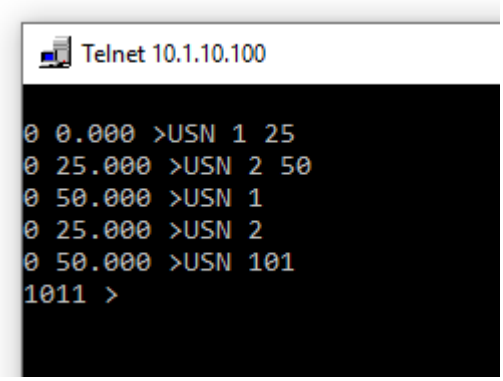
User Numbers can be used to transfer information between the host and resident controller programs. The 100 element array [1..100] of single precision floating point numbers can be read and written from both sides. Providing a parameter performs a write. Providing just the index but no parameter performs a read. There is no established convention for how the values are used. Generally they provide parameters to resident controller routines and return results back. The size of the array is a constant inside the AsciiCommands package that can be adjusted as required.

Escapes

If the array index is out of bounds a Parameter Out Of Range Escape Code will occcur.

Examples

The following list of commands assign and retrieve User Numbers and illustrate an error response to an index being out of bounds:



Related Topics

User Booleans
User Longints
Begin User Task
Abort User Task

Abort User Task

Command

```
AUT <task number>
```

Description

This command causes a procedure in the controller to abort execution. Resident application procedures to be managed should be named UserTask1, UserTask2, etc. The task number is referenced by this command.

A task can only be aborted with Abort Task if it had been started with Begin User Task. A Task Identity is the procedure number that was used with the Begin User Task command that started it.

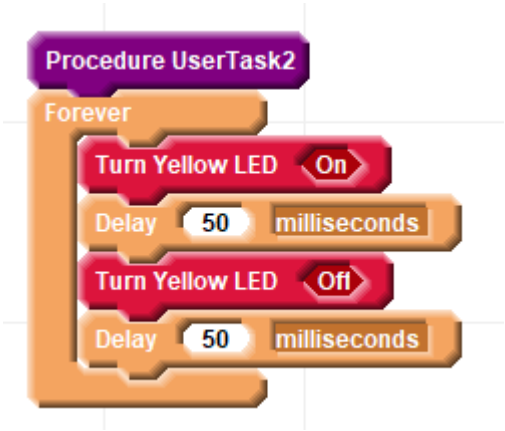
After aborting a task it is often necessary to "clean up" the residual condition of resources the task had been managing. For example if a homing procedure was using the Jog command to home and was counting on the detection of a sensor to stop it, aborting the homing routine would leave the motor still jogging but with no reason to stop. If a task managing a blinking light is aborted the light might remain in an active condition. It is usually better to avoid using the Abort Task command and to instead have the task terminate itself, and clean up after itself, based on a user boolean variable.

Escapes

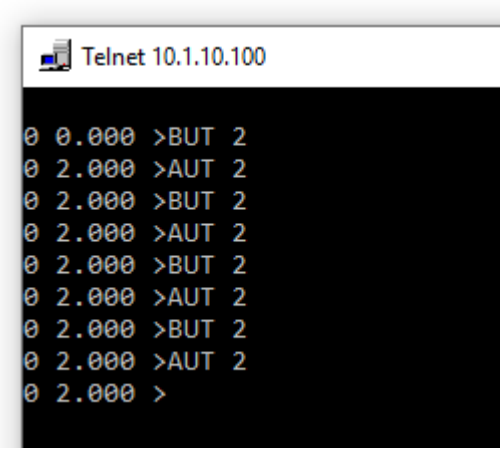
If the task number is specified that is less than 1 or greater than the MaxUserTasks constant inside the AsciiCommands package a Parameter Out Of Range Escape Code error will occur.

Examples

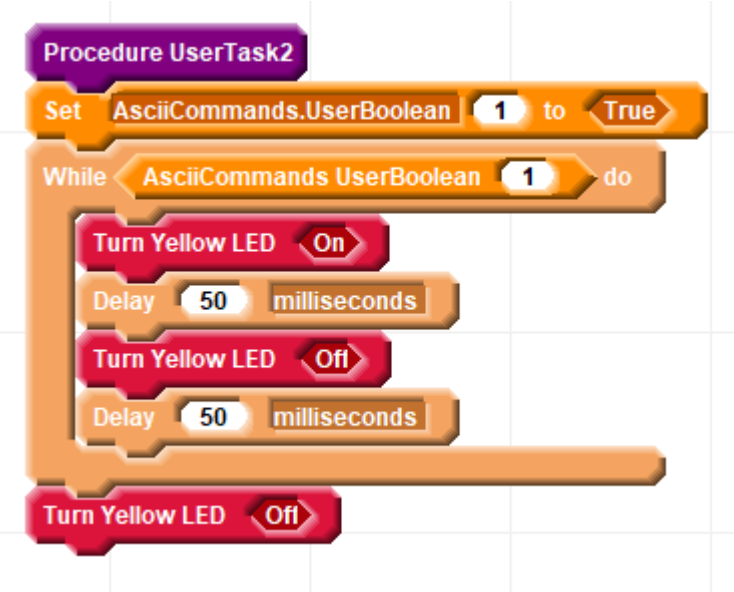
The following resident procedure named UserTask2 is described in blocks to blink a light:



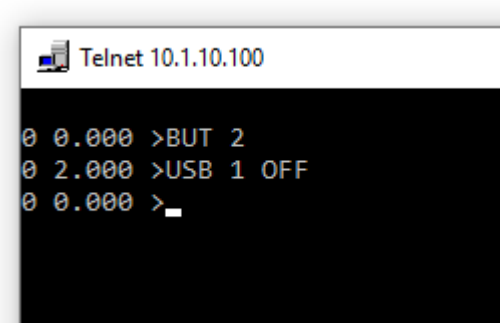
The following list of commands begin and abort the blinking user task behavior:



The problem is that half the time the led remains on and half the time the led remains off after it stops blinking. A better solution would be to use a User Boolean to indicate that the task should stop and establish the desired off condition for the LED:



This list of commands starts the blinking and then turns it off cleanly:



Related Topics

Yield
Begin User Task

User Task Present

Begin User Task

Command

```
BUT <task number>
```

Description

This command causes a procedure in the controller to begin executing as an independent activity concurrent along with with other tasks and motion. Application procedures to be managed as concurrent tasks should be named UserTask1, UserTask2, etc. The task number is referenced by this command.

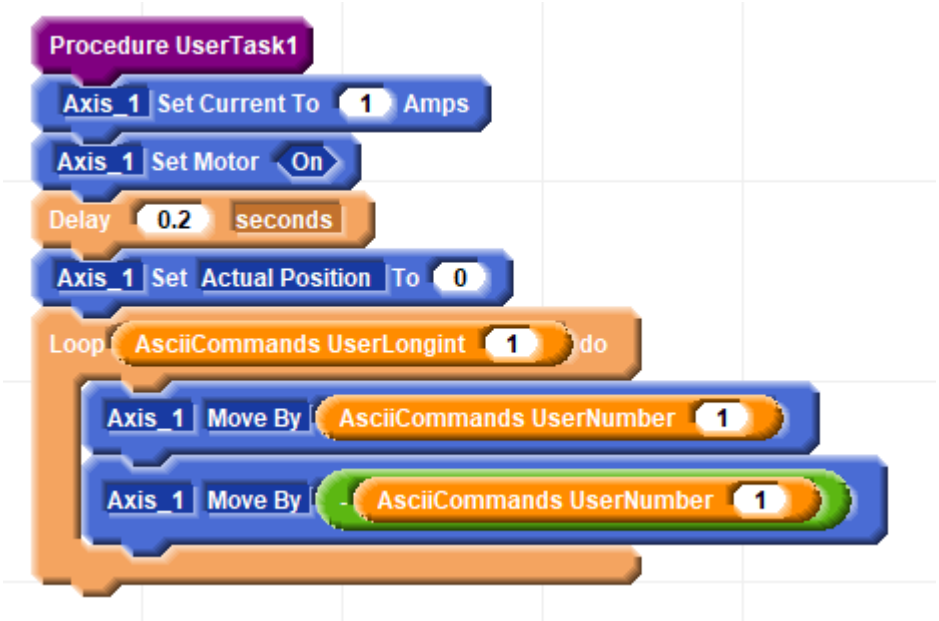
The MMC Family supports 16 concurrent tasks in addition to concurrent motion which does not consume task resources

Escapes

If all of the available task slots are filled an attempt to start another task will result in an Unable To Begin Task Escape Code. If the task number is specified that is less than 1 or greater than the MaxUserTasks constant inside the interpreter package a Parameter Out Of Range Escape Code error will occur. MaxUserTasks has a value of 16 but can be adjusted if more tasks need to be invoked from the host. Note that only 14 additional tasks can run at one time as two are used to sustain host communication.

Examples

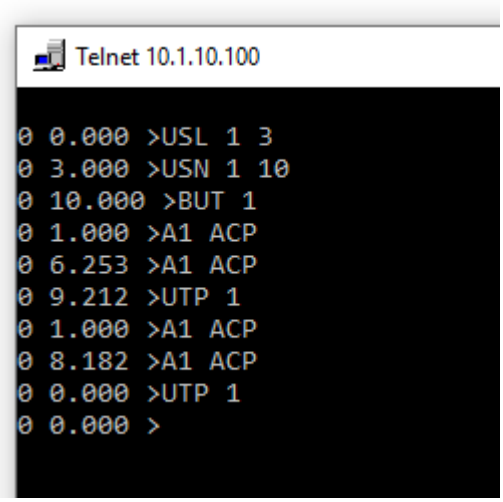
In this example a resident program has been described with blocks that look like this:



The current is set and the motor turned on. The position of the motor is set to 0. Then a number of loops are performed based on a Ascii Commands User Longint value at index 1. The motor moves back and forth by a

display specified as an Ascii Commands User Number value at index 1. This is a different array of values so even though the index is 1 in both cases these are two different values.

This series of commands provides the parameters needed by the resident routine and starts the routine:



The position of the motor is noted and seen to be changing. A check is made to see if User Task 1 is still operating and it is. Additional motion is seen until the motion stops at position 0. A check is made to see if User Task 1 is still running and it is seen that the task is no longer running.

Related Topics

Yield
Abort User Task
User Task Present

User Task Present

Command

```
UTP <task number>
```

Description

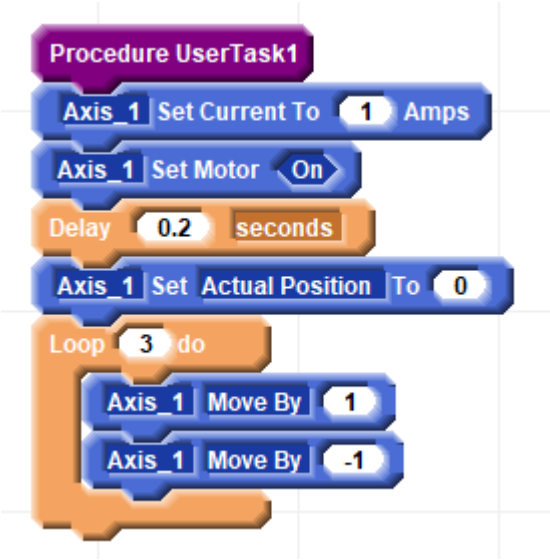
If a controller resident user task is running this commands returns true. If it is not running it returns false. This commands is often used to monitor a task to know when it has finished. The parameter is a number corresponding to the user task number in the name UserTask1, UserTask2, etc.

Escapes

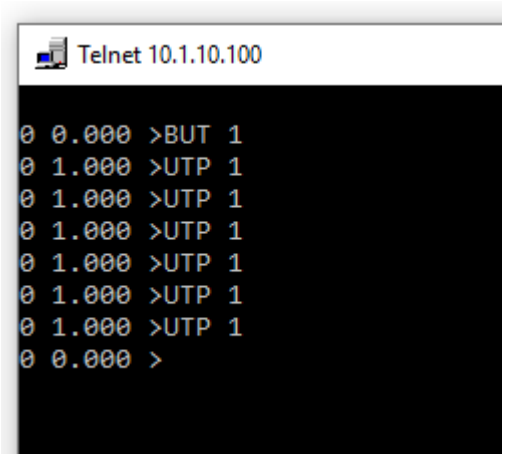
If the task number is specified that is less than 1 or greater than the MaxUserTasks constant inside the AsciiCommands package a Parameter Out Of Range Escape Code error will occur. MaxUserTasks has a value of 16 but can be adjusted if more tasks need to be invoked from the host. Note that only 14 additional tasks can run at one time as two are used to sustain host communication.

Examples

The following resident procedure named UserTask1 performs a variety of moves:



The following list of commands begin the task and then determine when it is finished:



Related Topics

Yield
Abort User Task
Begin User Task

IO Operations

Command Categories

IO Operations read inputs, set outputs, and adjust the controller sample rate which influences the bandwidth of IO activity.

Configure IO Bit As Output

Command

```
CIO <index> <ON or OFF>
```

Description

DIO points can operate as inputs or outputs. The "Input Bit" expression will report the level of the point whether in is being driven internally by an output driver or driven externally by a sensor. However it is necessary to indicate if the driver should be active as an output. This command turns on the driver so that the output voltage can be controlled.

Escapes

A Too Few Parameters Escape Code will be generated if the parameter is less than 1.

Examples

This command list configures DIO 1 as an output and controls it with the Set Output Bit command:

Related Topics

Input Bit
Set Output Bit

Input Bit

Command

```
INB <bit number>
```

Description

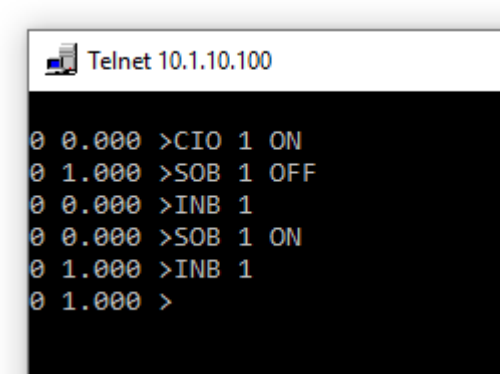
The Input Bit commands returns the state of the input bits referenced by the parameter. A non-zero value represents a high voltage and a 0 represents a low voltage. This commands returns a single bit which is useful for making decisions. However host communication benefits from packed information. Using a controller resident application program it is an option to consolidate input bit status into a single user word if desired.

Escapes

A Parameter Out Of Range Escape Code will be generated if the parameter references an input beyond the range of the controller.

Examples

This list of commands configure DIO 1 as an output, turns DIO 1 off, reads back the value as a input getting a response of 0, and checks the On case as well:



Related Topics

Set Output Bit

Sample Counter

Command

SPC

Description

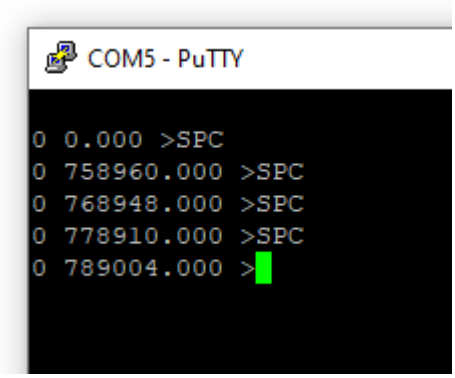
The Sample Counter commands increments each controller sample period and provides a measurement of discrete time in the controller.

Escapes

The Sample Counter commands does not produce any escapes.

Examples

This list of commands, each command separated by about 10 seconds, shows the sample counter incrementing:



Related Topics

Sample Rate

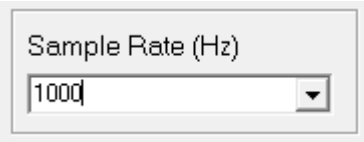
Sample Rate

Command

```
SPR <value in Hz>
```

Description

The Sample Rate commands sets and queries the Sample Rate of the controller. The sample rate is typically 1000 Hz with a discrete time period of 1000 microseconds. Every on-board task runs every sample period. The sample rate can be changed, although it is recommended this value not be changed unless explicitly recommended by technical support. Even though the sample rate can be changed during program execution it is recommended that it be set once during initialization and that all speed and acceleration settings be made after the sample rate has been set. Changing the sample rate after Speed and Accel settings have been established can result in distorted speeds and accels. To avoid this issue it is best and simplest to choose the sample rate from the drop-down list on the Setup Tab:

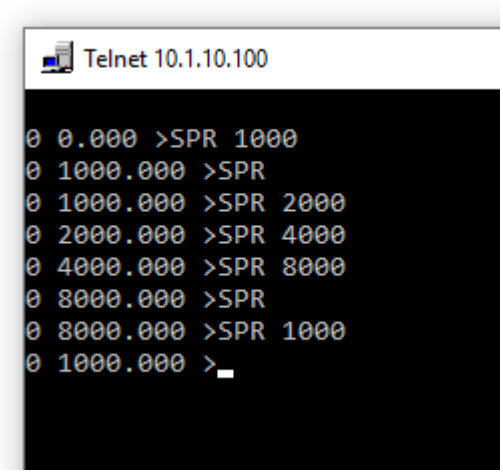


Escapes

If the sample rate is set to greater than 8 kHz a Parameter Out Of Range Escape Code will occur.

Examples

The following list of commands sets, and inspects, the controller sample rate:



Related Topics

Sample Counter
Parameter Out Of Range Escape Code

Set Output Bit

Command

```
SOB <bit number> <ON or OFF>
```

Description

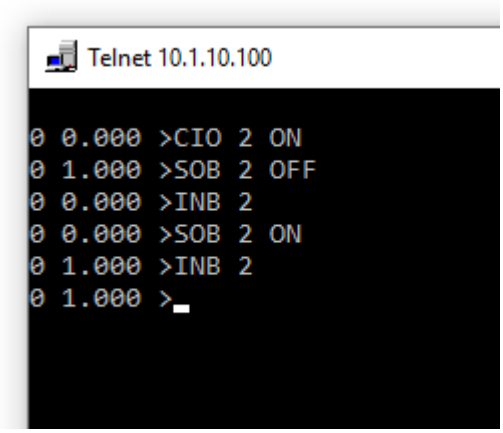
The Set Output commands establishes the voltage of an IO point that has been configured as an output. The "On" value switches the output to VLogic. "Off" makes the driver inactive similar to a disconnected condition. If an Input
block is used on an output, the Input block will be based on the voltage condition of the point regardless of that condition is created by the output driver or an external sensor.

Escapes

If the value added to OutputBit_ creates a sum outside the available output range an "event not defined" (3001) will be reported.

Examples

This list of commands configure an output bit as an output, set the output on and off, and reads back the value with the input bit command:



Related Topics

Input Bit

Safety

Command Categories

Safety commands relate to reporting errors, detecting disabling signals, and controlling communication.

Reset Watchdog

Command

RWD

Description

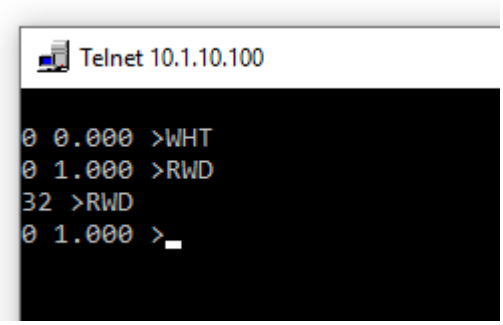
The Reset Watchdog commands restores the controller to an operating condition from a latched Watchdog Has Tripped shutdown condition. If the conditions that caused the watchdog to trip are not remedied before Reset Watchdog is used then Reset Watchdog will escape. The Initialize procedure in the standard.inc library calls Reset Watchdog when the program first starts. There should only be a need to use this command if the EStop signal can be tripped during machine operation.

Escapes

If the conditions that caused the watchdog to trip are not remedied, ResetWatchdog will generate a Watchdog Failed To Reset Escape Code.

Examples

This command list shows that the watchdog has been tripped by an EStop condition. An attempt to reset the watchdog fails because the EStop has not been satisfied. After restoring the EStop circuit the reset watchdog command succeeds.



Related Topics

Watchdog Has Tripped
User Has Disabled

User Has Disabled

Command

UHD

Description

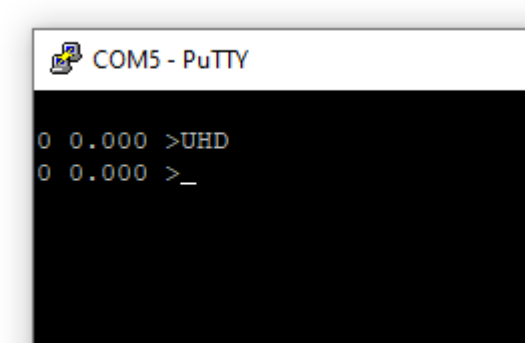
The User Has Disabled commands returns a non-zero true value if the system is being disabled by an external input. A 0 false value indicates the system can be operated. This signal is the unlatched condition. The latched condition of this signal having been tripped is reflected in the Watchdog commands which requires an explicit reset prior to continuing operation.

Escapes

The User Has Disabled commands does not generate any escapes.

Examples

This command shows that the user has not disabled the motor:



Related Topics

Reset Watchdog
Watchdog Has Tripped
Motor States

Watchdog Has Tripped

Command

`WHT`

Description

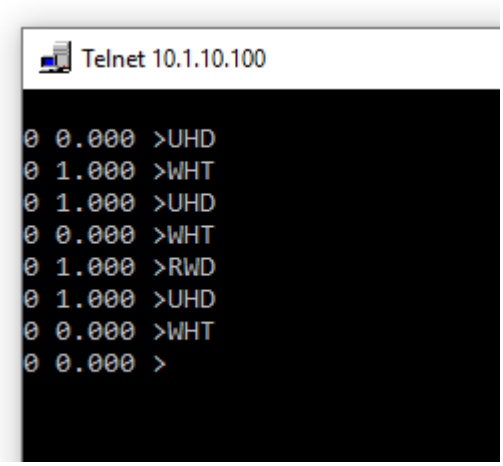
The Watchdog Has Tripped expression indicates the state of the controller's safety system. If tripped then motion is prevented at the hardware level regardless of software commands. Watchdog Has Tripped is a latched condition. One of the conditions that causes the watchdog to trip is firmware inattention. If the real-time operating system supporting the motion application fails to operate properly this hardware tripped condition stops motion. The second cause of a Watchdog Has Tripped condition is the EStop signal not being satisfied. EStop is an input requiring an active, asserted condition to permit motion. If a normally-closed EStop button releases the EStop signal, or if a wire breaks, or an insulation breakdown shorts the wire to a grounded conduit, then the EStop signal will not be satisfied and the hardware shutdown occurs. Restoring motion requires satisfying the EStop signal and using the Reset Watchdog command.

Escapes

If the cause of the watchdog safety system tripping is still present, such as a disable signal remaining active, then the watchdog safety system will fail and generate a Watchdog Failed To Reset escape code.

Examples

This command list uses WHT to understand the state of the safety system:



Related Topics

Reset Watchdog
User Has Disabled

Exception Handling

Math

Console Controls

Motion Concepts

Motion Concepts

This category shows how certain motion concepts can be approached.

User Units

Performing Motion in User Units

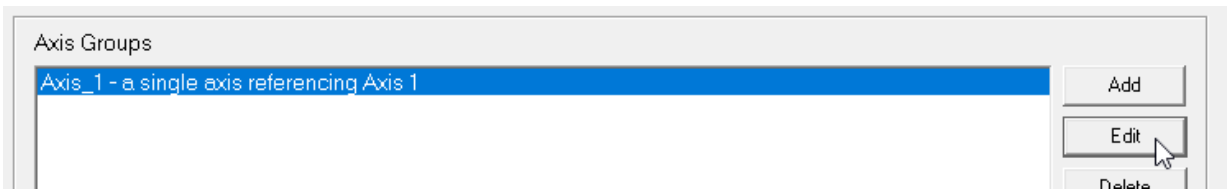
Projects go much smoother when we describe the solution in terms of the problem being solved rather than in the terms of the tools or components being used to solve the problem. User Units allow the description of positions, speeds, and accelerations in desired terms of the application itself rather than in steps, counts, or other units of measure normally used by motors or sensors. With User Units the machine can be told to go to a particular number of inches, or millimeters, or microns or degrees rather than stepper motor steps. Using desired user units requires understanding the total number of steps or counts required to produce a single user unit. This number is influenced by the resolution of the stepper motor or encoder and the effect of the transmissions between motor shaft rotation and resulting movement that is being described.

The first step is to identify what user units should be used. Select units that produce friendly numbers that do not have too many zeros after the decimal point or before the decimal point that are difficult to keep straight. As an example consider the case of describing the rotation of a turntable which is being directly driven by a microstepping stepper motor shaft. Angular degrees would be a good choice.

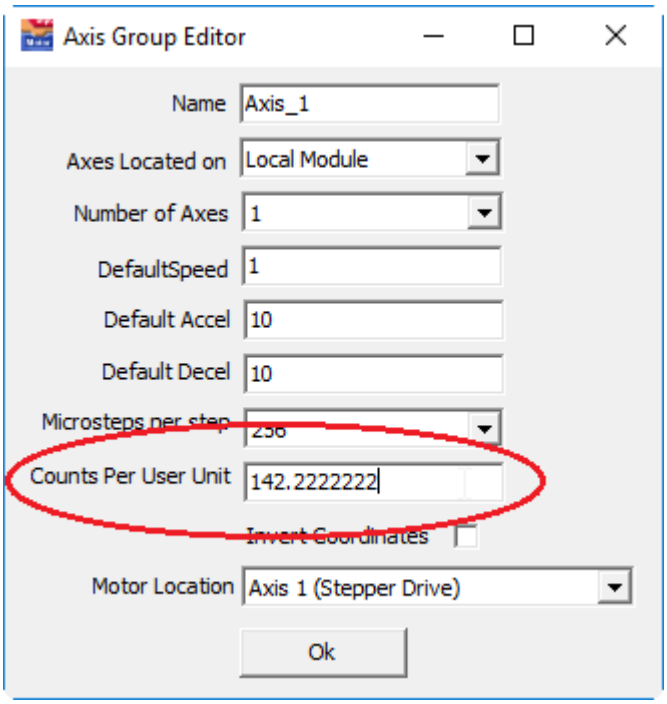
The second step is to consider a displacment of the machine and identify how many fundamental motor units occur during that displacement, and how many user units occur. Dividing the total counts for the displacement by the total user units for the displacement produces "Counts Per User Unit", the number we can use on the Setup Tab to configure an axis. In this example the motor is configured for 256 microsteps per step. This produces 51200 microsteps per motor rotation. In that one rotation there are 360 degrees. So the Counts Per User Unit for this direct drive turntable would be 51200/360 = 142.22222222.

To use this value click on the "Setup Tab":

Select the axis you want to configure from the Axis Group list and click the "Edit" button:

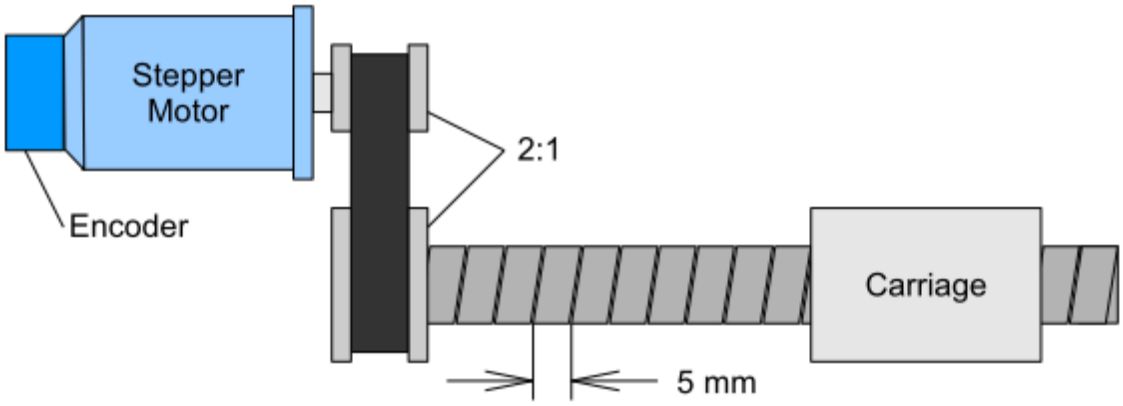


This produces a property editor for the axis that includes a "Counts Per User Unit" field:



Place the calculated value in this field. Note that the default speeds and accels will likely need to be changed in light of the new units.

Consider this more involved case. A stepper motor is driving a leadscrew through a timing belt:



We need to identify the counts per user unit for both the encoder and the stepper motor. First consider the stepper motor. Let's choose one rotation of the motor for the identifying displacement. One motor turn corresponds to 51200 counts (steps). One turn of the motor turns the shaft 1/2 turn. This produces 2.5mm of carriage motion. So the counts per user unit for the stepper axis is 51200/2.5 which is 20480.

For the encoder let's suppose it has 1000 slits. This encoder has 4000 counts per rotation because of the quadature effect. One rotation of the motor produces 4000 counts for the encoder and the same 2.5mm for the carriage. The counts per user unit for the encoder is then 4000/2.5 which is 1600. The stepper motor and

encoder axes are separate items in the Axis Group list on the setup page so they each have their own Counts Per User Units field.

Related Topics

Move By
Move To
Counts Per User Unit

Motor States

Description

Motors can be in the following states:

Not Enabled - In this condition the amplifier is not powered and no command signals are being sent to the motor drive (whether internal or external)

Enabled - The amplifier enable line is active but no position control law is active. If the motor is a servo motor the command signal will only change if the SetCommandedTorque command is used to explicitly change it.

Positioning - The amplifier enable line is active and actions are being taken by the controller to actively manage position.

Most of the time the only two states of interest are not enabled and servoing. The most frequently used command to move between these states is SetMotor. SetMotor(on) accomplishes the servoing state. SetMotor(off) accomplishes the not enabled state. It is also possible to request the enabled but not servoing state for open loop control principles where an application program applies criteria for setting the motor torque seperate from the control law. To accomplish an enabled but not servoing state, SetMotor(off) and then SetEnable(on). If an axis is servoing and the tracking error becomes too large the servo will move to the Not Enabled condition.

Related Topics

Set Motor
Set Enable
Set Error Limit

Homing

Description

Homing is the procedure a machine goes through to establish its coordinate system. Homing is needed if the machine does not have any position sensors or if the position sensors are incremental. Machines using absolute encoders do not home as their position can be determined directly.

The process involves moving each machine axis in a manner so as to find a machine feature with a know position for that axis. The feature might be a homing sensor or it might be a mechanical interference such as a hard stop. It is necessary to think through the homing process to insure that the machine is always moving in a way that prevents collisions. For example, a descending transfer machine will likely need to first home the vertical axis upwards and out of the way of material below the machine. Once the machine is up then it has freedom to move laterally without a collision threat. Homing must avoid collisions from any possible initial condition. If this is not possible absolute encoders may be required.

Homing To A Sensor

A series of examples will be shown from simplest to methods that offer higher performance.

Related Topics

Positive Limit
Negative Limit
CapturePosition

Escape Codes

Escape Codes Introduction

Escapes, part of the structured exception management system, occur when some error takes place. When this happens, the program "raises an exception" or "escapes" and program execution stops what it is doing and travels up through the nested levels of current procedures and functions seeking a Try Recover block that will handle the escape. If no application recover block is encountered the escape will travel all the way up to a default escape handler which displays an error message. Each escape has a unique value referenced by the symbolic names listed in this section.

10 Divide Overflow Escape Code

Description

Attempts to divide by 0 should result in this escape code being generated.

Related Topics

Escape Codes

11 Sample Overrun Escape Code

Description

This escape occurs when a CheckTask is used to confirm that a task has not taken too much time and executed beyond the end of the sample period. If the task has gone too long this escapecode results.

Related Topics

Escape Codes

12 Axis Not Valid Escape Code

Description

This escape gets generated when a parameter has been provided to an TNAxis initialization routine but the axis is not a legitimate T1Axis.

Related Topics

Escape Codes

13 Speed Negative Escape Code

Description

The speed for a motion profile should always be expressed as a positive number or 0 (if you want to suspend a move). You can accomplish negative velocities by jogging with a negative parameter or moving in the negative direction however the speed parameter specified with Set Speed should always be positive.

Related Topics

Escape Codes

15 Curve Buffer Not Linked Escape Code

Description

This escape generally occurs when you ask an axis to to perform curve related operations when no vector buffer has been provided. This most likely indicates forgetting to use the LinkTo command to attach to a TNAxis group an appropriate array.

Related Topics

Escape Codes

16 User Accels 0 Or Negative Escape Code

Description

The parameter to Set Accel must be positive. If the acceleration is 0 motion would take forever because the motors would never accelerate up to speed.

Related Topics

Escape Codes

17 User Decells 0 Or Negative Escape Code

Description

Even though an axis decelerates with a negative acceleration, the decel value itself is expressed as a positive number. The parameter to Set Decel should always be expressed as a positive deceleration.

Related Topics

Escape Codes

19 Unable To Resume Task Escape Code

Description

If a task is suspended it should be possible to resume the task. However there are some cases where a task you ask to resume cannot be resumed. One case is asking for a task to resume which was never present to begin with. Another possibility is that even though the task was suspended some other agent resumed it, i.e. another task, causing the suspended task to finish execution before you got around to asking it to resume.

Related Topics

Escape Codes

20 Unable To Begin Task Escape Code

Description

This escape code will generally occur when the maximum number of tasks are already running. If you request more tasks than the system supports you will not be able to start the requested task.

Related Topics

Escape Codes

21 Append Move Overflow Escape Code

Description

The number of segments that can be appended to a continuous path move is limited by the size of the array provided to the TNAxis group. If you attempt to put more segments into the array than it can hold then you overflow the array.

Related Topics

Escape Codes

23 Motion Overrun Escape Code

Description

This escape occurs when a directive is given to a move in progress which is not possible. Examples include moving to where you currently are, ie instantaneously stopping while still maintaining the decel specification. Moving to a destination "behind" the current axis while moving in a particular direction may result in this escape.

Related Topics

Escape Codes

24 Axis Is Busy Escape Code

Description

This escape generally occurs when you ask an axis to to do something which is already owned by another group.

Related Topics

Escape Codes

25 File Reset Escape Code

Description

If a TFile object attempts to reset (or open) a file, and is not able to find the file this escape will occur.

Related Topics

Escape Codes

26 File Rewrite Escape Code

Description

This escape will occur when a TFile attempts to create a new file but is not able to do so. Reasons could include a path specified by Assign which does not exist or write protection on the destination disk.

Related Topics

Escape Codes

27 Read Escape Code

Description

This escape can occur whenever a read error takes place. The most likely reason for a read error is that the receiving data type cannot hold the type of string that was provided, for example reading a longint from a text editor which does not contain a number will produce a read escape.

Related Topics

Escape Codes

28 Write Escape Code

Description

A Write escape will occur when a write operation is not successful.

Related Topics

Escape Codes

29 Object Not Initialized Escape Code

Description

This escape occurs usually when a TNAxis variable has been declared but has not been initialized or a File operation is performed on a file that has not been assigned but has not been reset or rewritten. When services are asked of this variable this escape occurs indicating that an initialization has not been performed.

Related Topics

Escape Codes

31 Parameter Out Of Range Escape Code

Description

This escape occurs when the parameter to a procedure of function is outside of the legal limits for that parameter. This may be an indication of an uninitialized variable or the absence of bounds checking from a user input. By placing the use of the procedure or function inside a Try Recover block it is possible to detect this escape and prompt the user for a value within bounds, for example.

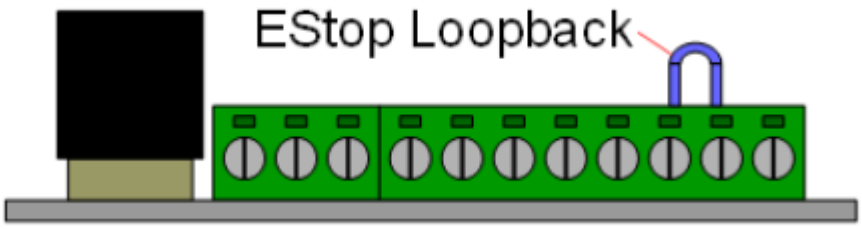
Related Topics

Escape Codes

32 Watchdog Failed To Reset Escape Code

Description

This error occurs when an attempt is made to reset the watchdog safety system and it fails to reset. The safety system requires a signal be present on the Estop input to permit motion and this signal is not present. If the system has an EStop switch confirm that the EStop button is not pushed. If the system is not using an EStop circuit then the EStop input must be satisifed with a loopback. On the left side of the controller is an 8 pin green connector. Place a loopback cable as shown:



If Reset Watchdog is being used without a Try Recover the problem will not be handled by the application. If there is an EStop button being used with the EStop input use a try recover block to catch the error and indicate to the operator the need to release the estop.

Related Topics

Escape Codes

33 Option Not Present Escape Code

Description

Not all product variations have all the features, for example asking for the capture position from a product configuration which does not have the capture option results in this escape.

Related Topics

Escape Codes

34 IO Authorization Escape Code

Description

This indicates that the communication state is disabled and commands will be disregarded until the communicaton state is enabled.

Related Topics

Escape Codes

35 Name Not Found Escape Code

Description

This escape is returned by the DLL when a name provided for referencing a symbol was not found.

Related Topics

Escape Codes

36 Library Not Found Escape Code

Description

This escape occurs when a program is unable to find the Dynamic Link Library indicated in a procedure or function external clause. Most likely the library is not accessible because of a missing path statement. The \Windows\System directory is a common place to keep a DLL.

Related Topics

Escape Codes

37 DLL Procedure Not Found Escape Code

Description

This escape occurs when a procedure or function cannot be found in the specified DLL. The external clause allows specifying a DLL and assumes the name of the procedure or function in the DLL is the same as the name of the externally declared procedure. Check to make sure the names are the same.

Related Topics

Escape Codes

38 Neighbor Communication Escape Code

Description

This error occurs when a reference is made to an axis or IO resource of a neighbor module and there is no response from the neighbor module. The most common reason for this error is the neighbor module not running the neighbor program which is necessary for it to respond. The presence of the neighbor program can be checked by starting the neighbor in isolation. On power on the yellow light should do a double blink and turn off if the neighbor program is present.

Related Topics

Unresponsive Remote Controller

39 String Error Escape Code

Description

Many string operations are monitored to insure that the expected terminating null character is found and that more information is not being written into a string than it can hold. If a string capacity or format error is detected the result will be a String Error Escape Code. Note that not all possible strings errors that can occur are covered by this checking process.

Related Topics

Escape Codes

40 Floating Point Error Escape Code

Description

This escape occurs when a floating point error is detected using the software floating point routines. These routines come into play when numbers of type single or double are manipulated with conventional infix notation. If you are using the math coprocessor this floating point escape is not used.

Related Topics

Escape Codes

41 Task Stack Overflow Escape Code

Description

Each task is provided with its own "stack", a place to keep track of temporary variables, parameters and procedure return points. Task stacks allow multiple tasks to share the same procedure, for example, and not interfere with each others information. This error has occurred because the task stack has run out of space. The best solution for this type of problem is to make variables global rather than "local", or stack based, by placing the "static" keyword after local variables. Vectors, in particular, consume task stack space because they can be large. This saves task stack space, however if the routine is used by more than one task at the same time, each task will be working with the same global memory. This technique should only be used if just one task is calling a particular procedure. Recursion techniques are permitted. If a recursive procedure does not have a proper termination condition this type of stack overflow error can occur. After experiencing a task stack overflow it is best to save your work, exit from the program, power cycle the controller, and return to the project.

Related Topics

Escape Codes

42 Address Is Nil Escape Code

Description

This escape is reported by routines which take a task address, such as BeginTask, but are given a nil pointer instead. Nil is the return result of TaskAddr when it does not find the name specified. Accordingly, Address Is Nil Escape Code means that the name given to TaskAddr was not found in the symol table. Remember to include prefix path information, such as a plate name, to fully specify the location of a task. This escape code should only be encountered when using the DLL interface. The program detects that the task is not found at compile time rather than run time.

Related Topics

Escape Codes

44 Plate Not Present

Description

This escape occurs when certain operations are performed on a plate which has not been popped up yet. Although the variables and procedures in a popup plate are available even if the plate is not popped up, certain operations such as drawing are not appropriate until it is.

Related Topics

Escape Codes

45 Position Limit Escape Code

Description

This escape occurs when an axis is commanded to move beyond its positive or negative software limits.

Related Topics

Escape Codes

46 Arm Capture Failure Escape Code

Description

This escape occurs when you attempt to arm high speed capture with Arm Capture. The escape occurs because the system could not get a proper synchronized reading from the hardware during setup of the capture. Contact Douloi Automation for assitance.

Related Topics

Escape Codes

47 Task Address Is Not Valid Escape Code

Description

This escape occurs when Begin Task is given a parameter which is not a legal procedure address.

Related Topics

Escape Codes

48 Bitmap File Not Found Escape Code

Description

This escape occurs when plate is asked to draw a bitmap however the bitmap file cannot be found. This most likely is related to the current directory, influenced by the file open dialog, has been changed. Bitmap files can most reliably be found in the filename is provided as an "absolute" path from the root directory of the disk.

Related Topics

Escape Codes

49 Capture Not Present Escape Code

Description

Not all controllers have all features. This controller, or this axis of the controller, does not support the high speed capture feature.

Related Topics

Escape Codes

50 Encoder Position Not Present Escape Code

Description

Not all controllers have all features. This controller, or this axis of the controller, does not support an explicit encoder position in addition to the standard axis actual position.

Related Topics

Escape Codes

51 Dac Number Out Of Range Escape Code

Description

An Digital To Analog Converter is being reference with an index beyond the number of converters available.

Related Topics

Escape Codes

52 Function Not Present Escape Code

Description

A function is being referenced that is not available in this particular controller.

Related Topics

Escape Codes

53 Axis Does Not Support Enable Status Escape Code

Description

Some controller axes are not able to readback enable status from the hardware. A motor may become disabled if an estop event occurs that disables the axis in hardware. Monitor the Watchdog status to determine if a motor may have disabled under hardware direction.

Related Topics

Escape Codes

54 Firmware Read Escape Code

Description

Memory read operations were performed in the firmware region of flash memory. Firmware is in a protected area and cannot be read back.

Related Topics

Escape Codes

55 Ethernet Socket Initialization Failed Escape Code

Description

An attempt to initialize a socket was not successful.

Related Topics

Escape Codes

56 Ethernet Socket Initialized With Port 0 Escape Code

Description

When initializing a socket a non-zero port handle must be used.

Related Topics

Escape Codes

57 Ethernet Socket Already In Use Escape Code

Description

The real-time RTOS executing controller programs is separate from the communication services section of the controller. If the controller program stops, restarts, and attempts to open up a socket that was previously being used the communication section will issue this error since the socket was never disconnected. To avoid this error make sure the client disconnects from the socket prior to termination of the program. Otherwise it is necessary to power cycle the controller before running the program to insure the socket is released.

Related Topics

Escape Codes

58 Ethernet Socket Limit Exceeded Escape Code

Description

Different controllers support various numbers of concurrent open sockets. The MMC family can support 8 open sockets. It may be necessary to reorganize the program to operate with fewer open sockets.

Related Topics

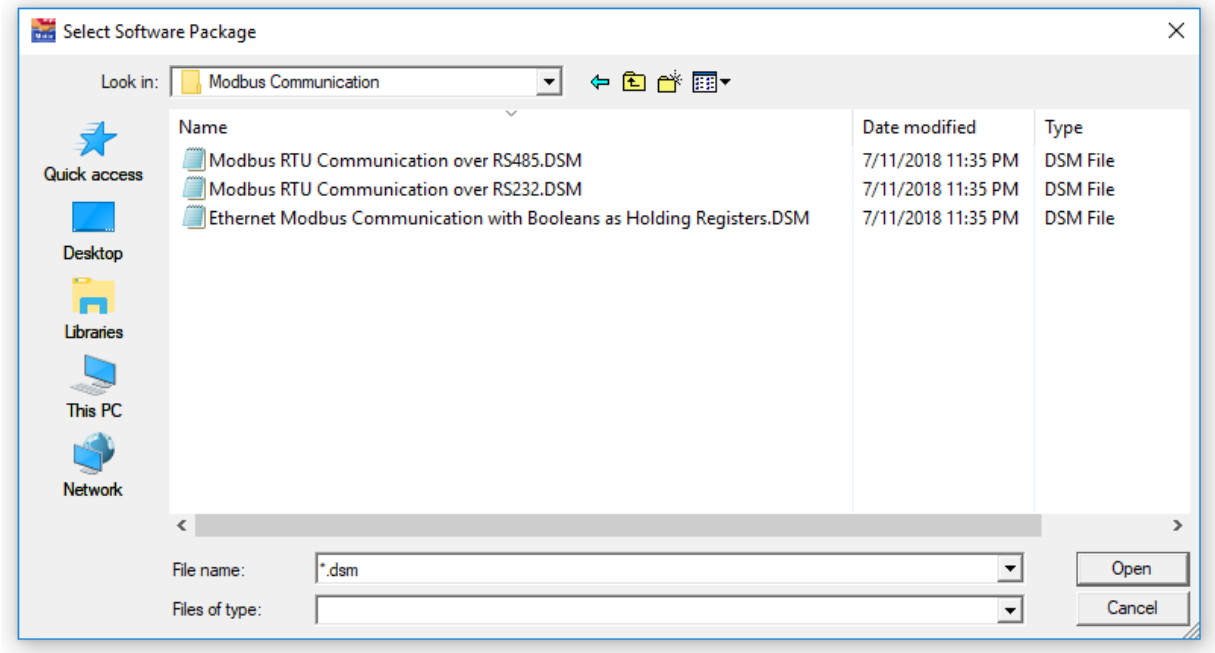
Escape Codes

59 Modbus Communication Package Required Escape Code

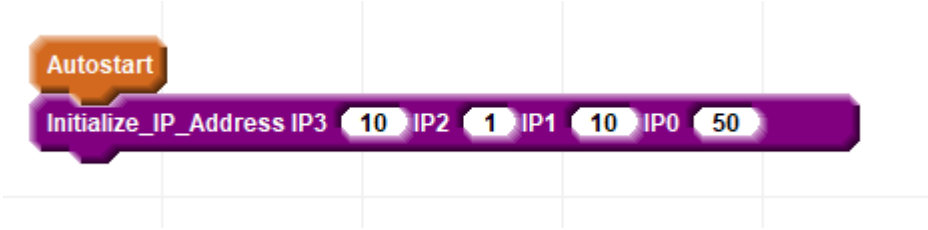
Modbus Communication Package Required

On the Setup Page modbus registers have been described. Now it is necessary to choose the type of Modbus communication that will be used to communicate to the master. On the Console Tab select the Add Software Package Tool:

Open up the Modbus Communication folder and choose a commuication package based from the filename description:



After choosing the package select the Blocks Tab and perform any configuration indicated. This is configuration for the Ethernet choice:



Related Topics

Begin Task
Abort Task
Wait Until Task Is Finished

60 Watchdog Has Tripped Escape Code

Description

This error occurs when an attempt is made to turn on a motor and the watchdog safety system is tripped preventing the motor from turning on. The watchdog safety system is reset when the program first starts but may turn off due to an EStop event or a software overrun. If the watchdog has tripped then use the Reset Watchdog commands. The Watchdog Has Tripped commands can be used to see the condition of the watchdog safety system.

Related Topics

Watchdog Has Tripped
Reset Watchdog

61 Unresponsive Remote Controller Escape Code

Description

This error occurs when a reference is made to an axis or IO resource of a remote controller and there is no response from the remote controller. The most common reason for this error is the remote controller not running the remote program which is necessary for it to respond.

Related Topics

Neighbor Communication Escape Code

1001 Grammar Escape Code

Description

This escape occurs when a command is recognized but contains a mistake that prevents it from being properly interpreted.

Related Topics

Escape Codes

1002 Unknown Command

Description

This escape occurs when a command is not recognized at all. Check the command letters and insure they are correct.

Related Topics

Escape Codes

1003 Axis Number Out Of Bounds

Description

This escape occurs when a an Axis receiver has an index that is less than 1 or greater than the number of axes supported by the controller. Most controllers support 16 axes with the lower axes being physically present and available for use and the remaining axes, up to 16, being virtual.

Related Topics

Escape Codes

1004 Group Number Out Of Bounds

Description

This escape occurs when the index used to identify a unique group of axes is less than 1 or greater than 10. Coordinated groups are indicated with a C <group number>> format. C1 and C5 are legal group numbers. C0 and C11 are not.

Related Topics

Escape Codes

1005 Illegal Dimension

Description

This escape gets generated when group is initialized to a dimension greater than is supported by the controller.

Related Topics

Escape Codes

1006 Group Not Initialized

Description

This escape occurs when a reference is made to a coordinated group that was never initialized. Confirm that the initialization command is being used first.

Related Topics

Escape Codes

1007 Coordinated Group Required

Description

This escape occurs when a command receiver needs to be a coordinated group (i.e. C1) but instead an axis is provided (i.e. A3). Commands producing this error need at least a 2 axis coordinated group.

Related Topics

Escape Codes

1008 Too Few Parameters

Description

This escape occurs when a command is expecting more parameters than are provided. For example, if a 3 axis coordinated group is being directed by a Begin Move By (BMB) command this error will occur if only 2 parameters are provided.

Related Topics

Escape Codes

1009 Too Many Parameters

Description

This escape occurs when a command is expecting fewer parameters than are provided. For example, if a 3 axis coordinated group is being directed by a Begin Move By (BMB) command this error will occur if 4 parameters are provided.

Related Topics

Escape Codes

1010 Assertion

Description

This escape occurs when an internal check in the software finds that something which should never be the case is indeed the case. This represents a structural problem in the software. Contat the factory if you get this escape code.

Related Topics

Escape Codes

1011 Out Of Curve Buffers

Description

Curve Buffers are needed for each coordinated group that is performing continuous path trajectories with arcs and vectors. This error occurs when more groups need buffers than buffers that are available. Contact factory for support to increase the number of buffers.

Related Topics

Escape Codes

1012 Unable To Find Named Procedure

Description

Named Procedures can make management of resident tasks with the Ascii Command Interpreter more readable. However it requires publishing procedure names and modifying the Ascii Commands Program. Contact the fatory if you are encountering this error.

Related Topics

Escape Codes

Deployment

Deployment Introduction

This chapter describes procedures to update the controller and deploy applications so that the development tools involved in constructing an application are not seen by the users of the application.

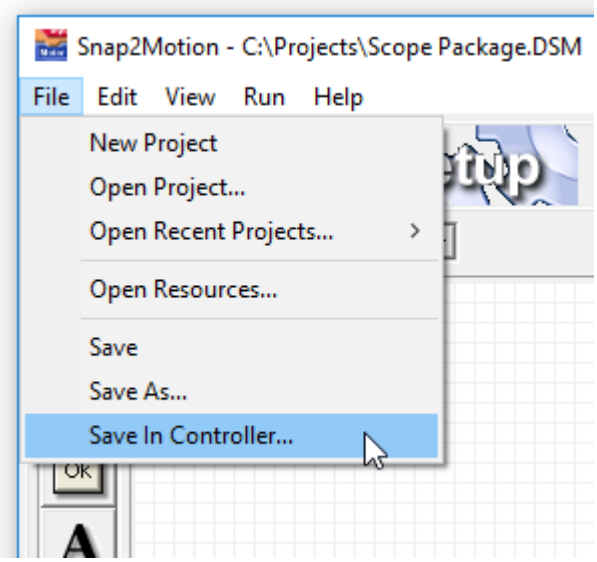
Commissioning

Description

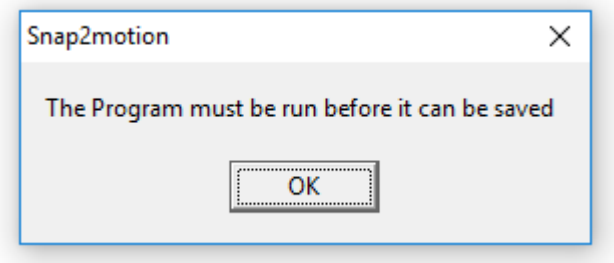
After developing an application it is necessary to commission or deploy the application for use.

Standalone Operation

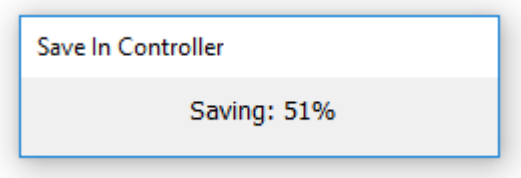
The application can be stored in the controller to run when the controller is powered. To store a project in the controller run the project, press the red stop to stop the project, and then select from the menu "File | Save In Controller":



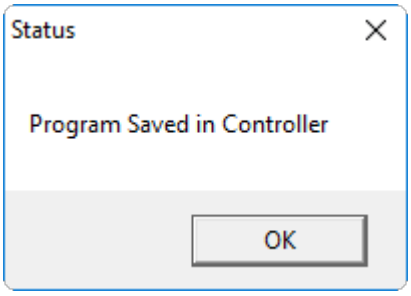
If the project is not run first this message is shown:



Snap2Motion will show file save progress...



...and conclude with this message:

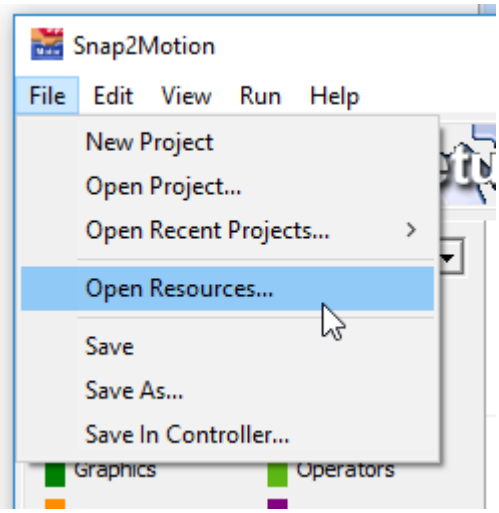


When the controller is powered any Autostart tasks will be performed and then the Main.Setup procedure will be performed (if it has been described). To stop application behavior when the controller is powered on save a blank project into the controller.

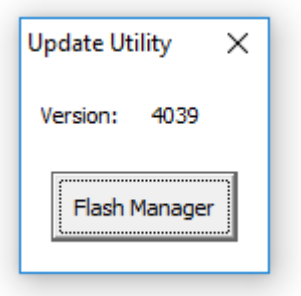
Updating Firmware

Description

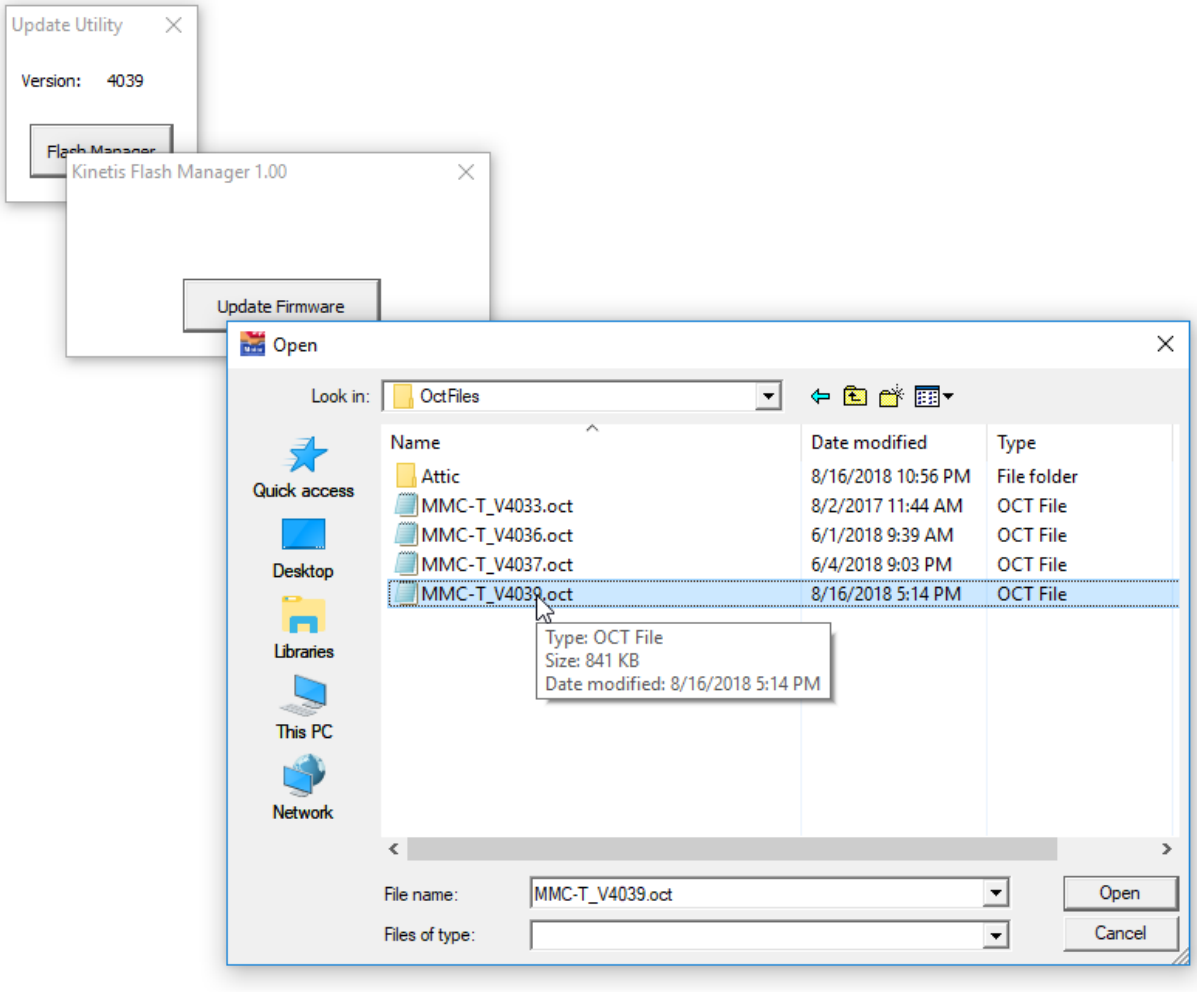
In the course of making controller corrections or providing additional controller features it may be necessary to update the firmware in the controller. This is done with a Snap2Motion utility. Select File | Open Resources:



Navigate to the Utility Program folder and open the Flash Manager. There is a version number following the name Flash Manager. Select the Play button to start the Flash Manager. A small console will appear showing the current version of the controller:



Click on the Flash Manager button, click on the Update Firmware button, and navigate to the OCT (Object Code Text) file provided for the update"



Once chosen the firmware update procedure will proceed taking as much as 10 minutes to finish. At the completion of the process the controller will perform a software reset which will cause Snap2Motion communication to drop-out. This indication that communication has been lost is to be expected during a controller reset.